

Manufacturing Defect Prediction Data Engineering Pipeline

Prepared By

69056012 ชมพูนุช สุนทรพจน์

69056015 ชัยชนะ เพียรประเสริฐ

69056060 วิรัลพัชระ วรรธนะคุณาสิน

Submitted To

Mr. Aekanun Thongtae (AI & Data Science Consultant)

รหัสวิชา: 05177012

ชื่อวิชา: สถาปัตยกรรมข้อมูลและวิศวกรรมข้อมูล

(DATA ARCHITECTURE & DATA ENGINEERING)

หลักสูตรวิทยาศาสตรมหาบัณฑิต สาขาวิชาวิทยาการข้อมูลและการวิเคราะห์

KMITL Digital Analytics and Intelligence Center (K-DAI)

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง (KMITL) ภาคการศึกษาที่ 1/2569

บทคัดย่อ

โครงงานนี้ได้จัดสร้าง Data Pipeline สำหรับ Manufacturing Defect จำนวน 3,240 รายการ 17 คอลัมน์ บน Databricks Free Edition ตามสถาปัตยกรรม Medallion (Bronze ⟶ Silver ⟶ Gold) ชั้น Bronze เก็บข้อมูลดิบจากไฟล์ CSV เป็น STRING พร้อม Ingestion metadata และตรวจ reconciliation ชั้น Silver ทำความสะอาดข้อมูล (มาตรฐานชื่อคอลัมน์ แปลง Placeholder เป็น NULL บังคับชนิดข้อมูล สร้าง record_id กำจัดข้อมูลซ้ำ ตรวจ Data Quality แยก quarantine และติด lineage) โดยพิสูจน์กลไก Data Quality ด้วยชุดข้อมูลสกปรก 15 กรณีที่สร้างขึ้น ผ่านการทดสอบครบ 19/19 รายการ ชั้น Gold แยกข้อมูลเป็น 6 ตารางตาม Business domain ที่เชื่อมกันด้วย Primary Key / Foreign Key แล้วโหลดเข้าทั้ง Delta Lake และ RDBMS คือ Databricks Lakebase (PostgreSQL 17) พร้อมทดสอบ integrity ผ่านครบ 14 รายการ ทั้งหมดทำงานแบบ Automate ด้วย Databricks Workflow ชื่อ Manufacturing _Defect _Pipeline ซึ่งเริ่มเองเมื่อมีไฟล์ใหม่วางใน Landing volume (File arrival trigger) อ่านเฉพาะไฟล์ใหม่ด้วย Auto Loader รัน DQ test เป็นด่านก่อน Silver ส่ง SQL Alert และอีเมลเมื่อมีปัญหา ผลทดสอบแบบ End-to-end ด้วยไฟล์ batch ใหม่ 308 แถว พบว่า Job ทำงานครบ 9 task ใน 22 นาที 33 วินาที แยกแถวผิด 8 แถวเข้า Quarantine ได้ถูกต้อง และส่งอีเมลแจ้งเตือนอัตโนมัติ

ข้อมูลใน Gold ถูกนำไปสร้าง Analytic marts และ Databricks AI/BI Dashboard เพื่อตอบคำถามทางธุรกิจ 9 ข้อ (รวม Unknown Question) โดยแยกตัวแปรเป็น ปัจจัยที่ทีมจัดการได้ (Controllable) และตัววัดผลด้านคุณภาพ (Outcome) และหา Threshold จากจุดเปลี่ยนจริงของข้อมูล ผลการวิเคราะห์พบว่า Batch ร้อยละ 84.04 เป็น High-defect และคิดเป็นร้อยละ 84.38 ของต้นทุนการผลิต ปัจจัยที่จัดการได้ที่มีผลชัดคือ ชั่วโมงซ่อมบำรุงเกิน 10 ชั่วโมงต่อสัปดาห์ (95.6% เทียบกับ 69.9%) และขนาด Batch เกิน 800 หน่วย (96.7% เทียบกับ 80.4%) ส่วน Defect rate (> 3) และ Quality score (< 75) เป็นตัววัดผลจึงใช้เป็น QC alert เมื่อ Batch ไม่มีปัจจัยเสี่ยงและผ่าน QC จะเป็น high-defect เพียงร้อยละ 3.12 นอกจากนี้พบว่ากฎ Threshold ทั้ง 4 ข้อตรงกับ label is_high_defect ถึงร้อยละ 96.11 จึงตั้งเป็น Unknown Question ว่า label นี้นิยามจากกฎ ไม่ใช่การนับของเสียจริง ซึ่งยืนยันได้อีกครั้งจากโมเดล Machine Learning ที่ไม่สามารถทำได้ดีกว่ากฎ 4 ข้อนี้

คำสำคัญ: Data Engineering, ETL, Medallion Architecture, Databricks, Delta Lake, PostgreSQL, Lakebase, Data Quality, Business Intelligence

สารบัญ

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญของโครงการ 1

1.2 ปัญหาที่ต้องการแก้ไข (Problem Statement) 1

1.3 วัตถุประสงค์ 1

1.4 ขอบเขตของโครงการ 2

บทที่ 2 Requirement และภาพรวมระบบ 3

2.1 Project Requirements 3

2.2 Technology Stack 4

2.3 Overall System Architecture 5

บทที่ 3 Data Understanding 6

3.1 Dataset Description 6

3.2 Data Dictionary 6

3.3 Data Domain 8

บทที่ 4 Bronze Layer 9

4.1 Bronze Architecture 9

4.2 Bronze Data Ingestion Bronze_Predictive_v2 10

4.3 Initial Data Profiling 11

4.4 Bronze Table Structure 11

4.5 Incremental Ingestion ด้วย Auto Loader 11

บทที่ 5 Silver Layer – Data Transformation & Quality 13

5.1 Silver ETL Flow 13

5.2 Column Name Standardization 14

5.3 Missing Value Handling 14

5.4 Data Type Enforcement 14

5.5 Deduplication 15

5.6 Data Quality Rules 15

5.7 Business Rule Warning 15

5.8 Quarantine Process 16

5.9 Lineage....16
5.10 Silver Data Quality Gate....16
5.11 Silver Output....17
บทที่ 6 Gold Layer – RDBMS Transformation....18
6.1 Gold Layer Objective....18
6.2 Split Silver into 6 DataFrames....18
6.3 Data Domain Mapping....18
6.4 RDBMS Design (ER Diagram)....19
6.5 Primary Key....20
6.6 Foreign Key....20
6.7 PK/FK Integrity Validation....20
6.8 Gold Table Schema และ Analytic Marts....21
6.8.1 PostgreSQL (Databricks Lakebase)....22
บทที่ 7 ETL Process Implementation....23
7.1 ETL Workflow....23
7.2 ETL Execution....23
7.3 Manual / Automated Execution....25
7.4 ETL Monitoring....29
บทที่ 8 Business Intelligence / Dashboard....31
8.1 Business Questions....31
8.2 KPI....31
8.3 Dashboard....32
8.4 Business Insight....33
8.5 Actionable Insight....35
8.6 Unknown Question และ Ad-hoc Analysis....36
8.7 Predictive Model (Machine Learning)....37
บทที่ 9 Data Quality & ETL Testing....39
9.1 Negative Test ใน PostgreSQL....39
9.2 Final Row Count Reconciliation....40

9.3 Dirty Data Test (ทดสอบกลไก Data Quality) 40
9.4 End-to-End Automation Test 42
บทที่ 10 ผลลัพธ์ของโครงการ 43
บทที่ 11 สรุปและข้อเสนอแนะ 45
11.1 สรุปผล 45
11.2 ปัญหาและข้อจำกัด 45
11.3 แนวทางในอนาคต 46
ภาคผนวก 47
Appendix A: Bronze Code 47
Bronze_Predictive_v2.py 47
Appendix B: Silver Transformation Code 51
Silver_Predictive_v2.py 51
Appendix C–E: Gold Transformation, PK/FK และ SQL Code 59
Gold_Predictive.py 59
Appendix F: Dashboard Code และ Screenshots 73
Dashboard_Predictive.py 73
Appendix G: DQ Test Code 78
DQ_Test_Dirty_Data.py 78
Appendix H: Automation Code 82
Bronze_AutoLoader.py 82
Setup_Workflow.py 84
Demo_Drop_New_Batch.py 88
Reset_Demo.py 89
Appendix I: ML_Predictive Code 92
ML_Predictive.py 92

สารบัญรูปภาพ

ภาพที่ 1: ปัญหาที่ต้องการแก้ไข (Problem Statement)....1
ภาพที่ 2: Technology Stack ของโครงงาน....4
ภาพที่ 3: Overall Architecture: Read CSV ⟶ Bronze ⟶ Silver ⟶ Gold ⟶ RDBMS ⟶ BI ⟶ Insight....5
ภาพที่ 4: Flow แบบย่อพร้อมปลายทางสองทางของ Gold (Delta marts และ Lakebase PostgreSQL)....5
ภาพที่ 5: การแบ่งกลุ่มคอลัมน์ตาม Business Domain....8
ภาพที่ 6: Bronze Flow ใน notebook Bronze_Predictive_v2....9
ภาพที่ 7: Bronze Table Structure....11
ภาพที่ 8: Silver ETL Flow ใน Silver_Predictive_v2....13
ภาพที่ 9: Data Type Enforcement....14
ภาพที่ 10: Quarantine Flow: Invalid ⟶ Quarantine, Valid ⟶ Silver....16
ภาพที่ 11: Silver Output....17
ภาพที่ 12: การแยก Silver เป็น 6 ตารางใน Gold....18
ภาพที่ 13: ER Diagram ของ Gold (PostgreSQL schema gold)....19
ภาพที่ 14: ขั้นตอนตรวจ PK/FK และ negative test....20
ภาพที่ 15: ETL Workflow: Extract ⟶ Transform ⟶ Validate ⟶ Load ⟶ Serve....23
ภาพที่ 16: Flow การทำงานทั้งหมดที่เกิดขึ้นจริง (A ตรวจของเดิม ถึง G Automation ด้วย Databricks Workflow)....24
ภาพที่ 17: โครงสร้าง Job Manufacturing_Defect_Pipeline (v2: dq_test รันขนานกับ bronze_autoloader)....25
ภาพที่ 18: Job run จริง (run 853932781211303) ทุก task Succeeded — รอบนี้ยังเป็นโครงแบบ v1 ที่ dq_test รันก่อน bronze_autoloader....27
ภาพที่ 19: Timeline ของ run เดียวกัน: เริ่ม 10:13 จบ 10:36 รวม 22 นาที 33 วินาที (Launched: By file arrival)....27
ภาพที่ 20: ETL Monitoring....29
ภาพที่ 21: อีเมลจาก SQL Alert "Quality gate failed" (TRIGGERED) หลังรับ batch_002 ที่มีแถวเข้า quarantine 8 แถว....30
ภาพที่ 22: อีเมลแจ้ง Job ล้ม (run 231279394655931) เมื่อ alert task ล้มจาก SQL Warehouse หยุดกลางคัน....30
ภาพที่ 23: Dashboard ส่วนบน: KPI, Controllable vs Outcome, Risk segment, Maintenance hours และ Production volume....32
ภาพที่ 24: Dashboard ส่วนล่าง: QC monitoring, Risk matrix, Operating window, Unknown Question และ Key insights....33
ภาพที่ 25: Data ⟶ Pattern ⟶ Insight ⟶ Business Meaning ⟶ Action....33
ภาพที่ 26: Dashboard หลัง Job ประมวลผล batch_002: total batches 3.54K (ตัวเลขทั้งหมดคำนวณใหม่อัตโนมัติ).42
ภาพที่ 27: Dashboard (ส่วนบน)....76
ภาพที่ 28: Dashboard (ส่วนล่าง)....77

สารบัญตาราง

ตารางที่ 1: การตอบโจทย์ Requirement ของโครงงาน 3
ตารางที่ 2: Data Dictionary (ชื่อเดิม ⟶ ชื่อใน Silver/Gold) 6
ตารางที่ 3: ผลตรวจ notebook Bronze_Predictive เดิม และสิ่งที่แก้ใน v2 10
ตารางที่ 4: ผล profiling และ reconciliation ของ Bronze 11
ตารางที่ 5: เปรียบเทียบ Bronze แบบ batch และแบบ incremental 12
ตารางที่ 6: Data Quality Rules (Hard error ⟶ Quarantine) 15
ตารางที่ 7: Gold Table – Business Domain – Columns 18
ตารางที่ 8: ผลตรวจ PK/FK ใน Spark ก่อนเขียน Gold 20
ตารางที่ 9: ตารางใน predictive_manufacring_defect.gold_predictive 21
ตารางที่ 10: การ map ขั้น ETL กับระบบ 23
ตารางที่ 11: Task ใน Job Manufacturing_Defect_Pipeline 25
ตารางที่ 12: เวลาที่ใช้ของแต่ละ task (รอบ demo และรอบ reset) 28
ตารางที่ 13: ตาราง log สำหรับ monitoring 29
ตารางที่ 14: KPI จาก gold_predictive.kpi_overview 31
ตารางที่ 15: บทบาทของตัวแปร: Controllable vs Outcome 33
ตารางที่ 16: ผลของแต่ละตัวแปรต่อ high-defect % ที่ threshold ดีที่สุด (mart_threshold_scan, mart_driver_impact) 34
ตารางที่ 17: High-defect % ตาม band ของตัวแปรหลัก (ขอบ band ตรงจุดเปลี่ยน) 34
ตารางที่ 18: Controllable risk segment (maintenance > 10, volume > 800) × QC alert (defect rate > 3 หรือ quality score < 75) 35
ตารางที่ 19: Insight ⟶ Business Meaning ⟶ Action 35
ตารางที่ 20: กฎ threshold เทียบกับ label is_high_defect (mart_label_rule, vw_label_rule_check) 36
ตารางที่ 21: การตั้งค่าการทดลอง 37
ตารางที่ 22: ผลบน test set 648 batch (ML_SUMMARY จาก run ใน Databricks) 38
ตารางที่ 23: สรุปผลการทดสอบ 39
ตารางที่ 24: Negative test: ฐานข้อมูลต้องปฏิเสธข้อมูลผิด 39
ตารางที่ 25: จำนวนแถวแต่ละขั้น 40
ตารางที่ 26: ผลทดสอบกลไก Data Quality ด้วยข้อมูลสกปรก (dq_test.test_results) 40
ตารางที่ 27: ผลทดสอบ end-to-end ด้วยไฟล์ batch_002 (308 แถว) 42
ตารางที่ 28: สิ่งที่ส่งมอบ 43

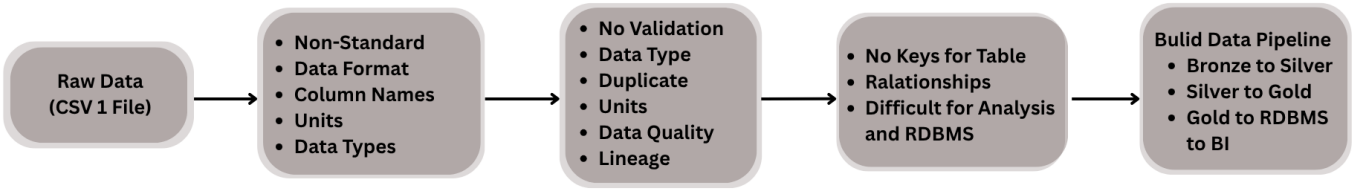
บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของโครงการ

องค์กรการผลิตมีข้อมูลที่เกี่ยวข้องกับคุณภาพสินค้าจากหลายมิติ ได้แก่ การผลิต (Production) ห่วงโซ่อุปทาน (Supply Chain) คุณภาพ (Quality) การซ่อมบำรุง (Maintenance) สินค้าคงคลัง (Inventory) แรงงานและความปลอดภัย (Workforce) พลังงาน (Energy) และการผลิตแบบ Additive Manufacturing ข้อมูลเหล่านี้อยู่รวมกันในไฟล์เดียวแบบ flat และยังไม่พร้อมสำหรับการวิเคราะห์หรือการใช้งานในระบบฐานข้อมูลเชิงสัมพันธ์

โครงงานนี้จึงสร้าง ETL Pipeline ที่นำข้อมูลดิบผ่านการจัดเก็บ ทำความสะอาด ตรวจคุณภาพ และจัดรูปแบบเป็นโครงสร้าง RDBMS เพื่อใช้ทำ Dashboard และหา Business Insight ว่าปัจจัยใดสัมพันธ์กับการเกิด defect สูง

1.2 ปัญหาที่ต้องการแก้ไข (Problem Statement)



ภาพที่ 1: ปัญหาที่ต้องการแก้ไข (Problem Statement)

1.3 วัตถุประสงค์

1. สร้าง ETL Pipeline บน Databricks ด้วย PySpark และ Delta Lake
2. จัดการข้อมูลผ่าน Bronze / Silver / Gold ตามหลัก Medallion Architecture
3. ตรวจคุณภาพข้อมูลด้วย Data Quality rules, quarantine, lineage และ quality gate พร้อมพิสูจน์กลไกด้วยชุดข้อมูลสกปรก
4. สร้างโครงสร้าง RDBMS 6 ตารางพร้อม Primary Key / Foreign Key บน PostgreSQL (Databricks Lakebase)
5. เตรียมข้อมูลสำหรับ Dashboard และสรุป Business Insight ที่แยกปัจจัยที่จัดการได้ออกจากตัววัดผลลัพธ์ ที่นำไปใช้ตัดสินใจได้
6. ทำ ETL แบบ Automate ด้วย Databricks Workflow ให้ pipeline เริ่มเองเมื่อมีไฟล์ใหม่ อ่านเฉพาะไฟล์ใหม่ (Auto Loader) มีด่านตรวจคุณภาพ และแจ้งเตือนอัตโนมัติ

7. สร้างโมเดลทำนาย high-defect จาก Gold ด้วย MLflow และเทียบกับกฎ threshold เพื่อตรวจคำตอบของ Unknown Question

1.4 ขอบเขตของโครงการ

- ข้อมูล: manufacturing_defect_dataset.csv 3,240 แถว 17 คอลัมน์ (16 features + target Defect Status)
- แพลตฟอร์ม: Databricks Free Edition (Serverless notebook / job, SQL Warehouse, Unity Catalog, Workflows, SQL Alerts)
- เทคโนโลยี: Apache Spark / PySpark, Delta Lake, Auto Loader, Databricks Lakebase (PostgreSQL 17), Databricks SDK / Jobs API 2.2, AI/BI Dashboard, MLflow และ scikit-learn
- Gold layer: 6 ตาราง RDBMS + 10 analytic marts (รวม batch_360 และ kpi_overview) + 5 PostgreSQL views
- Notebook ที่ส่งมอบ: Bronze_Predictive_v2, Bronze_AutoLoader, Silver_Predictive_v2, Gold_Predictive, Dashboard_Predictive, ML_Predictive, DQ_Test_Dirty_Data และชุด Automation คือ Setup_Workflow, Demo_Drop_New_Batch และ Reset_Demo

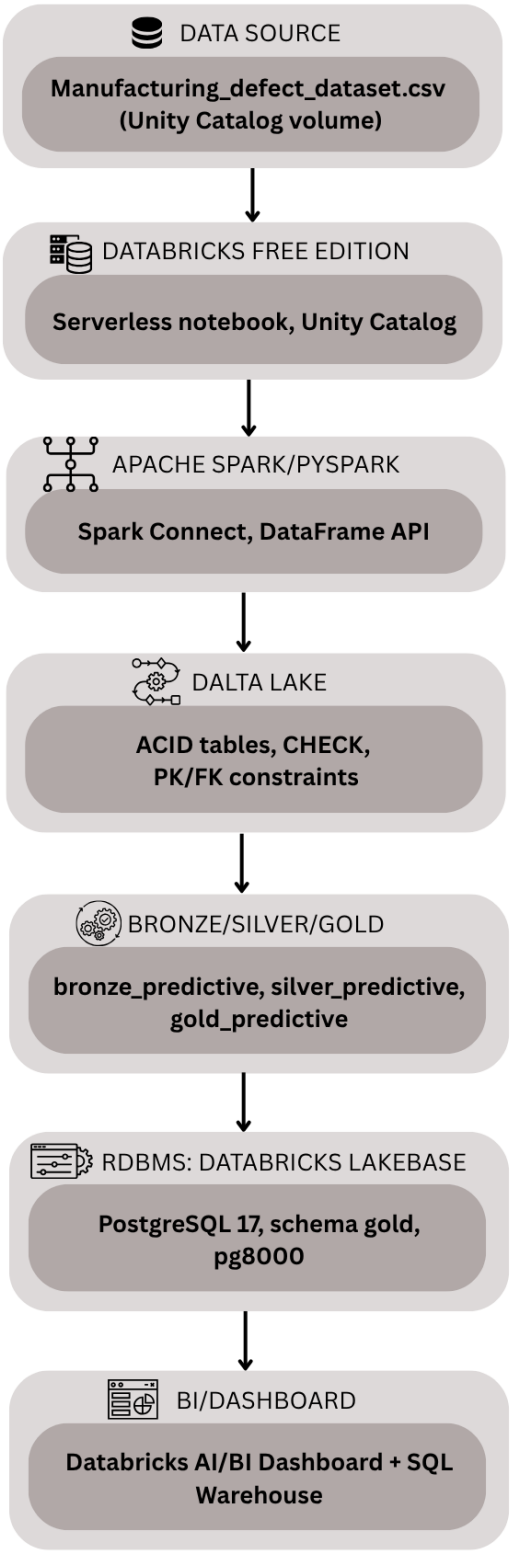
บทที่ 2 Requirement และภาพรวมระบบ

2.1 Project Requirements

ตารางที่ 1: การตอบโจทย์ Requirement ของโครงงาน

Requirement	**วิธีที่โครงการตอบโจทย์**
RDBMS / NoSQL	ใช้ RDBMS จริงคือ Databricks Lakebase (PostgreSQL 17) schema gold 6 ตาราง พร้อม PK/FK/CHECK และลงทะเบียนเป็น catalog manufacturing_gold_pg ใน Unity Catalog
ETL Process	PySpark บน Databricks notebook (Bronze ⟶ Silver ⟶ Gold ⟶ Dashboard ⟶ ML) ทำงานแบบ Automate ด้วย Databricks Workflow: file arrival trigger, Auto Loader, condition task, DQ test เป็นด่าน, SQL Alert 2 ตัว และอีเมลเมื่อ task ล้ม (ยังสั่ง Run all แบบ Manual ได้)
Data Transformation	Landing (CSV) ⟶ Bronze (Auto Loader, incremental) ⟶ Silver ⟶ Gold (Delta) ⟶ PostgreSQL
Business Usage	Databricks AI/BI Dashboard 17 widgets + Business Question 9 ข้อ (รวม Unknown Question) แยก Controllable / Outcome และหน้า Model จากโมเดลทำนาย High defect
Data Quality	DQ rules 30+ ข้อ, quarantine table, Delta CHECK constraints, quality gate, reconciliation และทดสอบกลไกด้วยข้อมูลสกปรก 15 กรณี (19/19 PASS)
Data Relationship	PK production_id ใน production และ FK production_id ใน 5 ตารางลูก (1:1)
Automation (โบนัส)	Job Manufacturing_Defect_Pipeline สร้างด้วยโค้ด (Setup_Workflow) 9 task ทดสอบจริงด้วยไฟล์ batch ใหม่: Job เริ่มเองจากไฟล์ที่วาง รันครบ 22 นาที 33 วินาที และส่งอีเมลแจ้งเตือน (บทที่ 7.3 และ 9.4)

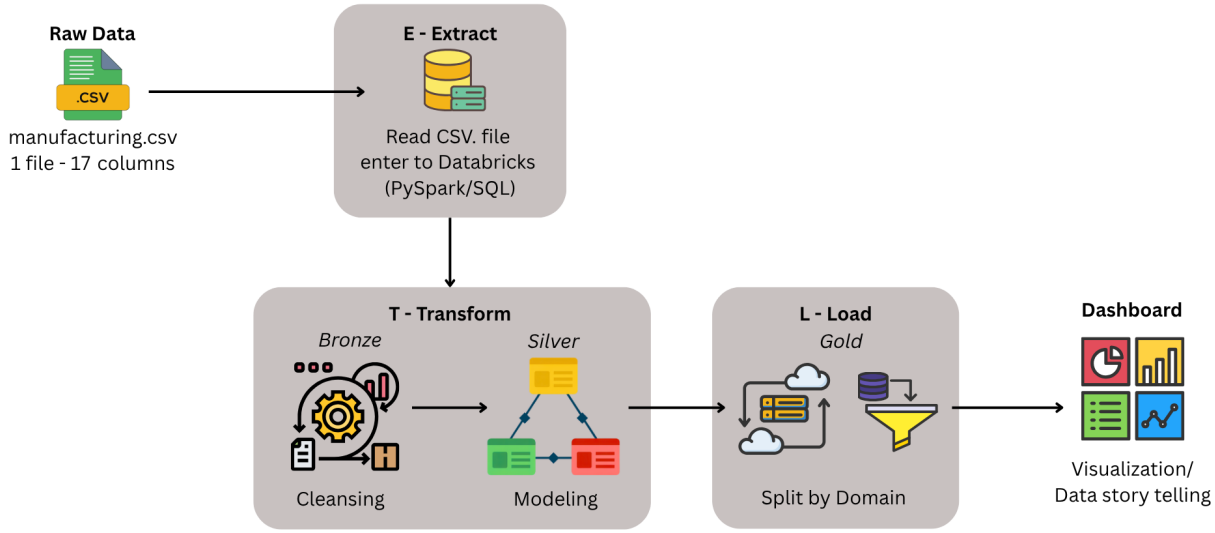
2.2 Technology Stack



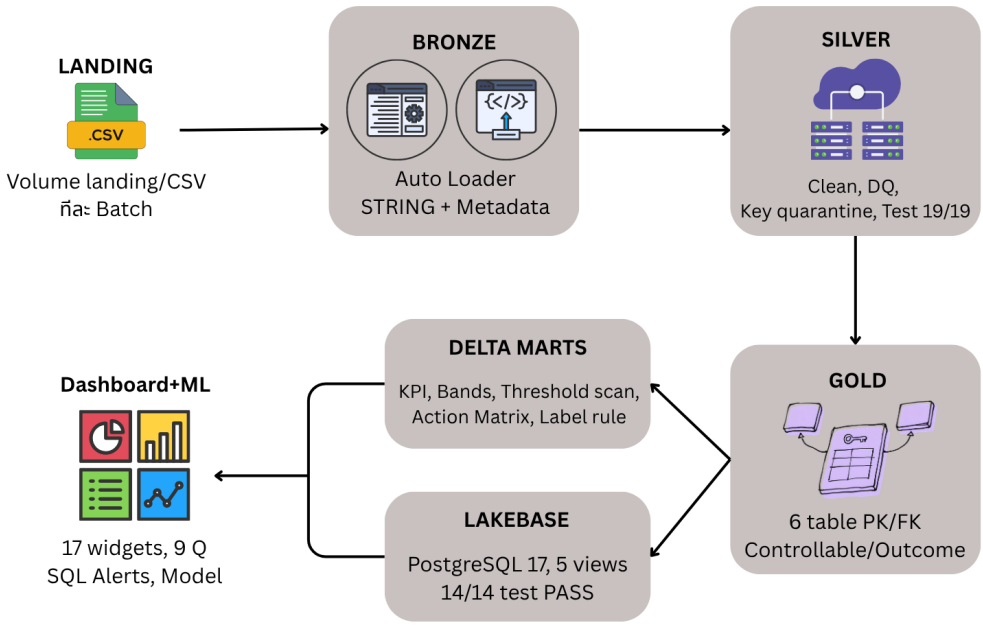
ภาพที่ 2: Technology Stack ของโครงงาน

2.3 Overall System Architecture

ภาพรวมระบบเริ่มจากไฟล์ CSV ที่วางใน landing volume ของ Unity Catalog ซึ่ง Auto Loader อ่านเข้า Bronze เฉพาะไฟล์ใหม่ จากนั้นผ่าน Silver และ Gold แล้ว Gold ถูกโหลดเข้า RDBMS (Lakebase) ใช้ใน Dashboard และโมเดลทำนาย เพื่อสรุปเป็น Business Insight ทุกขั้นถูกร้อยเป็น Databricks Workflow ที่ทำงานเองเมื่อมีไฟล์ใหม่



ภาพที่ 3: Overall Architecture: Read CSV ⟶ Bronze ⟶ Silver ⟶ Gold ⟶ RDBMS ⟶ BI ⟶ Insight



ภาพที่ 4: Flow แบบย่อพร้อมปลายทางสองทางของ Gold (Delta marts และ Lakebase PostgreSQL)

บทที่ 3 Data Understanding

3.1 Dataset Description

Manufacturing Defect Dataset มี 3,240 แถว 17 คอลัมน์ แต่ละแถวแทน 1 production batch ประกอบด้วย 16 ตัวแปรเชิงตัวเลขจาก 8 business domain และตัวแปรเป้าหมาย Defect Status ที่บอกว่า batch นั้นมีระดับ defect สูง (1) หรือต่ำ (0) จากการ profiling ไม่พบค่า missing และไม่พบแถวซ้ำ ตัวแปรเป้าหมายไม่สมดุล: high defect 2,723 แถว (84.04%) และ low defect 517 แถว (15.96%)

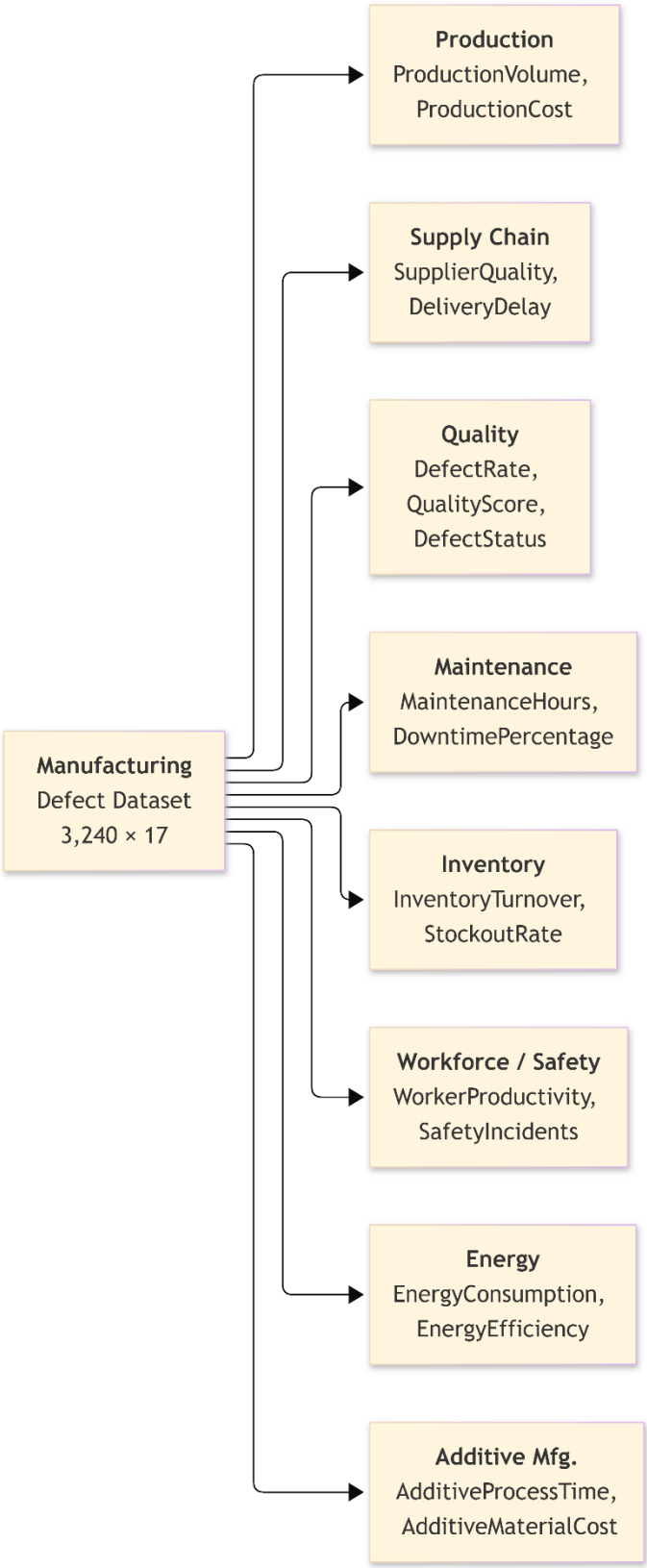
3.2 Data Dictionary

ตารางที่ 2: Data Dictionary (ชื่อเดิม ⟶ ชื่อใน Silver/Gold)

คอลัมน์ต้นทาง ⟶ ชื่อใน Silver	ความหมาย	Type	Domain	ช่วงค่าจริง
ProductionVolume ⟶ production_volume	จำนวนหน่วยที่ผลิตต่อ batch	INT	Production	100 – 999
ProductionCost ⟶ production_cost	ต้นทุนการผลิต (USD)	DOUBLE	Production	5,000 – 19,993
SupplierQuality ⟶ supplier_quality_pct	คะแนนคุณภาพ supplier (%)	DOUBLE	Supply Chain	80.0 – 100.0
Delivery Delay ⟶ delivery_delay_days	ความล่าช้าในการส่งวัตถุดิบ (วัน)	INT	Supply Chain	0 – 5
DefectRate ⟶ defect_rate	อัตราของเสียต่อ 1,000 หน่วย	DOUBLE	Quality	0.50 – 5.00
QualityScore ⟶ quality_score_pct	คะแนนคุณภาพ (%)	DOUBLE	Quality	60.0 – 100.0
MaintenanceHours ⟶ maintenance_hours_per_week	ชั่วโมงซ่อมบำรุงต่อสัปดาห์	INT	Maintenance	0 – 23
DowntimePercentage ⟶ downtime_pct	เวลาหยุดเครื่อง (%)	DOUBLE	Maintenance	0.00 – 5.00
InventoryTurnover ⟶ inventory_turnover_ratio	อัตราหมุนเวียนสินค้าคงคลัง	DOUBLE	Inventory	2.0 – 10.0
StockoutRate ⟶ stockout_rate	สัดส่วนสินค้าขาดสต็อก (0–1)	DOUBLE	Inventory	0.00 – 0.10
WorkerProductivity ⟶ worker_productivity_pct	ผลิตภาพแรงงาน (%)	DOUBLE	Workforce	80.0 – 100.0

คอลัมน์ต้นทาง ⟶ ชื่อใน Silver	ความหมาย	Type	Domain	ช่วงค่าจริง
SafetyIncidents ⟶ safety_incidents_count	จำนวนอุบัติเหตุ	INT	Workforce	0 – 9
EnergyConsumption ⟶ energy_consumption_kwh	พลังงานที่ใช้ (kWh)	DOUBLE	Energy	1,001 – 4,997
EnergyEfficiency ⟶ energy_efficiency_factor	ประสิทธิภาพพลังงาน (0–1)	DOUBLE	Energy	0.10 – 0.50
AdditiveProcessTime ⟶ additive_process_time_hrs	เวลา additive process (ชม.)	DOUBLE	Additive Mfg.	1.0 – 10.0
AdditiveMaterialCost ⟶ additive_material_cost_usd	ต้นทุนวัสดุ additive (USD)	DOUBLE	Additive Mfg.	100 – 500
DefectStatus ⟶ is_high_defect	Target: 1 = high defect, 0 = low defect	INT (0/1)	Quality	0 / 1

3.3 Data Domain

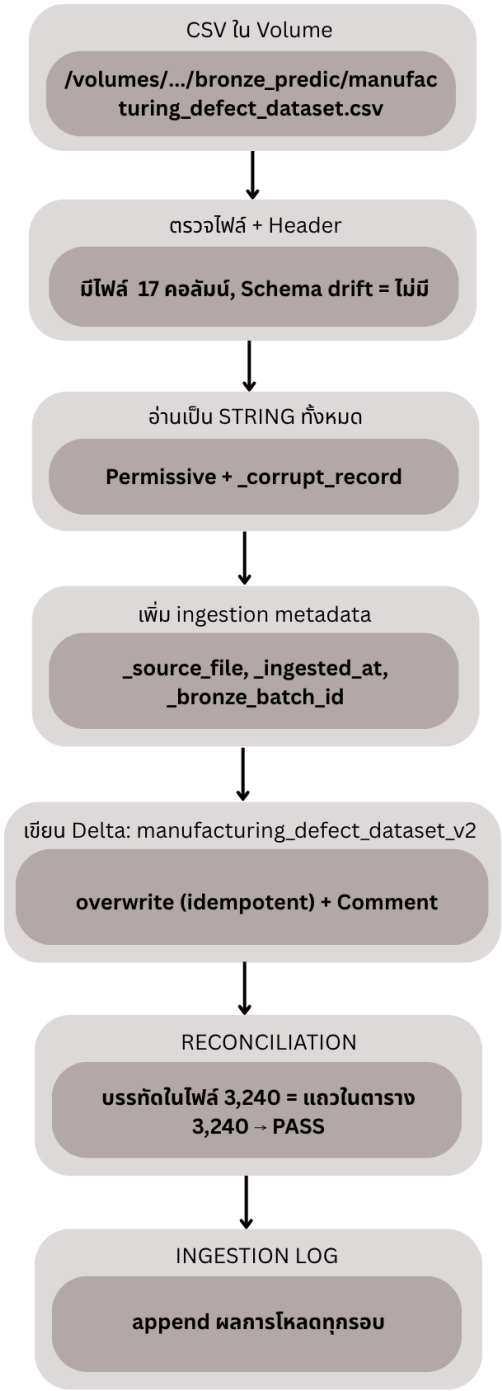


ภาพที่ 5: การแบ่งกลุ่มคอลัมน์ตาม Business Domain

บทที่ 4 Bronze Layer

4.1 Bronze Architecture

Bronze เป็นชั้นเก็บข้อมูลดิบให้เหมือนต้นฉบับมากที่สุด เพื่อให้ย้อนกลับไปตรวจสอบได้เสมอ การตีความชนิดข้อมูลและการทำความสะอาดทั้งหมดถูกย้ายไปทำใน Silver



ภาพที่ 6: Bronze Flow ใน notebook Bronze_Predictive_v2

4.2 Bronze Data Ingestion Bronze_Predictive_v2

ไฟล์ CSV ถูกอัปโหลดไว้ที่ /Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/notebook Bronze_Predictive_v2 อ่านไฟล์ครั้งเดียวด้วย schema ที่ประกาศชัดเจนให้ทุกคอลัมน์เป็น STRING ใช้ mode PERMISSIVE เก็บแถวที่อ่านไม่ได้ใน _corrupt_record แล้วเขียนเป็น Delta table แบบ overwrite ซึ่งรันซ้ำได้โดยข้อมูลไม่ซ้ำ notebook นี้ใช้สำหรับการรันแบบ Manual และการทดสอบ DQ ส่วนการรันแบบ Automate ใช้ Bronze_AutoLoader (หัวข้อ 4.5)

ตารางที่ 3: ผลตรวจ notebook Bronze_Predictive เดิม และสิ่งที่แก้ใน v2

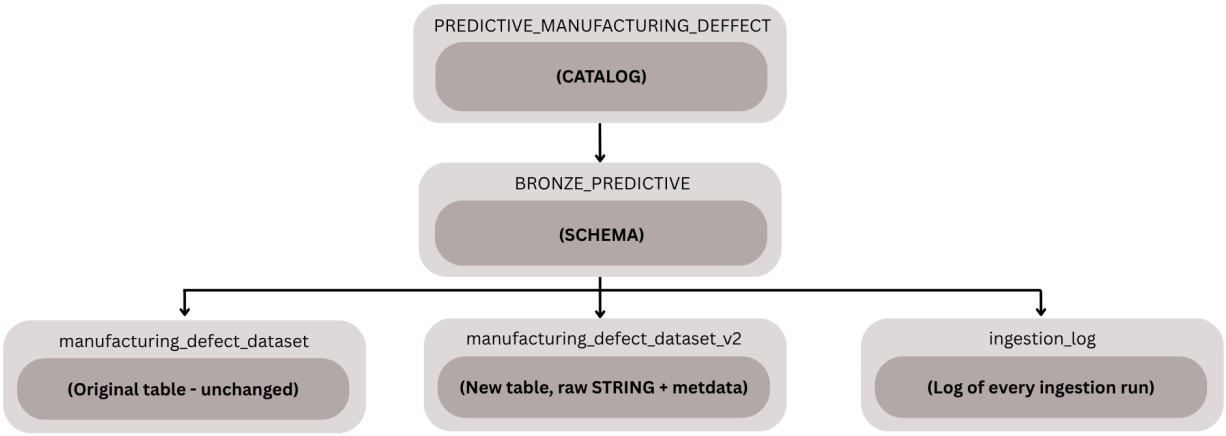
#	ประเด็นใน note-book เดิม	หลักการ Bronze	สิ่งที่แก้ใน v2
1	ใช้ infer-Schema=true ตอนเขียนตาราง	เก็บข้อมูลดิบ ไม่ตีความชนิด	อ่านทุกคอลัมน์เป็น STRING
2	ไม่มี ingestion metadata	ต้องรู้ว่ามาจากไฟล์ไหน เมื่อไร รอบไหน	_source_file, _source_file_modified_at, _in-gested_at, _bronze_batch_id
3	ไม่ตรวจ header/schema	จับ schema drift ตั้งแต่ทางเข้า	เทียบ header กับ EX-PECTED_COLUMNS
4	แถว format เสียหายเงียบ	เก็บ record ที่อ่านไม่ได้	PERMISSIVE + _corrupt_record
5	ไม่มี reconciliation	จำนวนแถวต้องตรงไฟล์	นับบรรทัดไฟล์เทียบตาราง
6	ไม่มี log การโหลด	ETL Monitoring	append ลง ingestion_log
7	อ่านไฟล์ 2 รอบด้วย option ต่างกัน	1 source ⟶ 1 วิธีอ่าน	อ่านครั้งเดียว
8	ไม่มี table com-ment	Data catalog	COMMENT ON TABLE
9	path และชื่อตารางถูก hard-code	ต้องทดสอบกับข้อมูลสกปรกได้โดยไม่ทับตารางจริง	รับ parameter ผ่าน widgets (ค่าเริ่มต้น = ตารางจริง)

4.3 Initial Data Profiling

ตารางที่ 4: ผล profiling และ reconciliation ของ Bronze

รายการ	ผลลัพธ์
จำนวนบรรทัดข้อมูลในไฟล์	3,240
จำนวนแถวใน Bronze v2	3,240 (Reconciliation = PASS)
จำนวนคอลัมน์ธุรกิจ	17 (ครบตาม EXPECTED_COLUMNS, ไม่มี schema drift)
แถว corrupt	0
ค่า NULL / ค่าว่าง ต่อคอลัมน์	0 ทุกคอลัมน์
Distinct DefectStatus	"0" = 517, "1" = 2,723
ชนิดข้อมูลใน Bronze	STRING ทุกคอลัมน์ + metadata 4 คอลัมน์

4.4 Bronze Table Structure



ภาพที่ 7: Bronze Table Structure

4.5 Incremental Ingestion ด้วย Auto Loader

เมื่อทำ pipeline แบบ Automate ข้อมูลจะมาเป็นไฟล์ทีละ batch การอ่านไฟล์ทั้งหมดแล้ว overwrite ทุกครั้งจะช้าลงเรื่อย ๆ และเสี่ยงอ่านไฟล์ซ้ำ จึงเพิ่ม notebook Bronze_AutoLoader ที่ใช้ Auto Loader (cloudFiles) อ่านเฉพาะไฟล์ใหม่ใน landing volume (/Volumes/predictive_manufacring_defect/bronze_preditive/landing/) แล้ว append ลงตาราง Bronze v2 เดิม Auto Loader จำไฟล์ที่อ่านแล้วไว้ใน checkpoint volume จึงอ่านแต่ละไฟล์เพียงครั้งเดียวแม้ Job จะรันซ้ำ

ตารางที่ 5: เปรียบเทียบ Bronze แบบ batch และแบบ incremental

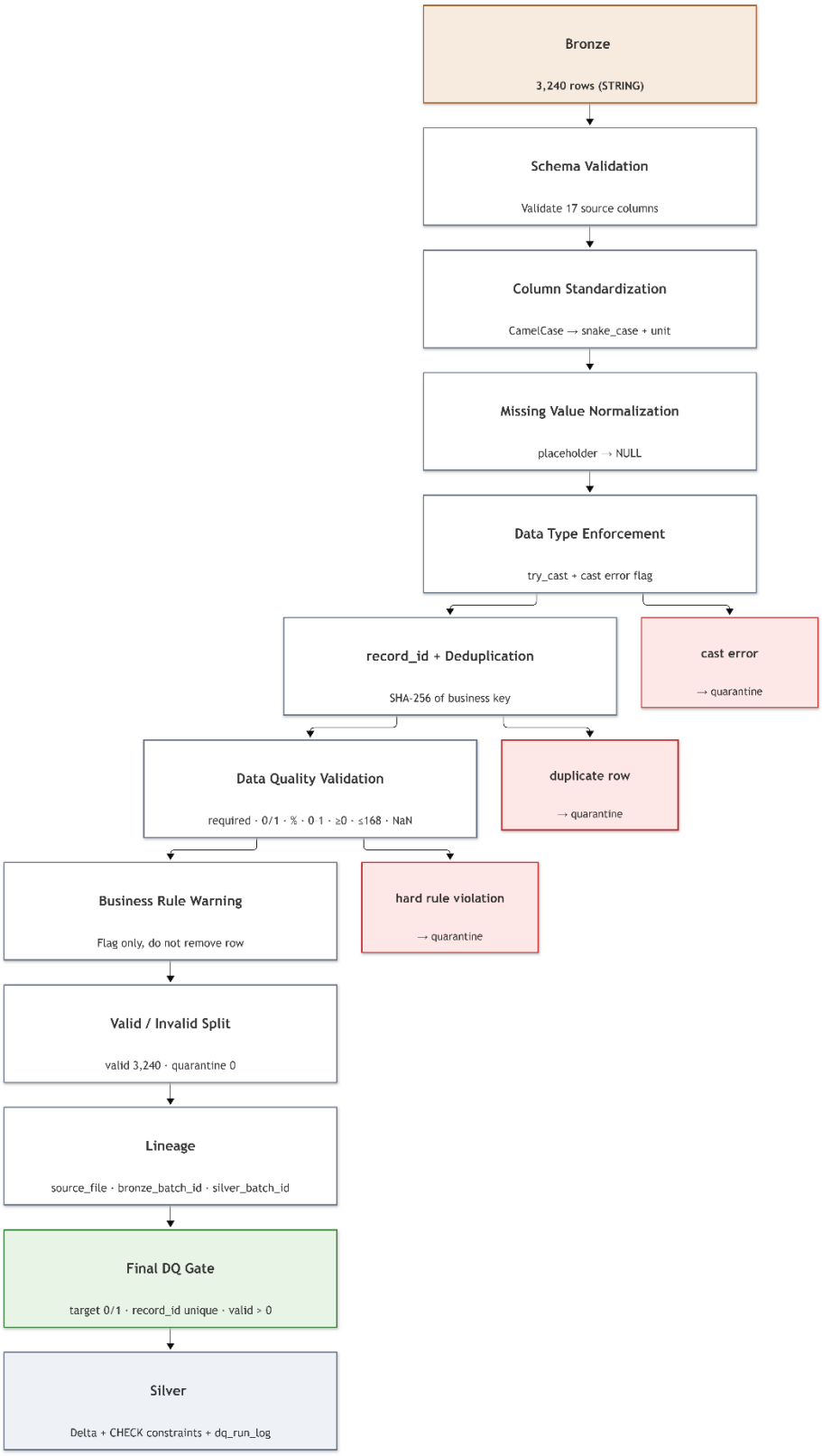
หัวข้อ	Bronze_Predictive_v2 (Manual)	Bronze_AutoLoader (Automate)
ต้นทาง	ไฟล์เดียวใน bronze_predic/	ทุกไฟล์ *.csv ใน landing/ (batch_001, batch_002, ...)
การอ่าน	อ่านทั้งไฟล์ทุกครั้ง	Auto Loader อ่านเฉพาะไฟล์ใหม่ trigger(availableNow) แล้วหยุด
การเขียน	overwrite	append ลงตารางเดิม + checkpoint
การ parse	STRING ทุกคอลัมน์ + PERMISSIVE + _corrupt_record	เหมือนกันทุกอย่าง (Silver ใช้ได้ทันที)
ตรวจ header	ก่อนโหลด	ทีละไฟล์ใหม่ ถ้าไม่ตรงเป็น SCHEMA_DRIFT ⟶ task ล้ม
Reconciliation	ไฟล์ = ตาราง	ทีละไฟล์: บรรทัดในไฟล์ = แถวที่โหลด บันทึกลง ingestion_log
ส่งต่อ task ถัดไป	–	taskValues new_rows และ bronze_batch_id

ชั้น Silver และ Gold ยังคำนวณใหม่จาก Bronze ทั้งตารางทุกรอบ (full recompute) เพราะข้อมูลมีขนาดเล็ก (หลักพันแถว) ข้อดีคือการ dedup ด้วย record_id ครอบคลุมข้อมูลทุก batch จึงจับแถวที่ส่งซ้ำข้ามไฟล์ได้ (เช่น แถวใน batch_002 ที่ซ้ำกับ batch_001) และ threshold, marts, dashboard และโมเดลอัปเดตตามข้อมูลรวมล่าสุดเสมอ ถ้าข้อมูลโตขึ้นมากจึงค่อยเปลี่ยน Silver/Gold เป็น MERGE แบบ incremental

notebook มี widget full_refresh สำหรับล้างตารางและ checkpoint แล้วอ่านทุกไฟล์ใน landing ใหม่ ใช้ตอนย้ายระบบมาใช้ Auto Loader ครั้งแรก และใช้ใน Reset_Demo เพื่อคืนข้อมูลกลับเป็น 3,240 แถว

บทที่ 5 Silver Layer – Data Transformation & Quality

5.1 Silver ETL Flow



ภาพที่ 8: Silver ETL Flow ใน Silver_Predictive_v2

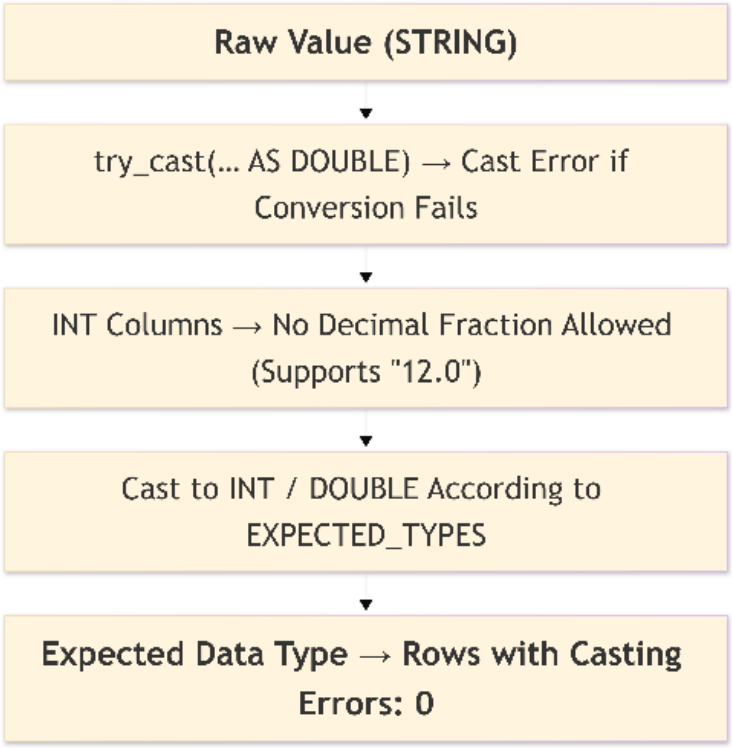
5.2 Column Name Standardization

เปลี่ยนชื่อคอลัมน์ทั้ง 17 คอลัมน์จาก CamelCase เป็น snake_case พร้อมระบุหน่วยในชื่อ เช่น ProductionVolume ⟶ production_volume, StockoutRate ⟶ stockout_rate, MaintenanceHours ⟶ maintenance_hours_per_week และ DefectStatus ⟶ is_high_defect (ดูตารางที่ 2) ก่อน rename มีการตรวจว่าคอลัมน์ต้นทางครบ (Source schema validation: PASS)

5.3 Missing Value Handling

คอลัมน์ธุรกิจทุกคอลัมน์เป็น STRING จาก Bronze จึงตรวจ placeholder 10 แบบ ('', null, none, nan, n/a, na, -, ?, missing, undefined) แบบไม่สนตัวพิมพ์เล็กใหญ่ และแปลงเป็น NULL ผลลัพธ์: พบ placeholder 0 ค่า และ NULL หลัง normalize 0 ค่าทุกคอลัมน์ (ข้อมูลต้นทางสะอาด จึงพิสูจน์ว่ากลไกทำงานจริงด้วยข้อมูลสกปรกในหัวข้อ 9.3)

5.4 Data Type Enforcement



ภาพที่ 9: Data Type Enforcement

5.5 Deduplication

สร้าง record_id = SHA-256 ของค่าธุรกิจทั้ง 17 คอลัมน์ (deterministic รันซ้ำได้ค่าเดิม) แล้วใช้ window row_number() แยกแถวซ้ำ เก็บไว้ 1 แถวและส่งแถวที่เหลือเข้า quarantine พร้อมเหตุผล exact duplicate of another record ผลลัพธ์: แถวซ้ำ 0 แถว (ข้อมูลต้นทางสะอาดจึงพิสูจน์ว่ากลไกทำงานจริงด้วยข้อมูลสกปรกในหัวข้อ 9.3)

5.6 Data Quality Rules

ตารางที่ 6: Data Quality Rules (Hard error ⟶ Quarantine)

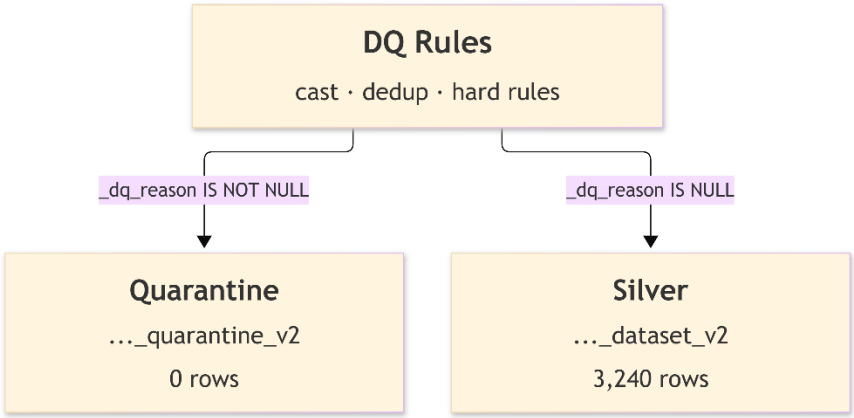
Rule	Condition	Action
Required	is_high_defect, production_volume, defect_rate IS NULL	Quarantine
Target Domain	is_high_defect ไม่ใช่ 0/1	Quarantine
Percentage	supplier_quality, quality_score, downtime, worker_productivity < 0 หรือ > 100	Quarantine
Ratio 0–1	stockout_rate, energy_efficiency_factor < 0 หรือ > 1	Quarantine
Non-negative	10 คอลัมน์ปริมาณ/ต้นทุน < 0	Quarantine
Max hours	maintenance_hours_per_week > 168	Quarantine
NaN	คอลัมน์ DOUBLE เป็น NaN	Quarantine
Cast error	แปลงชนิดไม่ได้ / INT มีเศษ	Quarantine

ทุก rule คืนค่า NULL เมื่อผ่าน และคืนข้อความเมื่อไม่ผ่าน แล้วรวมเป็น array (_dq_errors) แถวที่ _dq_reason IS NULL คือ valid ผลการตรวจทุก rule: ไม่ผ่าน 0 แถว (ข้อมูลต้นทางสะอาดจึงพิสูจน์ว่ากลไกทำงานจริงด้วยข้อมูลสกปรกในหัวข้อ 9.3)

5.7 Business Rule Warning

Warning ไม่ตัดแถว แต่ติดธงใน _dq_warning_reason เพื่อให้ Gold/BI ใช้ต่อ ได้แก่ production_volume = 0 แต่มีต้นทุน/พลังงาน, defect_rate > 4.5 แต่ label = 0 และ quality_score < 60 ผลลัพธ์: warning 0 แถว (ข้อมูลต้นทางสะอาด จึงพิสูจน์ว่ากลไกทำงานจริงด้วยข้อมูลสกปรกในหัวข้อ 9.3)

5.8 Quarantine Process



ภาพที่ 10: Quarantine Flow: Invalid ⟶ Quarantine, Valid ⟶ Silver

Quarantine รวม 3 แหล่ง คือ แถวที่ไม่ผ่าน DQ, แถวซ้ำ และแถว malformed จาก Bronze มีการ assert ว่า valid + quarantine = จำนวน Bronze (3,240 + 0 = 3,240) ในการทดสอบด้วยข้อมูลสกปรก: 203 + 12 = 215 (หัวข้อ 9.3)

5.9 Lineage

ทุกแถวใน Silver มีคอลัมน์ source_table, source_file, source_file_modified_at, bronze_batch_id, bronze_ingested_at, silver_processed_at และ silver_batch_id ทำให้ย้อนจากแถวใน Gold กลับไปถึงไฟล์ (รวมเวลาแก้ไขไฟล์) และรอบการโหลดได้ โดย source_file_modified_at เป็นคอลัมน์ที่เพิ่มใน v2.1 เพราะเดิมถูกเก็บไว้ใน Bronze แต่ไม่ได้ส่งต่อมาถึง Silver

5.10 Silver Data Quality Gate

- ต้องมี valid rows > 0 (กันเขียนตารางว่างทับ ซึ่งเคยเกิดใน Test2_Silver)
- คอลัมน์ครบตาม FINAL_COLUMNS
- is_high_defect ไม่เป็น NULL และเป็น 0/1 เท่านั้น
- record_id ไม่เป็น NULL และไม่ซ้ำ

ผล: **Silver Data Quality Gate: PASS** หลังเขียนตารางมีการเพิ่ม Delta CHECK constraints 9 ข้อ เช่น chk_target_binary, chk_stockout_ratio, chk_maint_hours

5.11 Silver Output



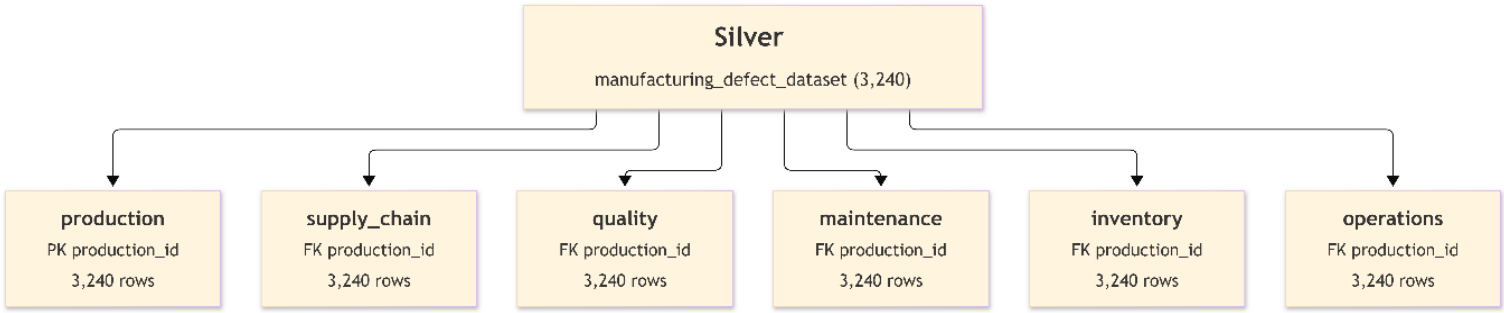
ภาพที่ 11: Silver Output

บทที่ 6 Gold Layer – RDBMS Transformation

6.1 Gold Layer Objective

Gold จัดข้อมูลให้อยู่ในรูปแบบที่เหมาะกับการใช้งานทางธุรกิจและ RDBMS โดยแยก Silver 1 ตารางออกเป็น 6 ตารางตาม business domain เชื่อมกันด้วย PK/FK โหลดเข้าทั้ง Delta Lake (Unity Catalog) และ PostgreSQL 17 บน Databricks Lakebase และสร้าง Analytic marts สำหรับ Dashboard

6.2 Split Silver into 6 DataFrames



ภาพที่ 12: การแยก Silver เป็น 6 ตารางใน Gold

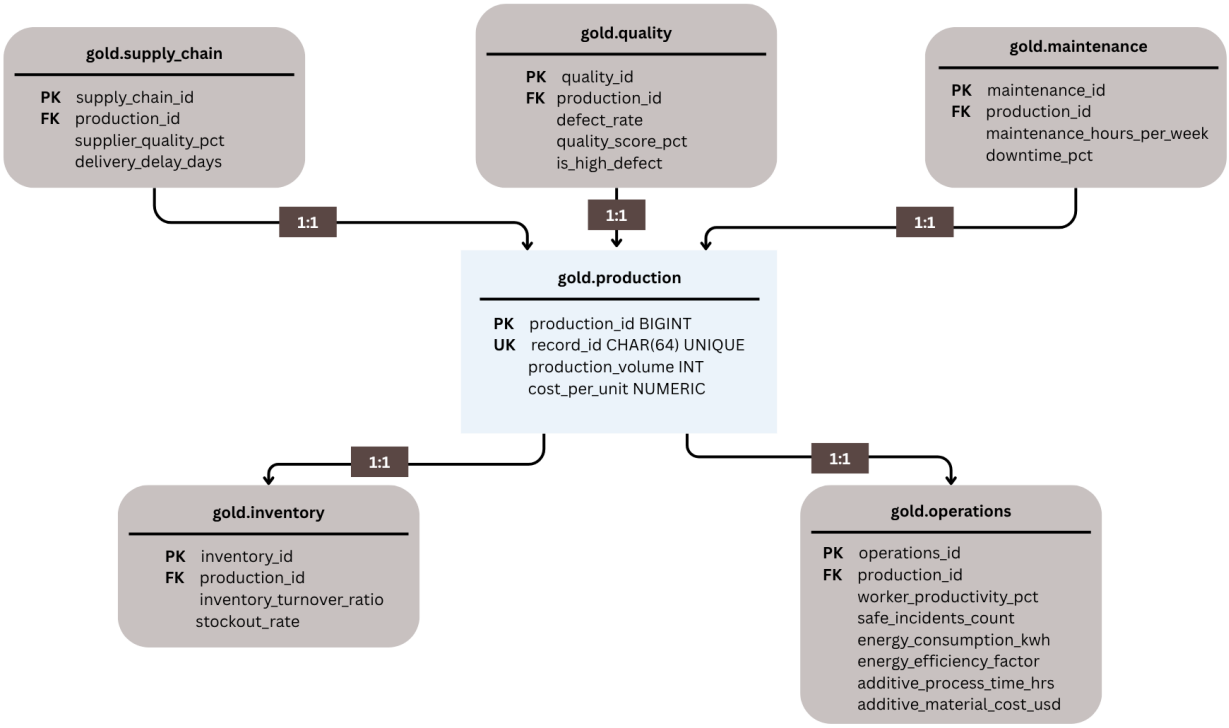
6.3 Data Domain Mapping

ตารางที่ 7: Gold Table – Business Domain – Columns

Gold Table	Domain	PK / FK	Columns
production	Production	PK production_id	record_id (UNIQUE), production_volume, production_cost, cost_per_unit, volume_band
supply_chain	Supply Chain	PK supply_chain_id, FK production_id	supplier_quality_pct, delivery_delay_days, bands
quality	Quality	PK quality_id, FK production_id	defect_rate, quality_score_pct, is_high_defect, bands, dq_warning_reason
maintenance	Maintenance	PK maintenance_id, FK production_id	maintenance_hours_per_week, downtime_pct, bands
inventory	Inventory	PK inventory_id, FK production_id	inventory_turnover_ratio, stockout_rate, stockout_band
operations	Workforce / Safety / Energy / Additive	PK operations_id, FK production_id	worker_productivity_pct, safety_incidents_count, energy_consump-

Gold Table	Domain	PK / FK	Columns
			tion_kwh, energy_efficiency_factor, energy_kwh_per_unit, additive_process_time_hrs, additive_material_cost_usd

6.4 RDBMS Design (ER Diagram)



ภาพที่ 13: ER Diagram ของ Gold (PostgreSQL schema gold)

ความสัมพันธ์เป็นแบบ 1:1 เพราะ 1 แถวของ dataset แทน 1 production batch ทุกตารางลูกจึงมี production_id ที่ไม่ซ้ำ (UNIQUE) และอ้างอิง production.production_id โครงสร้างนี้แยกข้อมูลตาม domain ให้แต่ละทีม (QC, Maintenance, Supply Chain) ดูแล และ query ตารางของตนเองได้ และขยายเป็น 1:N ได้ในอนาคตเมื่อมีข้อมูลหลายรายการต่อ batch

6.5 Primary Key

production_id เป็น surrogate key ชนิด BIGINT สร้างด้วย row_number() เรียงตาม record_id (SHA-256) จึงได้ค่าเดิมทุกครั้งที่รันกับข้อมูลชุดเดิม ตารางลูกใช้ PK ของตนเอง (supply_chain_id, quality_id, maintenance_id, inventory_id, operations_id) ส่วน record_id เป็น natural key แบบ UNIQUE ใน production

6.6 Foreign Key

ตารางทุกช่องมี production_id BIGINT NOT NULL UNIQUE REFERENCES gold.production (production_id) ON DELETE CASCADE ใน PostgreSQL ส่วนใน Delta Lake ใช้ ALTER TABLE ... ADD CONSTRAINT fk_<table>_production FOREIGN KEY ... REFERENCES (informational constraint ของ Unity Catalog)

6.7 PK/FK Integrity Validation



ภาพที่ 14: ขั้นตอนตรวจ PK/FK และ negative test

ตารางที่ 8: ผลตรวจ PK/FK ใน Spark ก่อนเขียน Gold

Table	PK	Rows	PK NULL	PK ซ้ำ	FK orphan	Status
production	production_id	3,240	0	0	–	PASS
supply_chain	supply_chain_id	3,240	0	0	0	PASS
quality	quality_id	3,240	0	0	0	PASS
maintenance	maintenance_id	3,240	0	0	0	PASS
inventory	inventory_id	3,240	0	0	0	PASS
operations	operations_id	3,240	0	0	0	PASS

6.8 Gold Table Schema และ Analytic Marts

ตารางที่ 9: ตารางใน predictive_manufacring_defect.gold_predictive

ตาราง	ประเภท	คำอธิบาย
production, supply_chain, quality, maintenance, inventory, operations	RDBMS (Delta)	6 ตาราง PK/FK ตามตารางที่ 7
batch_360	Mart	join 6 ตาราง + controllable flags 2 ข้อ + QC alert 2 ข้อ + risk_segment, qc_alert, label_rule_pred, risk_flag_count
kpi_overview	Mart	KPI 19 ตัว 1 แถว (รวม QC alert % และ label-rule agreement %)
mart_threshold_scan	Mart	threshold ที่ดีที่สุดต่อตัวแปร + บทบาท (Controllable / Outcome) + gap (pp)
mart_driver_bands	Mart	high-defect % แยกตาม band ของ 10 ตัวแปร + driver_role
mart_driver_impact	Mart	spread, correlation, threshold, gap และระดับผลกระทบต่อตัวแปร
mart_risk_matrix	Mart	maintenance band × volume band (ทั้งคู่เป็น Controllable)
mart_risk_segment	Mart	สรุปตาม controllable risk segment (Low / Medium / High)
mart_action_matrix	Mart	controllable segment × QC status พร้อม action
mart_label_rule	Mart	Unknown Question: กฎ threshold เทียบกับ label
mart_maint_hours_curve	Mart	high-defect % ต่อชั่วโมงซ่อมบำรุงทีละค่า
gold_run_log	Log	บันทึกการรัน Gold

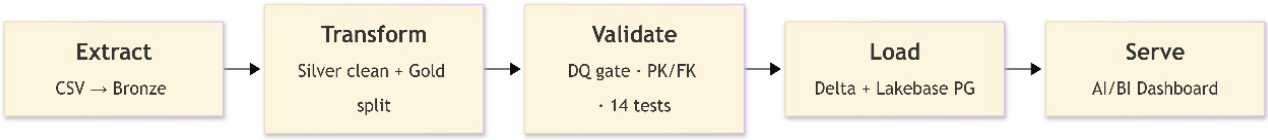
V2.1 เพิ่มขั้น Threshold scan ก่อนสร้าง marts: แต่ละตัวแปรจะถูกลอง cut-off ทุกจุด (ทุกค่าจำนวนเต็ม หรือ percentile 2-98) แล้วเลือกจุดที่ high-defect% ของฝั่งเสี่ยงกับฝั่งปลอดภัยต่างกันมากที่สุด โดยแต่ละฝั่งต้องมีอย่างน้อย 5% ของ batch จากนั้นปัดเป็นตัวเลขที่สื่อสารง่าย ตัวแปรที่ต่างกันอย่างน้อย 15 pp ถือเป็น key factor และแต่ละตัวแปรถูกกำหนดบทบาทเป็น Controllable (ทีมปรับได้) หรือ Outcome (ตัววัดผลคุณภาพ) ดังรายละเอียดในบทที่ 8

6.8.1 PostgreSQL (Databricks Lakebase)

- Project: manufacturing-defect-gold, branch production, endpoint primary (PostgreSQL 17)
- การเชื่อมต่อ: Databricks SDK ขอ OAuth token อายุสั้น (ไม่มี password ในโค้ด) + driver pg8000
- DDL: 6 ตารางพร้อม PK / FK / UNIQUE / CHECK และ COMMENT
- โหลดข้อมูล: COPY ... FROM STDIN (CSV) ภายใน transaction เดียว parent ก่อน child
- Views: vw_batch_360, vw_kpi_overview, vw_defect_by_maintenance, vw_risk_segment และ vw_label_rule_check (เงื่อนไข flag สร้างจาก threshold ชุดเดียวกับ Delta)
- ลงทะเบียนเป็น Unity Catalog catalog manufacturing_gold_pg ให้ SQL Warehouse และ Dashboard query ได้

บทที่ 7 ETL Process Implementation

7.1 ETL Workflow



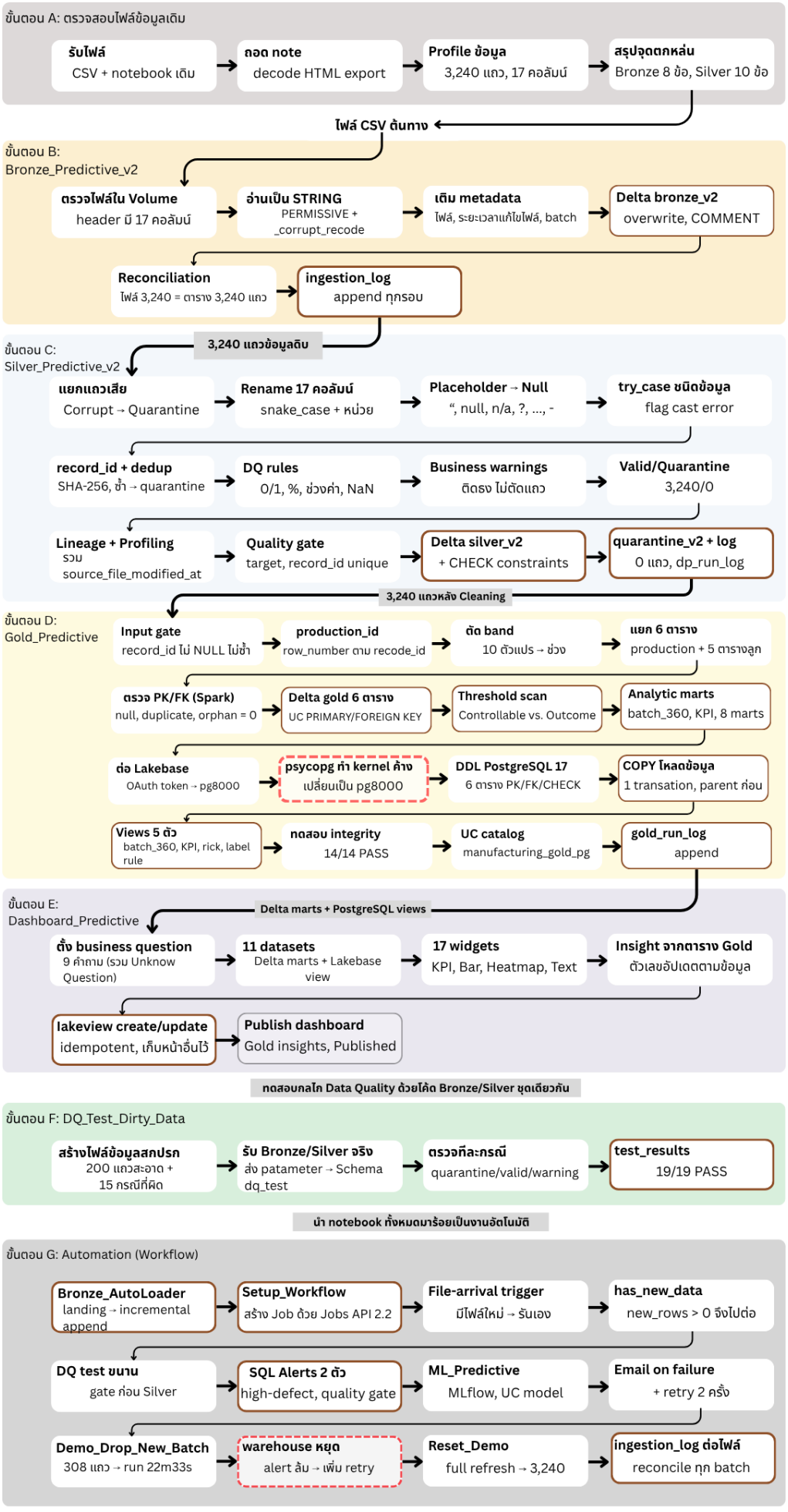
ภาพที่ 15: ETL Workflow: Extract ⟶ Transform ⟶ Validate ⟶ Load ⟶ Serve

ตารางที่ 10: การ map ขั้น ETL กับระบบ

ขั้น	งานในโครงการ	Notebook
Extract	Auto Loader อ่านไฟล์ใหม่จาก landing เป็น Bronze v2 (raw STRING + metadata) แบบ append / แบบ Manual อ่านไฟล์เดียวแล้ว overwrite	Bronze_AutoLoader (Automate), Bronze_Predictive_v2 (Manual)
Transform	Silver cleaning, DQ, dedup, lineage / Gold split 6 ตาราง, bands, marts	Silver_Predictive, Gold_Predictive
Validate	reconciliation, DQ gate, PK/FK (Spark), integrity test 14 ข้อ (PostgreSQL), dirty data test 19 ข้อ	ทุก notebook + DQ_Test_Dirty_Data
Load	Delta tables ใน Unity Catalog + PostgreSQL (Lakebase)	Silver_Predictive, Gold_Predictive
Serve	AI/BI Dashboard 17 widgets + หน้า Model, โมเดลทำนาย high-defect	Dashboard_Predictive, ML_Predictive
Orchestrate	Databricks Workflow: file arrival trigger, ลำดับ task, condition, retry, SQL Alert, อีเมล	Setup_Workflow ⟶ Job Manufacturing_Defect_Pipeline

7.2 ETL Execution

ทุกขั้นรันบน Databricks Free Edition ด้วย Serverless compute notebook ทั้งหมดอยู่ในโฟลเดอร์ Predictive_Manufaturring/Predictive_Manu (Git folder) ช่วงพัฒนาใช้การ Run all ทีละ notebook ตามลำดับ Bronze ⟶ Silver ⟶ Gold ⟶ Dashboard แล้วรัน DQ_Test_Dirty_Data เพื่อพิสูจน์กลไก Data Quality เมื่อทุก notebook ผ่านแล้วจึงนำมาร้อยเป็น Databricks Workflow ให้ทำงานเองเมื่อมีไฟล์ใหม่ (หัวข้อ 7.3) ภาพที่ 16 สรุปขั้นตอนทั้งหมดที่ทำจริงตั้งแต่ตรวจของเดิมจนถึง Automation



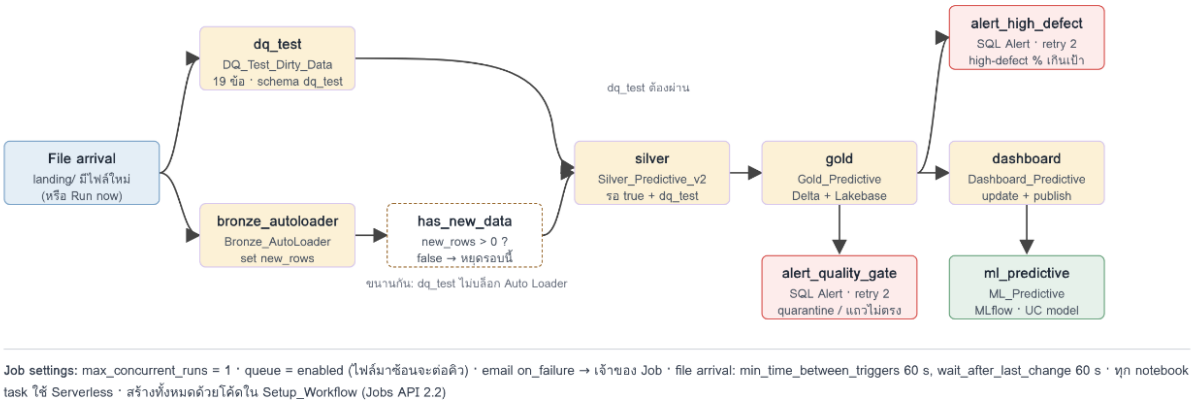
ภาพที่ 16: Flow การทำงานทั้งหมดที่เกิดขึ้นจริง (A ตรวจของเดิม ถึง G Automation ด้วย Databricks Workflow)

ปัญหาที่พบระหว่างรันและการแก้ไข

- Gold รอบแรก: psycopg[binary] ทำให้ Python kernel บน serverless ไม่ตอบสนอง (Fatal error) แม้ TCP/TLS ไปยัง Lakebase ทำงานปกติ ⟶ เปลี่ยนเป็น pg8000 (pure-Python) แล้วรันผ่าน
- Dashboard รอบแรก: ข้อความ markdown ต่อกันเป็นบรรทัดเดียว และสีของ heatmap สื่อความกลับด้าน ⟶ แก้เป็น 1 บรรทัดต่อรายการ, เรียงแท่งจากมากไปน้อย และปรับ color scale

7.3 Manual / Automated Execution

ETL ทำงานแบบ Automate ผ่าน Databricks Workflow ชื่อ Manufacturing_Defect_Pipeline ทุก notebook ออกแบบให้ idempotent (overwrite + ลบ/สร้าง constraint ใหม่) และรับ parameter ผ่าน widgets จึงนำมาเป็น task ของ Job ได้โดยตรง การทำงานเริ่มเมื่อมีไฟล์ CSV ใหม่วางใน landing volume (file arrival trigger) หรือกด Run now ก็ได้ ภาพที่ 17 แสดงโครงสร้าง Job และตารางที่ 11 อธิบายหน้าที่ของแต่ละ task



ภาพที่ 17: โครงสร้าง Job Manufacturing_Defect_Pipeline (v2: dq_test รันขนานกับ bronze_autoloader)

ตารางที่ 11: Task ใน Job Manufacturing_Defect_Pipeline

Task	ประเภท / notebook	ทำงานเมื่อ	หน้าที่
dq_test	Notebook: DQ_Test_Dirty_Data	เริ่ม Job	รัน Bronze/Silver ตัวจริงกับข้อมูลสกปรก 15 กรณี (schema dq_test) ต้องผ่าน 19/19 ก่อนแตะข้อมูลจริง
bronze_autoloader	Notebook: Bronze_AutoLoader	เริ่ม Job (ขนานกับ dq_test)	อ่านไฟล์ใหม่ append ลง Bronze ตรวจ header/reconciliation ทีละไฟล์ ส่ง new_rows

Task	ประเภท / notebook	ทำงานเมื่อ	หน้าที่
has_new_data	Condition task	bronze_autoloader สำเร็จ	new_rows > 0 จึงไปต่อ ถ้าไม่มีข้อมูลใหม่ task ที่เหลือถูกข้าม (ไม่เสีย compute)
silver	Notebook: Silver_Predictive_v2	has_new_data = true และ dq_test ผ่าน	ทำความสะอาด DQ dedup quarantine lineage จาก Bronze ทั้งตาราง
gold	Notebook: Gold_Predictive	silver สำเร็จ	แยก 6 ตาราง PK/FK, threshold scan, marts, โหลด Lakebase, integrity test
alert_high_defect	SQL Alert (retry 2)	gold สำเร็จ	แจ้งเตือนเมื่อ high-defect % เกินเป้า
alert_quality_gate	SQL Alert (retry 2)	gold สำเร็จ	แจ้งเตือนเมื่อมีแถวใน quarantine หรือจำนวนแถว Bronze/Silver/Gold ไม่ตรงกัน
dashboard	Notebook: Dashboard_Predictive	gold สำเร็จ	อัปเดต datasets/insight และ publish dashboard
ml_predictive	Notebook: ML_Predictive	dashboard สำเร็จ	train/evaluate โมเดล log MLflow register UC model และเพิ่มหน้า Model

การสร้าง Job ด้วยโค้ด (Setup_Workflow)

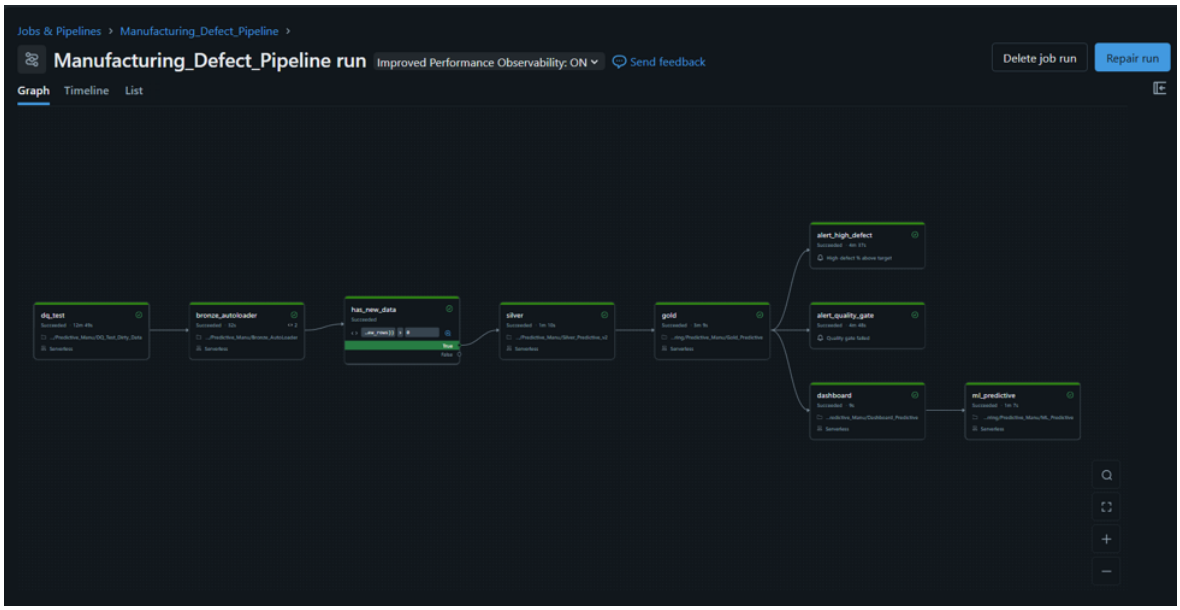
Job ไม่ได้สร้างด้วยการคลิกใน UI แต่สร้างด้วย notebook Setup_Workflow ผ่าน Databricks SDK และ Jobs API 2.2 จึงสร้างซ้ำได้เหมือนเดิมทุกครั้ง (idempotent: ถ้ามี Job ชื่อเดิมจะ reset ทับ) ขั้นตอนใน notebook มีดังนี้

- เตรียม landing และ checkpoints volume แล้วคัดลอกไฟล์ต้นทางเป็น manufacturing _defect_batch_001.csv
- (ครั้งแรก) ล้างตาราง Bronze เดิมและ checkpoint เพื่อให้ Auto Loader สร้างตารางใหม่จากไฟล์ใน landing โดยไม่เกิดแถวซ้ำ
- ค้นหา ID ของ SQL Alert 2 ตัว และ SQL Warehouse ที่ใช้รัน alert
- นิยาม task และ dependency ตามตารางข้างต้น กำหนด file_arrival trigger (min_time_between_triggers 60 วินาที, wait_after_last_change 60 วินาที) ตั้ง max_concurrent_runs = 1 พร้อม queue เพื่อให้ไฟล์ที่มาซ้อนกันต่อคิว และตั้ง email_notifications on_failure ไปยังเจ้าของ Job

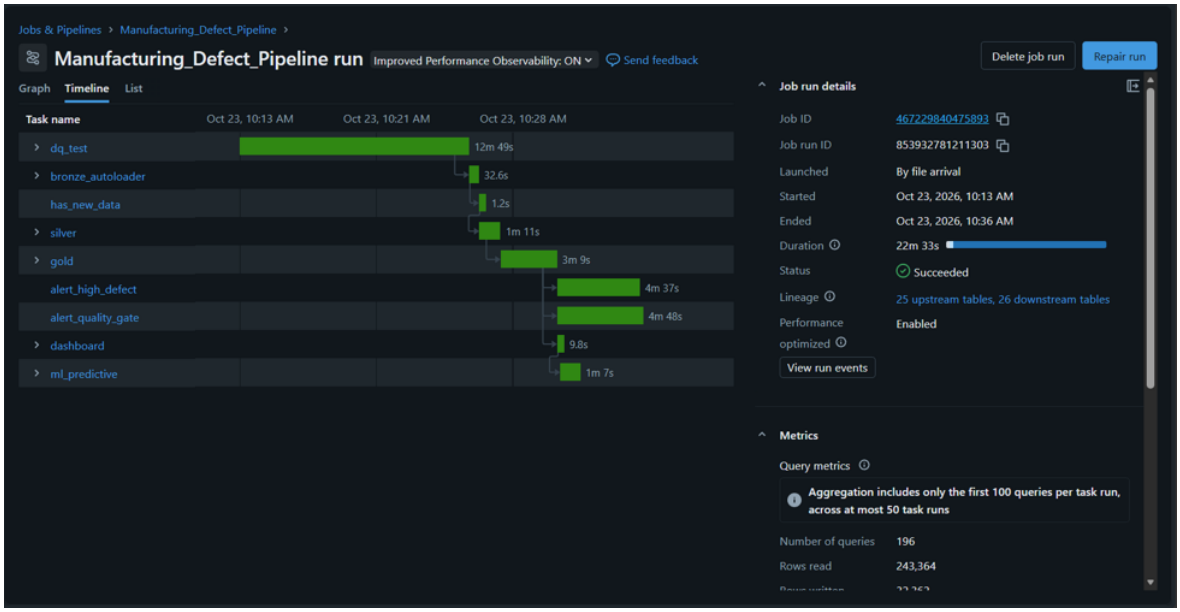
- มี fallback หลายระดับ: ถ้า workspace ไม่รองรับ alert_task จะใช้ sql_task แทน ถ้ายังไม่ได้จะสร้าง Job โดยไม่มี alert และถ้า trigger ใช้ไม่ได้จะสร้าง Job แบบไม่มี trigger

ผลการรันจริง

ทดสอบด้วย notebook Demo_Drop_New_Batch ที่เขียนไฟล์ manufacturing_defect_batch_002.csv (308 แถว) ลง landing จากนั้นไม่ต้องกดอะไรอีก Job ถูกปลุกเองโดย file arrival trigger (Launched: By file arrival) และทำงานครบทุก task ดังภาพที่ 18 และ 19



ภาพที่ 18: Job run จริง (run 853932781211303) ทุก task Succeeded — รอบนี้ยังเป็นโครงแบบ v1 ที่ dq_test รันก่อน bronze_autoloader



ภาพที่ 19: Timeline ของ run เดียวกัน: เริ่ม 10:13 จบ 10:36 รวม 22 นาที 33 วินาที (Launched: By file arrival)

ตารางที่ 12: เวลาที่ใช้ของแต่ละ task (รอบ demo และรอบ reset)

Task	รอบ demo (file arrival)	รอบ reset (full refresh)	หมายเหตุ
dq_test	12m 49s	9m 45s	ช้าที่สุด: เรียก notebook ลูก 2 ตัวบน serverless (cold start)
bronze_autoloader	32.6s	32s	เขียน 309 แถว (308 + log 1)
has_new_data	1.2s	–	new_rows = 308 ⟶ true
silver	1m 11s	1m 11s	
gold	3m 9s	3m 15s	รวมโหลด Lakebase และ integrity test
alert_high_defect / alert_quality_gate	4m 37s / 4m 48s	Failed	ส่วนใหญ่เป็นเวลาเปิด SQL Warehouse; รอบ reset warehouse ถูกหยุดกลางคัน
dashboard	9.8s	11s	
ml_predictive	1m 7s	1m 11s	เขียน predictions 3,240+ แถว
รวมทั้ง Job	22m 33s (Succeeded)	29m 32s (Failed เฉพาะ alert)	ข้อมูลทุกชั้นถูกต้องทั้ง 2 รอบ

บทเรียนจากการรันจริงและการปรับปรุง

- dq_test ใช้เวลา 10–13 นาทีจาก cold start ของ notebook ลูกบน serverless จึงตั้ง timeout ของ notebook ลูกเป็น 1,800 วินาที และปรับ Job เป็น v2 ให้ dq_test รันขนานกับ bronze_autoloader (silver รอทั้งสองทาง) ประหยัดเวลาได้ราว 35 วินาที เพราะ silver ยังต้องรอ dq_test อยู่ดี ทางเลือกต่อไปคือย้าย dq_test ไปเป็น Job แยกที่รันตามตารางหรือเมื่อแก้โค้ด
- รอบ reset (run 231279394655931) alert ทั้งสองล้มด้วยข้อความ warehouse was stopped และรอบถัดมา SQL Warehouse เปิดไม่ได้ด้วยข้อความ RESOURCE_EXHAUSTED ซึ่งเป็นโควตา serverless compute ของ Free Edition เพราะมี notebook ที่เปิด session ค้างไว้ (รวมถึง Reset_Demo เดิมที่นั่งรอ Job จบ) แย่งโควตากับ warehouse ขณะที่ task ข้อมูลทุกตัวสำเร็จและข้อมูลกลับเป็น 3,240 แถวถูกต้อง จึงเพิ่ม retry 2 ครั้ง (เว้น 60 วินาที) และ timeout 900 วินาทีให้ alert task ปรับ Reset_Demo ให้สั่ง Job แล้วจบทันทีไม่นั่งรอ และไม่สั่งรันซ้ำถ้ามี run ค้างอยู่ (กันกด Run all 2 ครั้งแล้ว run ต่อคิวกัน)

- อีเมลแจ้ง Job ล้มทำงานจริง (ภาพที่ 22) ทำให้รู้ปัญหาโดยไม่ต้องเปิดหน้า Job
- ก่อนสาธิตควรปิด (Terminate) compute ของ notebook ที่ไม่ใช้ และเปิด SQL Warehouse ไว้ก่อน เพื่อไม่ให้ชนโควตาและลดเวลารอของ alert task

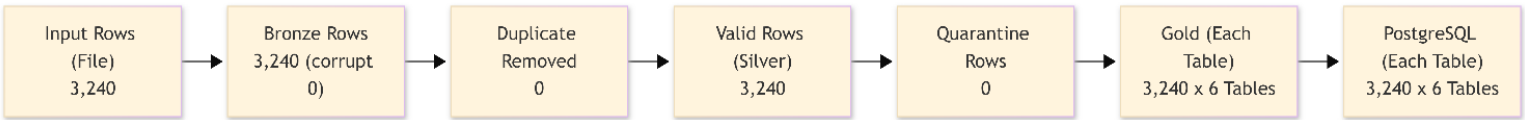
การสาธิตและการคืนสถานะ (Demo / Reset)

Demo_Drop_New_Batch สร้างไฟล์จาก seed คงที่ ได้ไฟล์เดิมทุกครั้ง ประกอบด้วยแถวใหม่ 300 แถว (ค่าขยับ ±2% จากแถวจริง), แถวซ้ำกับ batch_001 5 แถว และแถวค่าผิด 3 แถว (QualityScore 150, DefectStatus 2, ProductionCost "abc") ส่วน Reset_Demo ย้ายไฟล์ batch_002 ไปเก็บใน landing_archive (ไม่ลบ) แล้วสั่ง Job 1 รอบด้วย full_refresh = true ทำให้ทุกชั้นกลับเป็น 3,240 แถวและตัวเลขตรงกับรายงาน

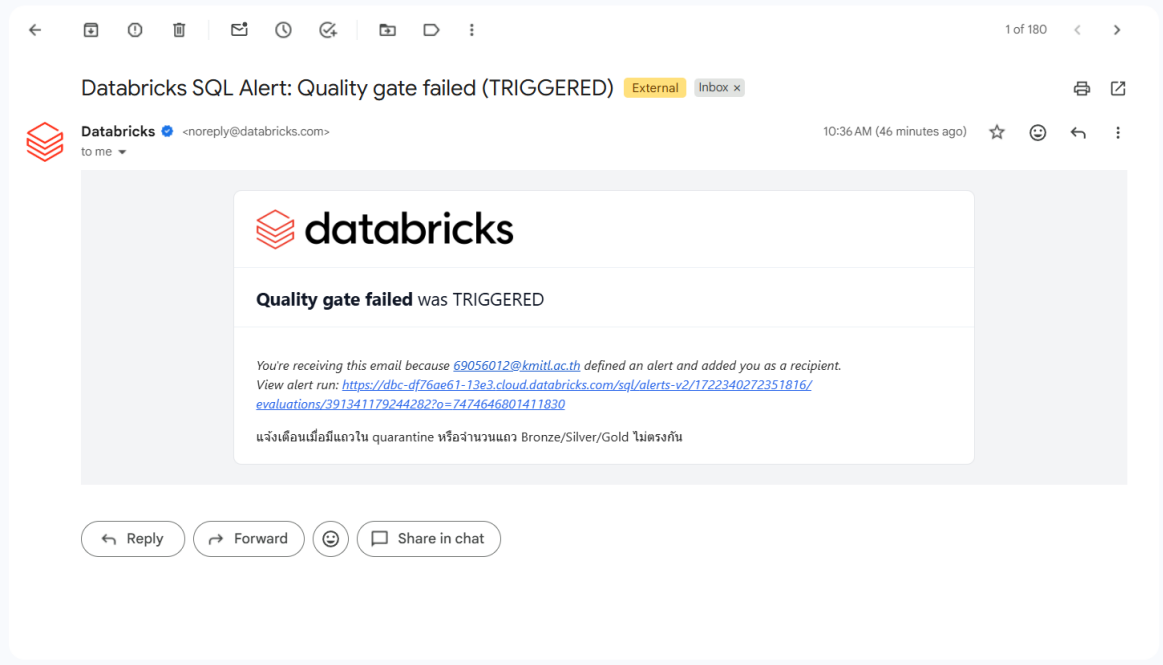
7.4 ETL Monitoring

ตารางที่ 13: ตาราง log สำหรับ monitoring

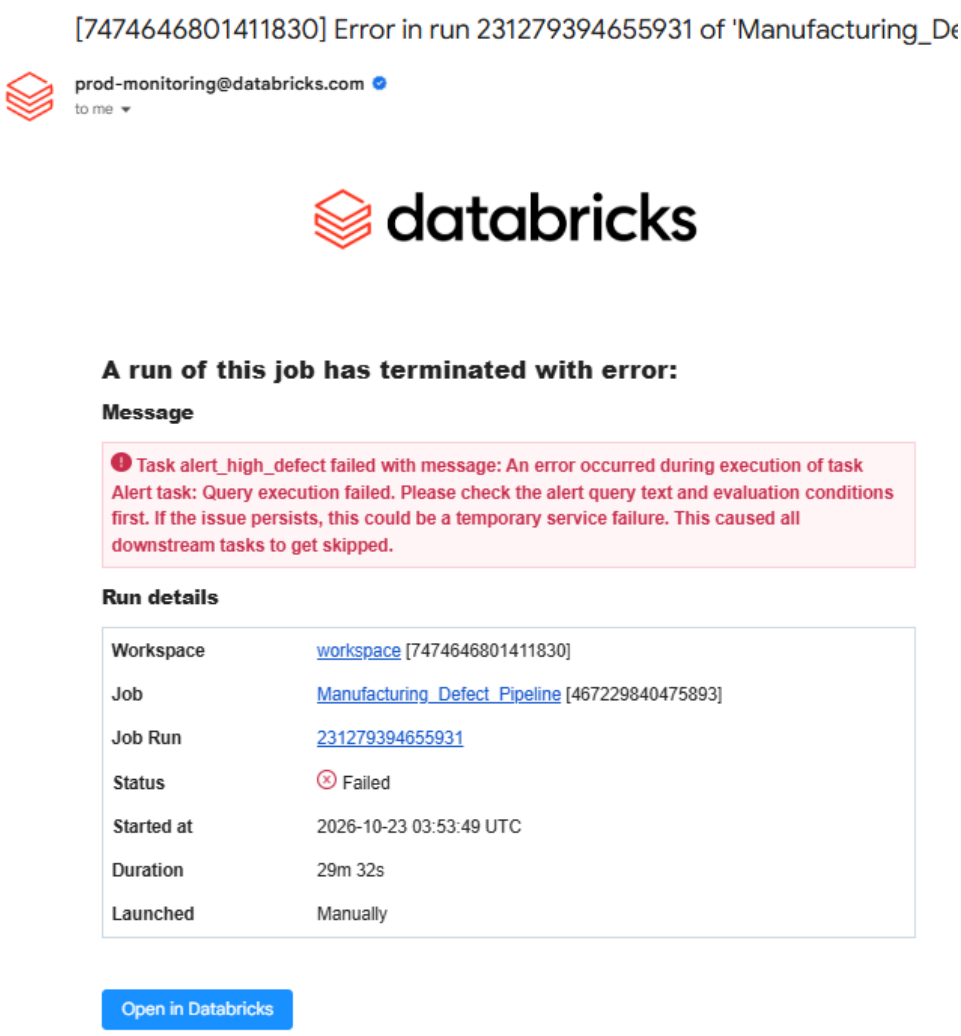
ตาราง	เก็บอะไร
bronze_preditive.ingestion_log	batch_id, source_file, source_rows, loaded_rows, corrupt_rows, status (PASS / FAIL / SCHEMA_DRIFT), started_at, finished_at — Auto Loader บันทึก 1 แถวต่อไฟล์ใหม่
silver_predictive.dq_run_log	row trace ทุกขั้นของ Silver ต่อ silver_batch_id
gold_predictive.gold_run_log	จำนวนแถวที่โหลดเข้า PostgreSQL ต่อตาราง ต่อ gold_batch_id
dq_test.test_results	ผลทดสอบกลไก Data Quality ทีละกรณี (expected / actual / PASS-FAIL)
Job run history	สถานะและเวลาของทุก task ต่อ run, trigger ที่ปลุก (file arrival / manual) ดูได้จากหน้า Jobs & Pipelines
SQL Alerts + อีเมล	alert_high_defect, alert_quality_gate ส่งอีเมลเมื่อ TRIGGERED และ email on_failure เมื่อ task ใดล้ม



ภาพที่ 20: ETL Monitoring



ภาพที่ 21: อีเมลจาก SQL Alert "Quality gate failed" (TRIGGERED) หลังรับ batch_002 ที่มีแถวเข้า quarantine 8 แถว



ภาพที่ 22: อีเมลแจ้ง Job ล้ม (run 231279394655931) เมื่อ alert task ล้มจาก SQL Warehouse หยุดกลางคัน

บทที่ 8 Business Intelligence / Dashboard

8.1 Business Questions

1. สถานการณ์คุณภาพโดยรวมเป็นอย่างไร และกระทบต้นทุนเท่าไร
2. ปัจจัยใดที่ทีมปรับได้ (Controllable) สัมพันธ์กับ high-defect มากที่สุด
3. ชั่วโมงซ่อมบำรุงมีผลอย่างไร และจุดเปลี่ยนอยู่ตรงไหน
4. Defect rate ระดับใดควรใช้เป็น QC alert (ตัววัดผล)
5. Quality score ระดับใดควรใช้เป็น QC alert (ตัววัดผล)
6. การผลิต batch ใหญ่เสี่ยงกว่าหรือไม่
7. ปัจจัยใดไม่ช่วยอธิบาย defect (ไม่ควรลงทุนเป็นหลัก)
8. มี operating window ที่ปลอดภัยหรือไม่ (ก่อนผลิต + หลังผลิต)
9. Unknown Question: label is_high_defect นิยามจากอะไร

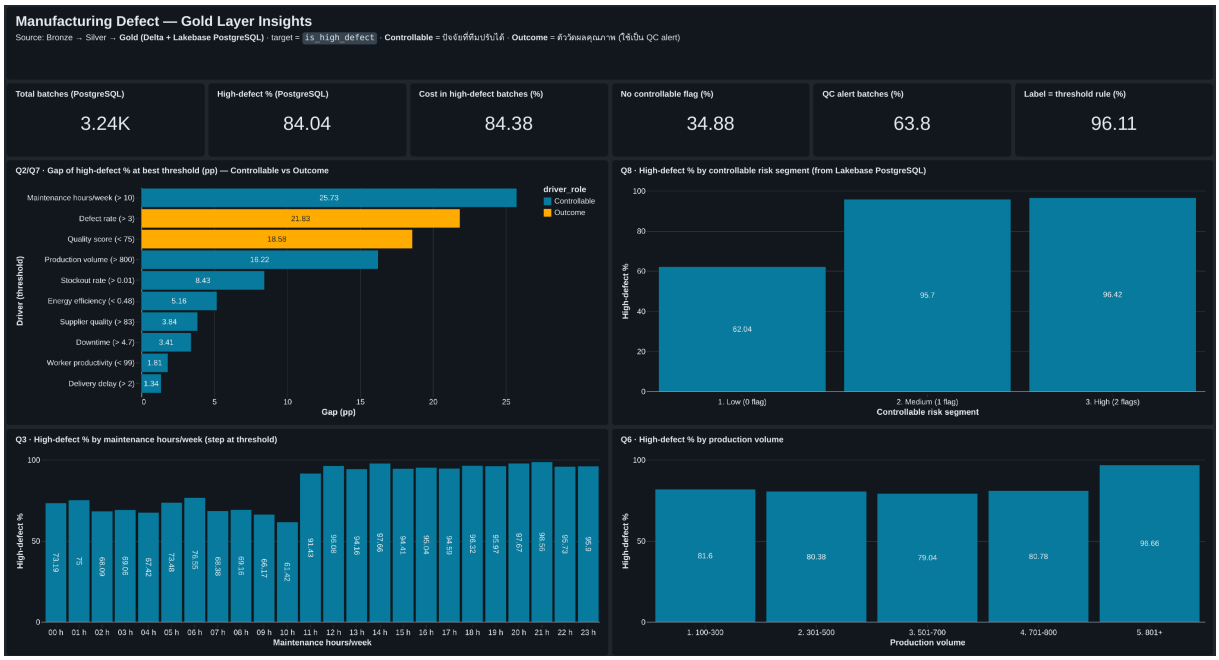
8.2 KPI

ตารางที่ 14: KPI จาก gold_predictive.kpi_overview

KPI	ค่า	KPI	ค่า
Total batches	3,240	High-defect batches	2,723 (84.04%)
Total units	1,777,215	Avg defect rate	2.749
Total production cost	$40,250,579.86	Avg cost per unit	$22.65
Cost in high-defect batches	84.38%	No controllable flag	34.88%
Avg quality score	80.13	Avg supplier quality	89.83
Avg maintenance hours/week	11.48	Avg downtime	2.50%
Avg stockout	5.09%	Avg worker productivity	90.04%
Total safety incidents	14,877	Total energy	9,682,722 kWh
QC alert batches	63.80%	Label = threshold rule	96.11%

8.3 Dashboard

Dashboard ชื่อ Manufacturing Defect - Gold Insights สร้างด้วยโค้ดใน Dashboard_Predictive (Lakeview API) มี 11 datasets และ 17 widgets ได้แก่ KPI counters 6 ตัว (จำนวน batch และ high-defect % ดึงจาก vw_kpi_overview ของ Lakebase PostgreSQL), กราฟ gap ของตัวแปรแยก Controllable / Outcome, กราฟ risk segment (ดึงจาก vw_risk_segment ของ PostgreSQL), กราฟ high-defect % ตามชั่วโมงซ่อมบำรุงทีละค่า, ตาม production volume, defect rate และ quality score, heatmap 2 ตัว (risk matrix และ operating window), กราฟ Unknown Question และกล่อง insight ที่ดึงตัวเลขจากตาราง Gold ตอนรัน หลังเพิ่ม ML_Predictive ใน Workflow dashboard มีหน้าที่ 2 ชื่อ Model สำหรับผลโมเดล (หัวข้อ 8.7)

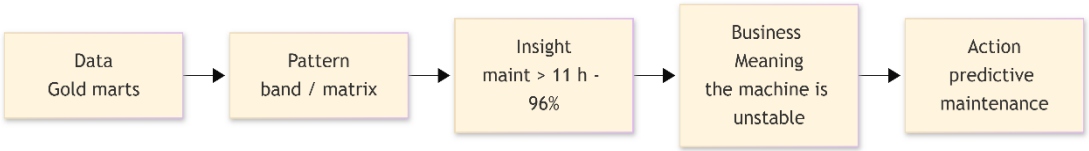


ภาพที่ 23: Dashboard ส่วนบน: KPI, Controllable vs Outcome, Risk segment, Maintenance hours และ Production volume



ภาพที่ 24: Dashboard ส่วนล่าง: QC monitoring, Risk matrix, Operating window, Unknown Question และ Key insights

8.4 Business Insight



ภาพที่ 25: Data ⟶ Pattern ⟶ Insight ⟶ Business Meaning ⟶ Action

ก่อนตีความตัวแปรถูกแบ่งเป็น 2 บทบาท เพราะ defect rate และ quality score เป็นผลลัพธ์ด้านคุณภาพที่วัดได้หลังผลิต ถ้านำมาใช้อธิบาย high-defect จะเป็นการอธิบาย defect ด้วย defect เอง (วนในตัวเอง) และไม่บอกว่าทีมควรปรับอะไร จึงใช้ ตัวแปรกลุ่มนี้เป็น QC alert แทน ส่วนตัวแปรที่ทีมปรับได้ก่อนหรือระหว่างผลิตใช้ทำ risk score ก่อนปล่อย batch

ตารางที่ 15: บทบาทของตัวแปร: Controllable vs Outcome

บทบาท	ตัวแปร	ใช้ทำอะไร
Controllable — ทีมปรับได้	maintenance hours, production volume, supplier quality, delivery delay, downtime, stockout, worker productivity, energy efficiency	หาคันโยก ⟶ risk score ก่อนปล่อย batch

บทบาท	ตัวแปร	ใช้ทำอะไร
Outcome — ตัววัดผลคุณภาพ	defect rate, quality score	QC alert / KPI เฝ้าระวัง (ไม่ใช่สาเหตุ)

ตารางที่ 16: ผลของแต่ละตัวแปรต่อ high-defect % ที่ threshold ดีที่สุด (mart_threshold_scan, mart_driver_impact)

ตัวแปร	บทบาท	เกณฑ์	ฝั่งเสี่ยง %	ฝั่งปลอดภัย %	Gap (pp)	ระดับ
Maintenance hours/week	จัดการได้	> 10	95.62	69.89	25.73	Strong
Production volume	จัดการได้	> 800	96.66	80.44	16.22	Strong
Stockout rate	จัดการได้	> 0.01	84.86	76.43	8.43	Moderate
Energy efficiency	จัดการได้	< 0.48	84.30	79.14	5.16	Weak
Supplier quality	จัดการได้	> 83	84.64	80.80	3.84	Weak
Downtime	จัดการได้	> 4.7	87.24	83.84	3.41	Weak
Worker productivity	จัดการได้	< 99	84.13	82.31	1.81	Weak
Delivery delay	จัดการได้	> 2	84.68	83.34	1.34	Weak
Defect rate	ตัววัดผล	> 3	96.18	74.35	21.83	Strong
Quality score	ตัววัดผล	< 75	95.80	77.22	18.58	Strong

Stockout rate ได้ระดับ Moderate เพราะฝั่งปลอดภัย (≤ 0.01) มีเพียงประมาณ 10% ของ batch จึงไม่นับเป็น key factor (ต้องต่างกันอย่างน้อย 15 pp) ส่วน threshold ใหม่ต่างจากฉบับก่อน (maintenance > 11, volume > 700, quality score < 80) เพราะการ scan พบจุดเปลี่ยนจริงที่ 10, 800 และ 75 เช่น batch ที่ volume 701–800 เป็น high-defect 80.78% เท่ากับค่าเฉลี่ย ความเสี่ยงเพิ่มขึ้นเฉพาะ batch ที่เกิน 800

ตารางที่ 17: High-defect % ตาม band ของตัวแปรหลัก (ขอบ band ตรงจุดเปลี่ยน)

ปัจจัย	Band (batches ⟶ high-defect %)
Maintenance hours/week (Controllable)	0–5 h (810) 70.99% · 6–10 h (648) 68.52% · 11–16 h (842) 94.77% · 17–23 h (940) 96.38%

ปัจจัย	Band (batches ⟶ high-defect %)
Production volume (Controllable)	100–300 (739) 81.60% · 301–500 (729) 80.38% · 501–700 (668) 79.04% · 701–800 (385) 80.78% · 801+ (719) 96.66% · ต้นทุน/หน่วย: $61.70 · $30.70 · $20.50 · $17.50 · $13.82
Defect rate (Outcome)	0.5–1 (366) 74.59% · 1–2 (730) 76.30% · 2–3 (705) 72.20% · 3–4 (699) 96.57% · 4–5 (740) 95.81%
Quality score (Outcome)	60–75 (1,190) 95.80% · 75–80 (406) 77.59% · 80–90 (802) 76.81% · 90–100 (842) 77.43%

ตารางที่ 18: Controllable risk segment (maintenance > 10, volume > 800) × QC alert (defect rate > 3 หรือ quality score < 75)

Segment (flags)	Batches (%)	High-defect %	% ต้นทุน	QC ผ่าน: batch (HD%)	QC เตือน: batch (HD%)
Low (0)	1,130 (34.88%)	62.04	34.67	417 (3.12%)	713 (96.49%)
Medium (1)	1,719 (53.06%)	95.70	53.25	611 (95.74%)	1,108 (95.67%)
High (2)	391 (12.07%)	96.42	12.07	145 (95.17%)	246 (97.15%)

แค่มี controllable flag 1 ข้อ ความเสี่ยงก็สูงถึงประมาณ 96% แล้ว (Medium ใกล้เคียง High) ส่วน batch ที่ไม่มี flag และผ่าน QC เป็น operating window ที่ปลอดภัยจริง (3.12%) ขณะที่ batch ที่ไม่มี flag แต่ QC เตือนยังเป็น high-defect 96.49% แสดงว่าปัญหาส่วนนี้ไม่ได้มาจากตัวแปรที่จัดการได้ในข้อมูล

8.5 Actionable Insight

ตารางที่ 19: Insight ⟶ Business Meaning ⟶ Action

#	Insight	Business Meaning	Action
1	84.04% ของ batch เป็น high-defect คิดเป็น 84.38% ของต้นทุน	ปัญหาคุณภาพเป็นเรื่องทั้งโรงงาน	ตั้ง high-defect % เป็น KPI หลัก
2	Maintenance > 10 ชม./สัปดาห์ ⟶ 95.6% (vs 69.9%) เป็นจุดกระโดด ไม่ใช่ค่อย ๆ เพิ่ม	เครื่องที่ต้องซ่อมบ่อยคือเครื่องไม่เสถียร (ชั่วโมงซ่อมอาจเป็นอาการ ไม่ใช่สาเหตุ)	Predictive maintenance / overhaul เครื่องกลุ่มนี้ก่อน
3	Volume > 800 ⟶ 96.7% (vs 80.4%)	Trade-off ระหว่างต้นทุนกับคุณภาพ	เพิ่ม QC sampling ใน batch ใหญ่แทนการลดขนาด

#	Insight	Business Meaning	Action
	แต่ต้นทุน/หน่วยต่ำสุด ($13.82)		
4	Defect rate > 3 ⟶ 96.2% · Quality score < 75 ⟶ 95.8%	เป็นตัววัดผล ไม่ใช่คันโยก	ใช้เป็น QC alert หลังผลิต (SPC alert ที่ 3.0, QC gate ที่ QS 75)
5	Supplier, delay, down-time, stockout, produc-tivity, energy ต่างกันไม่เกิน 8.4 pp	ไม่ใช่ต้นเหตุหลักของ defect	ไม่ใช้เป็นคันโยกหลัก ลดงบที่ไม่คุ้ม
6	ไม่มี controllable flag + QC ผ่าน ⟶ 3.12% (417 batch)	มี operating window ที่ปลอดภัย	ใช้ 2 controllable flags เป็น risk score ก่อนผลิต + QC alert หลังผลิต
7	กฎ threshold 4 ข้อตรงกับ label 96.11%	label น่าจะสร้างจากกฎ ไม่ใช่การนับของเสียจริง	ยืนยันนิยาม label กับเจ้าของข้อมูลก่อนตัดสินใจหรือทำโมเดล

8.6 Unknown Question และ Ad-hoc Analysis

Unknown Question คือคำถามที่ธุรกิจยังไม่ได้ถาม แต่ข้อมูลชี้ให้เห็น: batch ที่ defect rate ต่ำสุด (0.5–1) ยังเป็น high-defect ถึง 74.59% แปลว่า is_high_defect ไม่ได้มาจาก defect rate อย่างเดียว จึงตั้งคำถามว่า "label นี้นิยามจากอะไร" และทดสอบสมมติฐานว่า label = 1 เมื่อเกิน threshold ข้อใดข้อหนึ่ง (maintenance > 10 หรือ volume > 800 หรือ defect rate > 3 หรือ quality score < 75)

ตารางที่ 20: กฎ threshold เทียบกับ label is_high_defect (mart_label_rule, vw_label_rule_check)

ผลเทียบ	Batches	% ของ batch
กฎตรงกับ label (0/0 และ 1/1)	3,114	96.11
เกิน threshold แต่ label = 0	113	3.49
label = 1 แต่ไม่เกิน threshold ข้อใดเลย	13	0.40

กฎง่าย ๆ 4 ข้อนี้ตรงกับ label ถึง 96.11% และมีเพียง 13 batch ที่เป็น high-defect โดยไม่เกิน threshold ใดเลย ข้อสรุปคือ is_high_defect น่าจะเป็น label ที่สร้างจากกฎ (rule-based / synthetic) ไม่ใช่การนับของเสียจริง ผลกระทบทางธุรกิจคือ ต้องยืนยันนิยามของ label กับเจ้าของข้อมูลก่อนใช้ตัดสินใจ และถ้าจะทำ predictive model ต้องระวังว่าโมเดลจะเรียนรู้กฎนี้กลับมา (โดยเฉพาะถ้าใส่ defect rate และ quality score เป็น feature)

นอกจากนี้ ข้อมูล Gold ยังตอบคำถามใหม่ได้ทันทีผ่าน SQL บน PostgreSQL หรือ Unity Catalog ตัวอย่างเช่น

```
-- batch ที่ผ่าน QC (defect rate ≤ 3, quality score ≥ 75): high-defect % ต่างกันอย่างไรตามชั่วโมงซ่อมบำรุง
SELECT maintenance_band, COUNT(*) AS batches,
       ROUND(AVG(is_high_defect)*100, 2) AS high_defect_pct
FROM manufacturing_gold_pg.gold.vw_batch_360
WHERE defect_rate <= 3 AND quality_score_pct >= 75
GROUP BY maintenance_band ORDER BY maintenance_band;

-- batch เสี่ยงสูงสุด (watchlist)
SELECT production_id, maintenance_hours_per_week, defect_rate, quality_score_pct, production_volume
FROM predictive_manufacring_defect.gold_predictive.batch_360
WHERE risk_flag_count >= 3 ORDER BY risk_flag_count DESC, defect_rate DESC LIMIT 20;

-- Unknown Question: กฎ threshold ตรงกับ label กี่ %
SELECT * FROM manufacturing_gold_pg.gold.vw_label_rule_check ORDER BY label_rule_pred, is_high_de-
fect;
```

8.7 Predictive Model (Machine Learning)

เพื่อดูว่าข้อมูล Gold นำไปใช้ทำนาย batch ที่จะเป็น high-defect ได้แค่ไหน และตรวจคำตอบของ Unknown Question (label อาจสร้างจากกฎ threshold) จึงสร้าง notebook ML_Predictive ซึ่งเป็น task สุดท้ายของ Workflow โมเดลจึงถูก train ใหม่ทุกครั้งที่ข้อมูลเปลี่ยน

ตารางที่ 21: การตั้งค่าการทดลอง

หัวข้อ	รายละเอียด
Input / Target	gold_predictive.batch_360 (3,240 batch) / is_high_defect
Features	ตัวแปรเชิงตัวเลข 16 ตัว (รวม defect_rate และ quality_score_pct)
การแบ่งข้อมูล	train / test 80 / 20 แบบ stratified, random_state = 42 (test 648 batch)
Baseline	กฎ 4 risk flags: risk_flag_count ≥ 1 (maintenance > 10, volume > 800, defect rate > 3, quality score < 75)
Models	Logistic Regression (StandardScaler, class_weight = balanced) และ Gradient Boosting (200 trees, learning rate 0.05, depth 3)
Metrics	ROC-AUC, F1, Precision, Recall, Balanced accuracy, Confusion matrix (target ไม่สมดุล 84% จึงไม่ดู accuracy อย่างเดียว)
Tracking / Registry	MLflow experiment manufacturing_defect_model; โมเดลที่ ROC-AUC สูงสุดถูก register เป็น gold_predictive.defect_model alias champion

หัวข้อ	รายละเอียด
Output	ตาราง predictions, model_comparison, model_feature_importance และหน้า Model ใน dashboard

ตารางที่ 22: ผลบน test set 648 batch (ML_SUMMARY จาก run ใน Databricks)

Model	ROC-AUC	F1	Precision	Recall	Bal. Acc	TN / FP / FN / TP
Baseline (4 flags)	0.921	0.983	0.971	0.995	0.920	87 / 16 / 3 / 542
Logistic Regression	0.855	0.820	0.949	0.721	0.759	82 / 21 / 152 / 393
Gradient Boosting	0.916	0.981	0.971	0.991	0.918	87 / 16 / 5 / 540

ผลจาก run 772115549894494 ของ Workflow (ML_SUMMARY): Gradient Boosting มี ROC-AUC สูงกว่า Logistic Regression จึงถูก register เป็น gold_predictive.defect_model เวอร์ชัน 18 พร้อม alias champion

ผลที่สำคัญคือ กฎ 4 ข้อ (baseline) ทำได้ดีที่สุดทุกตัววัด Gradient Boosting ทำได้ใกล้เคียงแต่ไม่ดีกว่า ส่วน Logistic Regression แย่กว่า เพราะความสัมพันธ์เป็นแบบขั้นบันได (threshold) ไม่ใช่เส้นตรง feature importance ของ Gradient Boosting 4 อันดับแรกคือ defect_rate, quality_score_pct, maintenance_hours_per_week และ production_volume ซึ่งเป็น 4 ตัวแปรเดียวกับกฎ threshold (อันดับ 5 additive_material_cost_usd มีผลน้อยมาก) ผลนี้ยืนยัน Unknown Question ในหัวข้อ 8.6 ว่า label is_high_defect สร้างจากกฎ threshold ชุดนี้ โมเดลที่ซับซ้อนขึ้นจึงไม่ช่วย

ข้อควรระวัง: defect_rate และ quality_score_pct เป็นตัววัดผลที่รู้หลังผลิตแล้ว (Outcome) การใส่เป็น feature จะทำให้ผลดูดีเกินจริง หากต้องการทำนายก่อนผลิต (label leakage) จึงทดลองเพิ่มใน ML_Predictive (MLflow run gb_pre_production_only, train/test ชุดเดียวกัน) โดยใช้เฉพาะ 14 ตัวแปรที่รู้ก่อนผลิต Gradient Boosting ได้ ROC-AUC 0.805 และ F1 0.910 แต่ balanced accuracy เหลือเพียง 0.529 ใกล้เคียงการเดาสุ่ม เพราะจาก batch ปกติ 103 batch ในชุดทดสอบ โมเดลทายถูกเพียง 8 batch (TN 8, FP 95, FN 11, TP 534) คือแทบแยก batch ปกติไม่ได้ ส่วน F1 ที่สูงมาจากการที่ batch ส่วนใหญ่ (84%) เป็น high-defect อยู่แล้ว ข้อสรุปทางธุรกิจจึงเป็น ใช้กฎ maintenance > 10 และ volume > 800 เป็นด่านก่อนผลิต และใช้ defect rate / quality score เป็น QC alert หลังผลิต ตามหัวข้อ 8.5 ส่วนโมเดลเหมาะใช้เฝ้าดูว่าความสัมพันธ์ในข้อมูลยังเหมือนเดิมเมื่อมี batch ใหม่เข้ามา

บทที่ 9 Data Quality & ETL Testing

ตารางที่ 23: สรุปผลการทดสอบ

หมวด	การทดสอบ	ผล
Schema Validation	header ไฟล์ = 17 คอลัมน์ที่คาดไว้, Silver ตรวจคอลัมน์ต้นทางก่อน rename	PASS
Data Type Validation	try_cast ทุกคอลัมน์, INT ต้องไม่มีเศษ	cast error 0 ⟶ PASS
Missing Value Test	placeholder 10 แบบ + NULL profile	0 ค่า ⟶ PASS
Duplicate Test	record_id SHA-256 + row_number	ซ้ำ 0 ⟶ PASS
DQ Test	DQ rules ทุกข้อ + Delta CHECK 9 ข้อ + quality gate	fail 0 ⟶ PASS
PK/FK Test (Spark)	PK null/ซ้ำ, FK orphan ใน 6 ตาราง	PASS ทุกตาราง
PK/FK Test (PostgreSQL)	orphan 5 ตาราง, row count 6 ตาราง, negative test 3 ข้อ (หัวข้อ 9.1)	14/14 PASS
Row Count Reconciliation	ไฟล์ = Bronze = Silver = Gold = PostgreSQL (หัวข้อ 9.2)	3,240 ทุกชั้น ⟶ PASS
Dirty Data Test	ข้อมูลสกปรก 15 กรณี + reconciliation/lineage 5 ข้อ (หัวข้อ 9.3)	19/19 PASS
End-to-end Automation Test	วางไฟล์ batch ใหม่ 308 แถว ⟶ Job เริ่มเอง ⟶ ตรวจแถวทุกชั้น, quarantine, alert อีเมล และ reset (หัวข้อ 9.4)	PASS

9.1 Negative Test ใน PostgreSQL

ตารางที่ 24: Negative test: ฐานข้อมูลต้องปฏิเสธข้อมูลผิด

Test	คาดหวัง	ผลจริง	Status
INSERT quality ด้วย production_id = -999	Reject (23503)	SQLSTATE 23503 foreign_key_violation	PASS
INSERT production ด้วย production_id = 1 (ซ้ำ)	Reject (23505)	SQLSTATE 23505 unique_violation	PASS
INSERT quality ด้วย is_high_defect = 5	Reject (23514/23505)	ถูกปฏิเสธ	PASS

9.2 Final Row Count Reconciliation

ตารางที่ 25: จำนวนแถวแต่ละขั้น

Stage	Row Count
Source file (data lines)	3,240
Bronze v2	3,240
After Transformation (Silver)	3,240
Duplicate Removed	0
Quarantine	0
Valid Silver	3,240
Gold production / supply_chain / quality	3,240 / 3,240 / 3,240
Gold maintenance / inventory / operations	3,240 / 3,240 / 3,240
PostgreSQL (6 ตาราง)	3,240 ทุกตาราง
gold.batch_360	3,240

9.3 Dirty Data Test (ทดสอบกลไก Data Quality)

ข้อมูลต้นทางสะอาด จึงไม่มีแถวใดเข้า quarantine ใน production run และยังพิสูจน์ไม่ได้ว่า rule ทำงานจริง notebook DQ_Test_Dirty_Data จึงสร้างไฟล์ทดสอบจากแถวจริง 200 แถว แล้วเพิ่มแถวผิด 15 กรณี (แต่ละกรณีแก้ค่าเพียง 1 จุด) จากนั้นเรียก notebook Bronze_Predictive_v2 และ Silver_Predictive_v2 ตัวจริงผ่าน dbutils.notebook.run โดยส่ง parameter ให้เขียนลง schema dq_test จึงไม่ทับตารางจริง แล้วตรวจผลทีละกรณีเทียบกับที่คาดไว้ และบันทึกลงตาราง dq_test.test_results

ตารางที่ 26: ผลทดสอบกลไก Data Quality ด้วยข้อมูลสกปรก (dq_test.test_results)

Case	ข้อมูลที่ฉีดเข้าไป	ผลที่คาด / ผลจริง	Status
T01	placeholder 'N/A' ใน DefectRate (คอลัมน์บังคับ)	quarantine: required value is NULL	PASS
T02	ค่าว่างใน DefectStatus (target)	quarantine: required value is NULL	PASS
T03	ข้อความ 'abc' ใน ProductionCost	quarantine: invalid value for double	PASS
T04	ทศนิยม 12.5 ใน MaintenanceHours (INT)	quarantine: non-integer value for int	PASS
T05	DefectStatus = 2	quarantine: value must be 0 or 1	PASS
T06	QualityScore = 150	quarantine: between 0 and 100	PASS

Case	ข้อมูลที่ฉีดเข้าไป	ผลที่คาด / ผลจริง	Status
T07	StockoutRate = 5	quarantine: between 0 and 1	PASS
T08	ProductionVolume = -10	quarantine: cannot be negative	PASS
T09	MaintenanceHours = 200	quarantine: cannot exceed 168	PASS
T10	แถวซ้ำกับแถวสะอาดทุกคอลัมน์	quarantine: exact duplicate	PASS
T11	placeholder 'N/A' ใน EnergyEfficiency (ไม่บังคับ)	valid, ค่าเป็น NULL	PASS
T12	ช่องว่างหน้า/หลังตัวเลข ' 512 '	valid, trim เป็น 512	PASS
T13	defect_rate 4.8 แต่ label = 0	valid + business warning	PASS
T14–T15	แถวคอลัมน์เกิน / ขาด	Bronze corrupt 2 ⟶ quarantine 2	PASS
R1	Bronze rows = บรรทัดในไฟล์	215 = 215	PASS
R2	Silver valid rows	203	PASS
R3	Quarantine rows	12	PASS
R4	valid + quarantine = Bronze	203 + 12 = 215	PASS
R5	Lineage ครบทุกแถว (รวม source_file_modified_at)	0 แถวที่ว่าง	PASS

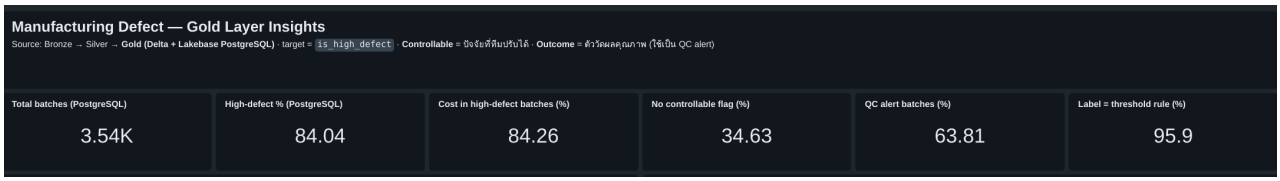
Row trace ของรอบทดสอบ: Bronze 215 แถว ⟶ แยกแถว format เสีย 2 แถว เหลือ 213 ⟶ dedup เหลือ 212 ⟶ valid 203 และไม่ผ่าน DQ 9 แถว ⟶ quarantine รวม 12 แถว (DQ 9 + ซ้ำ 1 + format เสีย 2) และ valid + quarantine = 215 ตรงกับ Bronze ผลทดสอบผ่านครบ 19/19 รายการ แสดงว่ากลไก missing value, type enforcement, range rules, deduplication, quarantine, business warning และ lineage ทำงานจริง

9.4 End-to-End Automation Test

ทดสอบว่า pipeline แบบ Automate ทำงานถูกต้องตั้งแต่ไฟล์มาถึงจนถึง dashboard โดยใช้ไฟล์ batch_002 จาก Demo_Drop_New_Batch ซึ่งรู้ผลที่ควรเกิดล่วงหน้า แล้วเทียบกับผลจริงหลัง Job จบ

ตารางที่ 27: ผลทดสอบ end-to-end ด้วยไฟล์ batch_002 (308 แถว)

รายการตรวจ	ผลที่คาด	ผลจริง	Status
Trigger	Job เริ่มเองเมื่อมีไฟล์ใหม่	Launched: By file arrival (10:13)	PASS
Auto Loader อ่านเฉพาะไฟล์ใหม่	อ่าน batch_002 เท่านั้น 308 แถว	new_rows = 308, ingestion_log PASS	PASS
Bronze	3,240 + 308 = 3,548	3,548	PASS
Silver valid	3,240 + 300 = 3,540	3,540	PASS
Quarantine	ซ้ำ 5 + ค่าผิด 3 = 8	8	PASS
valid + quarantine = Bronze	3,540 + 8 = 3,548	ตรงกัน	PASS
Gold / PostgreSQL / Dashboard	อัปเดตเป็น 3,540 batch	Dashboard แสดง 3.54K batches	PASS
SQL Alert quality gate	TRIGGERED เพราะมี quarantine	อีเมล TRIGGERED 10:36	PASS
Reset (full refresh)	กลับเป็น 3,240 / 3,240 / 0	3,240 / 3,240 / 0	PASS



ภาพที่ 26: Dashboard หลัง Job ประมวลผล batch_002: total batches 3.54K (ตัวเลขทั้งหมดคำนวณใหม่อัตโนมัติ)

ผลทดสอบแสดงว่า Workflow ไม่ได้แค่รัน notebook ต่อกัน แต่กลไก Data Quality ทำงานกับข้อมูลที่เข้ามาใหม่จริง แถวซ้ำข้าม batch ถูกจับได้จาก record_id ข้อมูลไม่หาย (reconciliation ตรงทุกชั้น) และผู้รับผิดชอบได้รับอีเมลแจ้งเตือนโดยอัตโนมัติ

บทที่ 10 ผลลัพธ์ของโครงการ

Bronze	⟶	Silver	⟶	Gold	⟶	RDBMS (Lakebase PostgreSQL)	⟶	Dashboard	⟶	Insight
3,240		3,240		6×3,240		6×3,240 + 5 views		17 widgets		9 คำถาม

- ETL Pipeline 7 notebook ที่รันจริงบน Databricks ทำงานแบบ Automate ด้วย Databricks Workflow (file arrival trigger) และรันซ้ำได้โดยข้อมูลไม่ซ้ำ พร้อม notebook ทดสอบกลไก Data Quality
- ข้อมูลไม่สูญหายระหว่างชั้น (3,240 แถวทุกชั้น) พร้อม lineage ย้อนถึงไฟล์ต้นทาง
- โครงสร้าง RDBMS 6 ตาราง PK/FK ที่ฐานข้อมูลบังคับจริง ผ่าน integrity test 14/14
- Analytic marts 10 ตาราง และ PostgreSQL views 5 ตัว สำหรับ BI และ ad-hoc query
- AI/BI Dashboard ที่ publish แล้ว ใช้ข้อมูลทั้งจาก Delta และ PostgreSQL
- Business Insight ที่แยกปัจจัยที่จัดการได้ (maintenance, volume) ออกจากตัววัดผล (defect rate, quality score) พร้อม operating window และ Unknown Question เรื่องนิยามของ label
- กลไก Data Quality ผ่านการทดสอบด้วยข้อมูลสกปรก 19/19 รายการ
- Workflow 9 task สร้างด้วยโค้ด ทดสอบ end-to-end ด้วยไฟล์ใหม่สำเร็จใน 22 นาที 33 วินาที พร้อม SQL Alert และอีเมลแจ้งเตือน
- โมเดลทำนาย high-defect ใน MLflow / Unity Catalog ที่ยืนยันว่า label สร้างจากกฎ threshold

ตารางที่ 28: สิ่งที่ส่งมอบ

ไฟล์ / ออบเจกต์	**รายละเอียด**
Bronze_Predictive_v2	notebook Bronze (ไฟล์ใหม่)
Silver_Predictive_v2	notebook Silver (ไฟล์ใหม่)
Gold_Predictive	notebook Gold: Delta + Lakebase + integrity test
Dashboard_Predictive	notebook สร้าง/อัปเดต/publish dashboard
Manufacturing Defect - Gold Insights	AI/BI Dashboard (published)
manufacturing-defect-gold	Lakebase project (PostgreSQL 17)
DQ_Test_Dirty_Data	notebook ทดสอบกลไก Data Quality ด้วยข้อมูลสกปรก (schema dq_test)

ไฟล์ / ออบเจกต์	รายละเอียด
Bronze_AutoLoader	notebook Bronze แบบ incremental (Auto Loader) ใช้ใน Work-flow
ML_Predictive	notebook โมเดลทำนาย high-defect + MLflow + หน้า Model
Setup_Workflow	notebook สร้าง/อัปเดต Job ด้วย Jobs API 2.2
Manufacturing_Defect_Pipeline	Databricks Job 9 task (file arrival trigger, alert, email)
High-defect % above target, Quality gate failed	SQL Alerts 2 ตัว (แจ้งเตือนทางอีเมล)
Demo_Drop_New_Batch / Re-set_Demo	notebook สาธิตการรับไฟล์ใหม่ และคืนสถานะข้อมูลเป็น 3,240 แถว
gold_predictive.defect_model	โมเดลใน Unity Catalog (alias champion)

บทที่ 11 สรุปและข้อเสนอแนะ

11.1 สรุปผล

โครงงานสร้าง Data Pipeline แบบ Medallion บน Databricks ได้ครบ ตั้งแต่ข้อมูลดิบจนถึง Business Insight โดยแก้จุดที่ขัดหลักการของ notebook เดิม (Bronze infer type, Silver ไม่มี key และ rule บางข้อไม่ตรงกับช่วงค่าจริง) พิสูจน์กลไก Data Quality ด้วยข้อมูลสกปรก เพิ่มชั้น RDBMS จริงบน PostgreSQL ที่บังคับ PK/FK และแสดงผลบน Dashboard ผลการวิเคราะห์ชี้ว่า ปัจจัยที่ทีมจัดการได้และมีผลชัด คือชั่วโมงซ่อมบำรุง (> 10) และขนาด batch (> 800) ส่วน defect rate และ quality score เป็นตัววัดผลที่เหมาะกับการใช้เป็น QC alert และพบว่า label is_high_defect น่าจะสร้างจากกฎ threshold (ตรงกัน 96.11%) นอกจากนี้ ได้พัฒนา pipeline เป็นแบบ Automate ด้วย Databricks Workflow ที่ทำงานเองเมื่อมีไฟล์ใหม่ ทดสอบจริงด้วยไฟล์ batch ใหม่ ได้ผลถูกต้องทุกชั้น พร้อมอีเมลแจ้งเตือน และสร้างโมเดล Machine Learning ซึ่งยืนยันว่า กฎ threshold 4 ข้ออธิบาย label ได้ดีกว่าโมเดล

11.2 ปัญหาและข้อจำกัด

- Dataset ไม่มี business key จริง (เช่น batch id, วันที่, สายการผลิต) จึงต้องสร้าง key จาก hash ของค่าธุรกิจ และวิเคราะห์เชิงเวลาไม่ได้
- ข้อมูลมีลักษณะคล้าย synthetic data และ label is_high_defect ตรงกับกฎ threshold ถึง 96.11% จึงอาจไม่ได้สะท้อนของเสียจริง ข้อสรุปเป็นความสัมพันธ์ ไม่ใช่เหตุและผล
- Threshold ได้จากการ scan หลายจุด (multiple testing) ตัวแปรที่ผลต่างน้อยอาจดูสูงเกินจริง จึงตั้งเกณฑ์ key factor ไว้ที่ 15 pp และควรยืนยันกับข้อมูลชุดใหม่
- Target ไม่สมดุล (84% high defect) ต้องระวังเมื่อนำไปทำ predictive model
- บาง business rule (เช่น threshold ของ warning) ต้องอาศัย domain knowledge เพิ่ม
- บน Free Edition มีโควตา serverless compute จำกัด notebook ลูกของ DQ test จึงมี cold start นาน (10–13 นาที) และ SQL Warehouse อาจเปิดไม่ได้ (RESOURCE_EXHAUSTED) ทำให้ alert task ล้มได้ (แก้ด้วย retry และปิด session ที่ไม่ใช้) และ driver psycopg ใช้บน serverless ไม่ได้ ต้องใช้ pg8000
- Silver และ Gold คำนวณใหม่ทั้งตารางทุก batch (full recompute) เหมาะกับข้อมูลขนาดเล็ก แต่จะช้าลงเมื่อข้อมูลโตขึ้น
- โมเดลที่ใช้ defect rate / quality score เป็น feature มี label leakage ถ้าต้องการทำนายก่อนผลิต

11.3 แนวทางในอนาคต

- แยก dq_test เป็น Job ที่รันตามตาราง (เช่น วันละครั้ง) หรือเมื่อแก้โค้ด เพื่อให้ Job หลักเหลือประมาณ 10 นาทีต่อ batch
- เปลี่ยน Silver และ Gold เป็น incremental MERGE เมื่อข้อมูลมีขนาดใหญ่ขึ้น และจัดการ Job/โค้ดด้วย Databricks Asset Bundles (CI/CD)
- เก็บข้อมูลที่รู้ก่อนผลิตเพิ่ม (เช่น สายการผลิต วันที่ วัตถุดิบ) แล้วสร้างโมเดลจากตัวแปร Controllable เท่านั้น พร้อม model monitoring เพื่อดู drift เมื่อมี batch ใหม่
- ตั้ง schedule refresh ของ Dashboard และแจ้งเตือนเมื่อ high-defect % เกินเป้า

ภาคผนวก

ภาคผนวกนี้รวม source code ของ notebook ที่ใช้งานจริง (Databricks source format) ไฟล์เดียวกับที่ import เข้า workspace รวมถึง notebook ชุด Automation และ ML

Appendix A: Bronze Code

Bronze_Predictive_v2.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Bronze_Predictive_v2 — Manufacturing Defect Dataset (Bronze Layer)
# MAGIC
# MAGIC **สร้างใหม่จาก `Bronze_Predictive` (ไม่แก้ไฟล์เดิม)** — เขียนลงตารางใหม่ `..._v2` เพื่อให้ย้อนตรวจสอบตารางเดิมได้
# MAGIC
# MAGIC | # | ประเด็นใน notebook เดิม | หลักการ Bronze | สิ่งที่แก้ใน v2 |
# MAGIC |---|---|---|---|
# MAGIC | 1 | ใช้ `inferSchema=true` ตอนเขียนตาราง | Bronze = เก็บข้อมูล **ตีบตามต้นฉบับ** ไม่ตีความชนิดข้อมูล (type enforcement เป็นหน้าที่ Silver) | อ่านทุกคอลัมน์เป็น `STRING` ด้วย schema ที่ประกาศชัดเจน |
# MAGIC | 2 | ไม่มี ingestion metadata | Bronze ต้องตอบได้ว่าข้อมูลมาจากไฟล์ไหน เข้ามาเมื่อไร รอบไหน | เพิ่ม `_source_file`, `_source_file_modified_at`, `_ingested_at`, `_bronze_batch_id` |
# MAGIC | 3 | ไม่ตรวจ header/schema ของไฟล์ | ต้องจับ schema drift ตั้งแต่ทางเข้า | ตรวจ header เทียบ `EXPECTED_COLUMNS` ก่อนโหลด |
# MAGIC | 4 | แถวที่ format เสียจะหายเงียบ ๆ | ต้องเก็บ record ที่อ่านไม่ได้ไว้ตรวจสอบ | ใช้ `PERMISSIVE` + `_corrupt_record` |
# MAGIC | 5 | ไม่มี reconciliation | จำนวนแถวต้องตรงกับไฟล์ต้นทาง | นับบรรทัดในไฟล์เทียบกับจำนวนแถวในตาราง |
# MAGIC | 6 | ไม่มี log การโหลด | ETL Monitoring | append ผลทุกรอบลง `ingestion_log` |
# MAGIC | 7 | cell 1 และ cell 3 อ่านไฟล์ซ้ำกัน 2 รอบ ด้วย option ต่างกัน | 1 source ⟶ 1 วิธีอ่าน | อ่านครั้งเดียวด้วย reader function เดียว |
# MAGIC | 8 | ไม่มี table/column comment | Data catalog / data dictionary | เพิ่ม `COMMENT` ให้ตาราง |
# MAGIC | 9 | (v2.1) path/ตารางถูก hard-code | ต้องทดสอบกับข้อมูลสกปรกได้โดยไม่ทับตารางจริง | รับ **parameter ผ่าน widgets** (ค่า default = ตารางจริง) ⟶ `DQ_Test_Dirty_Data` เรียกใช้ notebook นี้ได้ |


# COMMAND ----------


# MAGIC %md
# MAGIC ## 0. Config (widgets: ค่า default = production)
# MAGIC รันตามปกติ (Run all) จะใช้ค่า default ทั้งหมด · notebook ทดสอบส่งค่าใหม่มาผ่าน `dbutils.notebook.run(...)`


# COMMAND ----------


import os
import uuid
from datetime import datetime, timezone
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType


CATALOG       = "predictive_manufacring_defect"      # ใช้ชื่อเดิมใน Unity Catalog (สะกดตามของเดิม)
BRONZE_SCHEMA = f"{CATALOG}.bronze_preditive"


dbutils.widgets.text("source_file",  "/Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/manufacturing_defect_dataset.csv")
dbutils.widgets.text("bronze_table", f"{BRONZE_SCHEMA}.manufacturing_defect_dataset_v2")
dbutils.widgets.text("ingest_log",   f"{BRONZE_SCHEMA}.ingestion_log")


SOURCE_FILE  = dbutils.widgets.get("source_file")
VOLUME_DIR   = os.path.dirname(SOURCE_FILE)
BRONZE_TABLE = dbutils.widgets.get("bronze_table")
INGEST_LOG   = dbutils.widgets.get("ingest_log")


# คอลัมน์ที่ต้องมีในไฟล์ต้นทาง (ตามลำดับ)
EXPECTED_COLUMNS = [
    "ProductionVolume", "ProductionCost", "SupplierQuality", "DeliveryDelay",
    "DefectRate", "QualityScore", "MaintenanceHours", "DowntimePercentage",
    "InventoryTurnover", "StockoutRate", "WorkerProductivity", "SafetyIncidents",
    "EnergyConsumption", "EnergyEfficiency", "AdditiveProcessTime",
    "AdditiveMaterialCost", "DefectStatus",
]


bronze_batch_id = str(uuid.uuid4())
run_started_at  = datetime.now(timezone.utc)
print("Source file    :", SOURCE_FILE)
```

```
print("Bronze table   :", BRONZE_TABLE)
print("Bronze batch id:", bronze_batch_id)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. ตรวจไฟล์ต้นทาง (existence + header / schema drift)


# COMMAND ----------


files = [f for f in dbutils.fs.ls(VOLUME_DIR) if f.name.endswith(".csv")]
display(spark.createDataFrame([(f.name, f.size, f.modificationTime) for f in files],
                              "file_name string, size_bytes long, modification_time_ms long"))


if os.path.basename(SOURCE_FILE) not in [f.name for f in files]:
    raise FileNotFoundError(f"ไม่พบไฟล์ต้นทาง: {SOURCE_FILE}")


# อ่านเฉพาะบรรทัด header แบบ text (ไม่ตีความข้อมูล)
raw_lines    = spark.read.text(SOURCE_FILE)
header_line  = raw_lines.first()["value"].lstrip("").strip()
file_columns = [c.strip() for c in header_line.split(",")]


missing_cols    = [c for c in EXPECTED_COLUMNS if c not in file_columns]
unexpected_cols = [c for c in file_columns if c not in EXPECTED_COLUMNS]
print("Header columns :", len(file_columns))
print("Missing        :", missing_cols)
print("Unexpected     :", unexpected_cols)


if missing_cols:
    raise ValueError(f"Schema drift: ไฟล์ขาดคอลัมน์ {missing_cols}")
if file_columns != EXPECTED_COLUMNS:
    print("WARNING: ลำดับ/ชุดคอลัมน์ต่างจากที่คาดไว้ — จะอ่านตามชื่อ header ของไฟล์")


# จำนวนแถวข้อมูลในไฟล์ (ไม่นับ header และบรรทัดว่าง) ใช้ทำ reconciliation
source_line_count = raw_lines.filter(F.length(F.trim("value")) > 0).count() - 1
print("Data lines in source file:", source_line_count)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. Extract — อ่านเป็น STRING ทั้งหมด (raw fidelity) + เก็บแถวที่เสีย


# COMMAND ----------


raw_schema = StructType(
    [StructField(c, StringType(), True) for c in file_columns]
    + [StructField("_corrupt_record", StringType(), True)]
)


df_raw = (
    spark.read.format("csv")
    .option("header", "true")
    .option("mode", "PERMISSIVE")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .option("encoding", "UTF-8")
    .schema(raw_schema)
    .load(SOURCE_FILE)
)


# COMMAND ----------


# MAGIC %md
```

```
# MAGIC ## 3. เพิ่ม Ingestion Metadata (lineage ระดับไฟล์)


# COMMAND ----------


df_bronze = (
    df_raw
    .withColumn("_source_file",              F.col("_metadata.file_path"))
    .withColumn("_source_file_modified_at", F.col("_metadata.file_modification_time"))
    .withColumn("_ingested_at",              F.current_timestamp())
    .withColumn("_bronze_batch_id",          F.lit(bronze_batch_id))
)


df_bronze.printSchema()
display(df_bronze.limit(5))


# COMMAND ----------


# MAGIC %md
# MAGIC ## 4. Load — เขียน Delta Table (overwrite = idempotent, รันซ้ำไม่ซ้ำข้อมูล)


# COMMAND ----------


spark.sql(f"CREATE SCHEMA IF NOT EXISTS {BRONZE_TABLE.rsplit('.', 1)[0]}")
(
    df_bronze.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .saveAsTable(BRONZE_TABLE)
)


spark.sql(f"""
COMMENT ON TABLE {BRONZE_TABLE} IS
'Bronze (raw) copy of {os.path.basename(SOURCE_FILE)}. All business columns stored as STRING exactly as in source. Metadata: _source_file, _source_file_modified_at, _ingested_at, _bronze_batch_id. Malformed
rows kept in _corrupt_record.'
""")
print("Written:", BRONZE_TABLE)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 5. Reconciliation & Bronze Profiling
# MAGIC (อ่านกลับจาก Delta table — การนับ `_corrupt_record` ตรงจากไฟล์ CSV ไม่ได้รับอนุญาตใน Spark)


# COMMAND ----------


bronze_tbl   = spark.table(BRONZE_TABLE)
bronze_rows  = bronze_tbl.count()
corrupt_rows = bronze_tbl.filter(F.col("_corrupt_record").isNotNull()).count()


print(f"Source data lines : {source_line_count}")
print(f"Bronze rows       : {bronze_rows}")
print(f"Corrupt rows      : {corrupt_rows}")


reconcile_status = "PASS" if bronze_rows == source_line_count else "FAIL"
print("Reconciliation    :", reconcile_status)


# Profiling: NULL / empty-string ต่อคอลัมน์ (ยังไม่แก้ — แค่รายงาน, การแก้อยู่ใน Silver)
profile = bronze_tbl.select(
    [F.sum(F.when(F.col(c).isNull() | (F.trim(F.col(c)) == ""), 1).otherwise(0)).alias(c) for c in file_columns]
)
print("NULL/empty values per column (raw):")
display(profile)
```

```
print("Distinct values of DefectStatus (raw):")
display(bronze_tbl.groupBy("DefectStatus").count().orderBy("DefectStatus"))


if corrupt_rows > 0:
    display(bronze_tbl.filter(F.col("_corrupt_record").isNotNull()).select("_corrupt_record").limit(20))


# COMMAND ----------


# MAGIC %md
# MAGIC ## 6. Ingestion Log (ETL Monitoring)


# COMMAND ----------


log_row = [(
    bronze_batch_id, SOURCE_FILE, BRONZE_TABLE,
    int(source_line_count), int(bronze_rows), int(corrupt_rows),
    reconcile_status, run_started_at, datetime.now(timezone.utc),
)]
log_schema = """batch_id string, source_file string, target_table string,
                source_rows long, loaded_rows long, corrupt_rows long,
                status string, started_at timestamp, finished_at timestamp"""


spark.createDataFrame(log_row, log_schema).write.format("delta").mode("append").saveAsTable(INGEST_LOG)
display(spark.table(INGEST_LOG).orderBy(F.desc("finished_at")).limit(10))


if reconcile_status != "PASS":
    raise ValueError("Bronze reconciliation failed: จำนวนแถวไม่ตรงกับไฟล์ต้นทาง")
print("Bronze layer: PASS")
dbutils.notebook.exit(f"PASS|rows={bronze_rows}|corrupt={corrupt_rows}")
```

Appendix B: Silver Transformation Code

Silver_Predictive_v2.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Silver_Predictive_v2 — Manufacturing Defect Dataset (Silver Layer)
# MAGIC
# MAGIC **สร้างใหม่ต่อยอดจาก `Test3_Silver` (ไม่แก้ไฟล์เดิม)** — อ่านจาก `bronze_preditive.manufacturing_defect_dataset_v2` และเขียนลงตาราง `..._v2`
# MAGIC
# MAGIC โครงหลักของ Test3 ถูกต้องตามหลัก Silver แล้ว (rename → normalize missing → type enforcement → dedup → DQ rules → quarantine → lineage → quality gate)
# MAGIC ส่วนที่ **เสริม/แก้** ใน v2:
# MAGIC
# MAGIC | # | ประเด็นใน Test3 | ผลกระทบ | แก้ใน v2 |
# MAGIC |---|---|---|---|
# MAGIC | 1 | Bronze เติมถูก `inferSchema` มาแล้ว → ชั้น 2 (placeholder→NULL) และ 3 (try_cast) **ไม่เคยทำงานจริง** ("No string columns found") | Silver ไม่ได้พิสูจน์ว่าจัดการข้อมูลสกปรกได้ | Bronze v2 เก็บเป็น STRING → ทุกชั้นทำงานจริง (พิสูจน์ด้วย `DQ_Test_Dirty_Data`) |
# MAGIC | 2 | `StockoutRate` ค่าจริง 0–0.10 (สัดส่วน) แต่ตั้งชื่อ `stockout_pct` และเช็คช่วง 0–100 | ชื่อสื่อความผิด + rule หลวมเกินไป 1000 เท่า | เปลี่ยนชื่อเป็น `stockout_rate` และเช็คช่วง 0–1 |
# MAGIC | 3 | `EnergyEfficiency` (0.1–0.5) เช็คแค่ ≥ 0 | ค่าผิด เช่น 45 ผ่านได้ | เช็คช่วง 0–1 |
# MAGIC | 4 | `maintenance_hours_per_week` ไม่มีเพดาน | 1 สัปดาห์มีแค่ 168 ชม. | เพิ่ม rule ≤ 168 |
# MAGIC | 5 | ไม่มี key ประจำแถว | Gold ต้องมี PK/FK ตามแผน RDBMS | เพิ่ม `record_id` = SHA-256 ของค่าธุรกิจ (deterministic, รันซ้ำได้ค่าเดิม) |
# MAGIC | 6 | แถวซ้ำถูก `dropDuplicates` ทิ้งเงียบ ๆ | ตรวจย้อนไม่ได้ว่าอะไรถูกตัด | ส่งแถวซ้ำเข้า quarantine พร้อมเหตุผล `exact duplicate` |
# MAGIC | 7 | lineage มีแค่ `source_table` | ย้อนไปถึงไฟล์/รอบ Bronze ไม่ได้ | ส่งต่อ `_source_file`, `_source_file_modified_at`, `_bronze_batch_id`, `_ingested_at` |
# MAGIC | 8 | ไม่มี constraint บนตาราง | ข้อมูลผิดอาจถูกเขียนทับภายหลังได้ | เพิ่ม Delta `CHECK` constraints + table comment |
# MAGIC | 9 | ไม่มี run log | ETL Monitoring (บทที่ 7.4 ในแผนรายงาน) | append row trace ลง `silver_predictive.dq_run_log` |
# MAGIC | 10 | ไม่มี profiling หลัง clean | ใช้เขียนบท Data Understanding | เพิ่ม summary statistics + class balance ของ target |
# MAGIC | 11 | (v2.1) ตารางถูก hard-code | ทดสอบข้อมูลสกปรกไม่ได้โดยไม่ทับตารางจริง | รับ parameter ผ่าน widgets (default = ตารางจริง) |


# COMMAND ----------


# MAGIC %md
# MAGIC # 0. INIT (widgets: ค่า default = production)


# COMMAND ----------


import uuid
from datetime import datetime, timezone
import pyspark.sql.functions as F
from pyspark.sql.window import Window


CATALOG       = "predictive_manufacring_defect"
SILVER_SCHEMA = f"{CATALOG}.silver_predictive"


dbutils.widgets.text("source_table",     f"{CATALOG}.bronze_preditive.manufacturing_defect_dataset_v2")
dbutils.widgets.text("silver_table",     f"{SILVER_SCHEMA}.manufacturing_defect_dataset_v2")
dbutils.widgets.text("quarantine_table", f"{SILVER_SCHEMA}.manufacturing_defect_dataset_quarantine_v2")
dbutils.widgets.text("dq_run_log",       f"{SILVER_SCHEMA}.dq_run_log")


SOURCE_TABLE     = dbutils.widgets.get("source_table")
SILVER_TABLE     = dbutils.widgets.get("silver_table")
QUARANTINE_TABLE = dbutils.widgets.get("quarantine_table")
DQ_RUN_LOG       = dbutils.widgets.get("dq_run_log")


WRITE_MODE = "overwrite"
MISSING_VALUES = ["", "null", "none", "nan", "n/a", "na", "-", "?", "missing", "undefined"]


silver_batch_id = str(uuid.uuid4())
run_started_at  = datetime.now(timezone.utc)


ROW_TRACE = []
def trace(step, df):
    n = df.count()
    ROW_TRACE.append((step, n))
    print(f"[row trace] {step:<38} {n:>8}")
    return n


df_silver = spark.table(SOURCE_TABLE)
```

```
print(f"Source table: {SOURCE_TABLE}")
print(f"Silver table: {SILVER_TABLE}")
bronze_count = trace("0. bronze", df_silver)


# แถวที่ Bronze อ่านไม่ได้ (malformed) ⟶ quarantine ทันที
bronze_corrupt = df_silver.filter(F.col("_corrupt_record").isNotNull())
df_silver      = df_silver.filter(F.col("_corrupt_record").isNull()).drop("_corrupt_record")
corrupt_count  = bronze_corrupt.count()
print("Malformed rows from Bronze:", corrupt_count)
df_silver.printSchema()


# COMMAND ----------


# MAGIC %md
# MAGIC # 1. Column Name Standardization


# COMMAND ----------


RENAME_MAP = {
    # 1. Production
    "ProductionVolume": "production_volume",
    "ProductionCost": "production_cost",
    # 2. Supply Chain
    "SupplierQuality": "supplier_quality_pct",
    "DeliveryDelay": "delivery_delay_days",
    # 3. Quality
    "DefectRate": "defect_rate",
    "QualityScore": "quality_score_pct",
    # 4. Maintenance
    "MaintenanceHours": "maintenance_hours_per_week",
    "DowntimePercentage": "downtime_pct",
    # 5. Inventory
    "InventoryTurnover": "inventory_turnover_ratio",
    "StockoutRate": "stockout_rate",                # v2: สัดส่วน 0–1 ไม่ใช่ %
    # 6. Workforce & Safety
    "WorkerProductivity": "worker_productivity_pct",
    "SafetyIncidents": "safety_incidents_count",
    # 7. Energy
    "EnergyConsumption": "energy_consumption_kwh",
    "EnergyEfficiency": "energy_efficiency_factor",
    # 8. Additive Manufacturing
    "AdditiveProcessTime": "additive_process_time_hrs",
    "AdditiveMaterialCost": "additive_material_cost_usd",
    # 9. Target
    "DefectStatus": "is_high_defect",
}


# v2.1: ส่งต่อ _source_file_modified_at ด้วย (เดิมหายไประหว่าง Bronze ⟶ Silver)
BRONZE_META = {
    "_source_file": "source_file",
    "_source_file_modified_at": "source_file_modified_at",
    "_ingested_at": "bronze_ingested_at",
    "_bronze_batch_id": "bronze_batch_id",
}


missing_source_cols = sorted(set(RENAME_MAP) - set(df_silver.columns))
if missing_source_cols:
    raise ValueError("Missing required source columns: " + ", ".join(missing_source_cols))
print("Source schema validation: PASS")


df_silver = df_silver.select(
    *[F.col(o).alias(n) for o, n in RENAME_MAP.items()],
    *[F.col(o).alias(n) for o, n in BRONZE_META.items()],
)
BUSINESS_COLUMNS = list(RENAME_MAP.values())
print(df_silver.columns)
trace("1. after rename", df_silver)


# COMMAND ----------
```

```
# MAGIC %md
# MAGIC # 2. Missing Value Normalization
# MAGIC Placeholder (เช่น "N/A", "null", "") ⟶ NULL — ใน v2 ขั้นนี้ทำงานจริงเพราะ Bronze เป็น STRING


# COMMAND ----------


string_cols = [c for c, t in df_silver.dtypes if t == "string" and c in BUSINESS_COLUMNS]
print("String business columns:", len(string_cols))


placeholder_counts = df_silver.select([
    F.sum(F.when(F.lower(F.trim(F.col(c))).isin(MISSING_VALUES), 1).otherwise(0)).alias(c)
    for c in string_cols
])
print("Placeholder missing values BEFORE normalization:")
display(placeholder_counts)


for c in string_cols:
    cleaned = F.trim(F.col(c))
    df_silver = df_silver.withColumn(
        c, F.when(F.lower(cleaned).isin(MISSING_VALUES), F.lit(None)).otherwise(cleaned)
    )


print("NULL count AFTER normalization:")
display(df_silver.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in BUSINESS_COLUMNS]))
trace("2. after missing normalization", df_silver)


# COMMAND ----------


# MAGIC %md
# MAGIC # 3. Data Type Enforcement
# MAGIC - `try_cast` ⟶ ค่าที่แปลงไม่ได้เป็น NULL (ไม่ throw ใน ANSI mode) แล้ว flag ไว้
# MAGIC - integer: cast ผ่าน double (รองรับ "12.0") และ flag ถ้ามีเศษทศนิยม


# COMMAND ----------


EXPECTED_TYPES = {
    "production_volume": "int",
    "production_cost": "double",
    "supplier_quality_pct": "double",
    "delivery_delay_days": "int",
    "defect_rate": "double",
    "quality_score_pct": "double",
    "maintenance_hours_per_week": "int",
    "downtime_pct": "double",
    "inventory_turnover_ratio": "double",
    "stockout_rate": "double",
    "worker_productivity_pct": "double",
    "safety_incidents_count": "int",
    "energy_consumption_kwh": "double",
    "energy_efficiency_factor": "double",
    "additive_process_time_hrs": "double",
    "additive_material_cost_usd": "double",
    "is_high_defect": "int",
}


def try_cast(col_name, target_type):
    return F.expr(f"try_cast(`{col_name}` AS {target_type})")


cast_error_expressions = []
for c, target_type in EXPECTED_TYPES.items():
    as_double = try_cast(c, "double")
    cast_error_expressions.append(
        F.when(F.col(c).isNotNull() & as_double.isNull(), F.lit(f"{c}: invalid value for {target_type}"))
    )
    if target_type == "int":
        cast_error_expressions.append(
            F.when(as_double.isNotNull() & ~F.isnan(as_double) & (as_double != F.floor(as_double)),
                   F.lit(f"{c}: non-integer value for int"))
```

```
        )


df_silver = df_silver.withColumn("_dq_cast_errors", F.array_compact(F.array(*cast_error_expressions)))


for c, target_type in EXPECTED_TYPES.items():
    if target_type == "int":
        df_silver = df_silver.withColumn(c, try_cast(c, "double").cast("int"))
    else:
        df_silver = df_silver.withColumn(c, try_cast(c, target_type))


print("Rows with casting errors:", df_silver.filter(F.size("_dq_cast_errors") > 0).count())
df_silver.printSchema()
trace("3. after type enforcement", df_silver)


# COMMAND ----------


# MAGIC %md
# MAGIC # 4. Record Key + Deduplication
# MAGIC - `record_id` = SHA-256 ของค่าธุรกิจทั้ง 17 คอลัมน์ ⟶ ใช้เป็น natural key ส่งต่อให้ Gold (PK/FK)
# MAGIC - แถวซ้ำ (record_id เดียวกัน) เก็บไว้ 1 แถว ที่เหลือส่ง quarantine พร้อมเหตุผล (ไม่ทิ้งเงียบ)


# COMMAND ----------


df_silver = df_silver.withColumn(
    "record_id",
    F.sha2(F.concat_ws("|", *[F.coalesce(F.col(c).cast("string"), F.lit("<NULL>")) for c in BUSINESS_COLUMNS]), 256),
)


w_dup = Window.partitionBy("record_id").orderBy(F.col("bronze_ingested_at"))
df_silver = df_silver.withColumn("_dup_rank", F.row_number().over(w_dup))


df_duplicates = df_silver.filter(F.col("_dup_rank") > 1)
df_silver     = df_silver.filter(F.col("_dup_rank") == 1).drop("_dup_rank")


dup_count = df_duplicates.count()
count_after = trace("4. after dedup", df_silver)
print(f"Exact duplicate rows moved to quarantine: {dup_count}")


# COMMAND ----------


# MAGIC %md
# MAGIC # 5. Data Quality Validation (Hard rules ⟶ Quarantine)
# MAGIC ทุก rule คืน **NULL เมื่อผ่าน** / ข้อความเมื่อไม่ผ่าน ⟶ รวมเป็น array ⟶ แถวที่ `_dq_reason IS NULL` = valid


# COMMAND ----------


REQUIRED_COLUMNS = ["is_high_defect", "production_volume", "defect_rate"]


PERCENT_COLUMNS = [             # 0–100
    "supplier_quality_pct", "quality_score_pct", "downtime_pct", "worker_productivity_pct",
]
RATIO_0_1_COLUMNS = [           # 0–1  (v2)
    "stockout_rate", "energy_efficiency_factor",
]
NON_NEGATIVE_COLUMNS = [
    "production_volume", "production_cost", "delivery_delay_days", "defect_rate",
    "maintenance_hours_per_week", "inventory_turnover_ratio", "safety_incidents_count",
    "energy_consumption_kwh", "additive_process_time_hrs", "additive_material_cost_usd",
]


dq_rules = []
for c in REQUIRED_COLUMNS:
```

```
    dq_rules.append((f"required:{c}", F.when(F.col(c).isNull(), F.lit(f"{c}: required value is NULL"))))


dq_rules.append(("target_domain",
    F.when(F.col("is_high_defect").isNotNull() & ~F.col("is_high_defect").isin(0, 1),
           F.lit("is_high_defect: value must be 0 or 1"))))


for c in PERCENT_COLUMNS:
    dq_rules.append((f"pct_range:{c}",
        F.when(F.col(c).isNotNull() & ((F.col(c) < 0) | (F.col(c) > 100)),
               F.lit(f"{c}: value must be between 0 and 100"))))


for c in RATIO_0_1_COLUMNS:
    dq_rules.append((f"ratio_range:{c}",
        F.when(F.col(c).isNotNull() & ((F.col(c) < 0) | (F.col(c) > 1)),
               F.lit(f"{c}: value must be between 0 and 1"))))


for c in NON_NEGATIVE_COLUMNS:
    dq_rules.append((f"non_negative:{c}",
        F.when(F.col(c).isNotNull() & (F.col(c) < 0), F.lit(f"{c}: value cannot be negative"))))


dq_rules.append(("max_hours_per_week",
    F.when(F.col("maintenance_hours_per_week") > 168,
           F.lit("maintenance_hours_per_week: cannot exceed 168 hours/week"))))


for c, t in EXPECTED_TYPES.items():
    if t == "double":
        dq_rules.append((f"nan:{c}", F.when(F.isnan(F.col(c)), F.lit(f"{c}: NaN is not allowed"))))


df_silver = df_silver.withColumn(
    "_dq_errors",
    F.concat(F.col("_dq_cast_errors"), F.array_compact(F.array(*[e for _, e in dq_rules]))),
).withColumn(
    "_dq_reason", F.when(F.size("_dq_errors") > 0, F.array_join("_dq_errors", "; "))
)


rule_fail_counts = df_silver.select(
    F.sum((F.size("_dq_cast_errors") > 0).cast("int")).alias("cast_error"),
    *[F.sum(e.isNotNull().cast("int")).alias(n.replace(":", "__")) for n, e in dq_rules],
)
print("Rows failing each DQ rule:")
display(rule_fail_counts)


# COMMAND ----------


# MAGIC %md
# MAGIC # 6. Business Rule Warnings (ไม่ตัดแถว — ติดธงไว้ให้ Gold/BI ใช้)


# COMMAND ----------


warning_expressions = [
    F.when((F.col("production_volume") == 0) & (F.col("production_cost") > 0),
           F.lit("production_volume=0 but production_cost>0")),
    F.when((F.col("production_volume") == 0) & (F.col("energy_consumption_kwh") > 0),
           F.lit("production_volume=0 but energy_consumption_kwh>0")),
    # v2: defect_rate สูงแต่ถูก label ว่า low defect (หรือกลับกัน) — label อาจไม่สอดคล้อง
    F.when((F.col("defect_rate") > 4.5) & (F.col("is_high_defect") == 0),
           F.lit("defect_rate>4.5 but is_high_defect=0")),
    F.when((F.col("quality_score_pct") < 60),
           F.lit("quality_score_pct below 60 (unusual)")),
]


df_silver = df_silver.withColumn(
    "_dq_warning_reason", F.array_join(F.array_compact(F.array(*warning_expressions)), "; ")
).withColumn(
    "_dq_warning_reason", F.when(F.length("_dq_warning_reason") > 0, F.col("_dq_warning_reason"))
)
```

```
# COMMAND ----------


# MAGIC %md
# MAGIC # 7. Split Valid / Quarantine
# MAGIC Quarantine = (DQ fail) ∪ (exact duplicates) ∪ (malformed rows จาก Bronze)


# COMMAND ----------


df_silver_valid  = df_silver.filter(F.col("_dq_reason").isNull())
df_dq_fail       = df_silver.filter(F.col("_dq_reason").isNotNull())


valid_count   = trace("7a. valid", df_silver_valid)
dq_fail_count = trace("7b. dq fail", df_dq_fail)
assert valid_count + dq_fail_count == count_after, "valid + dq_fail ต้องเท่ากับจำนวนหลัง dedup"


BRONZE_LINEAGE = ["source_file", "source_file_modified_at", "bronze_batch_id", "bronze_ingested_at"]
QUAR_COLS = BUSINESS_COLUMNS + ["record_id"] + BRONZE_LINEAGE


df_quarantine = (
    df_dq_fail.select(*QUAR_COLS, "_dq_reason", "_dq_warning_reason")
    .unionByName(
        df_duplicates.select(*QUAR_COLS)
        .withColumn("_dq_reason", F.lit("exact duplicate of another record"))
        .withColumn("_dq_warning_reason", F.lit(None).cast("string"))
    )
)
if corrupt_count > 0:
    df_quarantine = df_quarantine.unionByName(
        bronze_corrupt.select(
            *[F.lit(None).cast(EXPECTED_TYPES[c]).alias(c) for c in BUSINESS_COLUMNS],
            F.lit(None).cast("string").alias("record_id"),
            F.col("_source_file").alias("source_file"),
            F.col("_source_file_modified_at").alias("source_file_modified_at"),
            F.col("_bronze_batch_id").alias("bronze_batch_id"),
            F.col("_ingested_at").alias("bronze_ingested_at"),
            F.concat(F.lit("malformed CSV row: "), F.col("_corrupt_record")).alias("_dq_reason"),
            F.lit(None).cast("string").alias("_dq_warning_reason"),
        ),
        allowMissingColumns=True,
    )


invalid_count = trace("7c. quarantine total", df_quarantine)
assert valid_count + invalid_count == bronze_count, "valid + quarantine ต้องเท่ากับจำนวน Bronze (reconciliation)"


if invalid_count > 0:
    display(df_quarantine.groupBy("_dq_reason").count().orderBy(F.desc("count")))
else:
    print("No invalid records found.")


# COMMAND ----------


# MAGIC %md
# MAGIC # 8. Business Rule Warning Report


# COMMAND ----------


df_warn = df_silver_valid.filter(F.col("_dq_warning_reason").isNotNull())
warning_count = df_warn.count()
print(f"Rows with business-rule warnings: {warning_count}")
if warning_count > 0:
    display(df_warn.groupBy("_dq_warning_reason").count().orderBy(F.desc("count")))


# COMMAND ----------
```

```
# MAGIC %md
# MAGIC # 9. Lineage Metadata
# MAGIC ทุกแถวย้อนได้ถึง: Bronze table ⟶ ไฟล์ต้นทาง (+ เวลาแก้ไขไฟล์) ⟶ รอบโหลด Bronze ⟶ รอบ Silver


# COMMAND ----------


def add_lineage(df):
    return (df.withColumn("source_table", F.lit(SOURCE_TABLE))
              .withColumn("silver_processed_at", F.current_timestamp())
              .withColumn("silver_batch_id", F.lit(silver_batch_id)))


LINEAGE_COLUMNS = ["source_table"] + BRONZE_LINEAGE + ["silver_processed_at", "silver_batch_id"]


FINAL_COLUMNS = ["record_id"] + BUSINESS_COLUMNS + ["_dq_warning_reason"] + LINEAGE_COLUMNS
df_silver_final      = add_lineage(df_silver_valid).select(*FINAL_COLUMNS)
df_quarantine_final = add_lineage(df_quarantine).select(
    "record_id", *BUSINESS_COLUMNS, "_dq_reason", "_dq_warning_reason", *LINEAGE_COLUMNS)


df_silver_final.printSchema()
display(df_silver_final.limit(10))


# COMMAND ----------


# MAGIC %md
# MAGIC # 10. Silver Profiling (ใช้ประกอบบท Data Understanding)


# COMMAND ----------


display(df_silver_final.select(*BUSINESS_COLUMNS).summary("count", "mean", "stddev", "min", "25%", "50%", "75%", "max"))


print("Target class balance (is_high_defect):")
display(
    df_silver_final.groupBy("is_high_defect").count()
    .withColumn("pct", F.round(F.col("count") / F.lit(valid_count) * 100, 2))
    .orderBy("is_high_defect")
)


# COMMAND ----------


# MAGIC %md
# MAGIC # 11. Final Data Quality Gate


# COMMAND ----------


print("Row trace summary:")
for step, n in ROW_TRACE:
    print(f"  {step:<38} {n:>8}")


if bronze_count > 0 and valid_count == 0:
    raise ValueError("Silver quality gate failed: 0 valid rows")


missing_final_cols = set(FINAL_COLUMNS) - set(df_silver_final.columns)
if missing_final_cols:
    raise ValueError(f"Final Silver is missing columns: {sorted(missing_final_cols)}")


bad_target = df_silver_final.filter(F.col("is_high_defect").isNull() | ~F.col("is_high_defect").isin(0, 1)).count()
if bad_target > 0:
    raise ValueError(f"{bad_target} NULL/invalid values in is_high_defect")
```

```
dup_keys = df_silver_final.groupBy("record_id").count().filter("count > 1").count()
null_keys = df_silver_final.filter(F.col("record_id").isNull()).count()
if dup_keys or null_keys:
    raise ValueError(f"record_id not unique/not null (dup={dup_keys}, null={null_keys})")


print("Silver Data Quality Gate: PASS")


# COMMAND ----------


# MAGIC %md
# MAGIC # 12. Write Quarantine & Silver Tables (+ constraints, comments, run log)


# COMMAND ----------


for t in {SILVER_TABLE, QUARANTINE_TABLE, DQ_RUN_LOG}:
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {t.rsplit('.', 1)[0]}")


(df_quarantine_final.write.format("delta").mode(WRITE_MODE)
    .option("overwriteSchema", "true").saveAsTable(QUARANTINE_TABLE))
print(f"Quarantine table written: {QUARANTINE_TABLE} ({invalid_count} rows)")


(df_silver_final.write.format("delta").mode(WRITE_MODE)
    .option("overwriteSchema", "true").saveAsTable(SILVER_TABLE))
print(f"Silver table written: {SILVER_TABLE} ({valid_count} rows)")


# Delta CHECK constraints — ป้องกันข้อมูลผิดถูกเขียนเข้ามาภายหลัง
CHECKS = {
    "chk_target_binary":   "is_high_defect IN (0, 1)",
    "chk_record_id":       "record_id IS NOT NULL",
    "chk_pct_quality":     "quality_score_pct BETWEEN 0 AND 100",
    "chk_pct_supplier":    "supplier_quality_pct BETWEEN 0 AND 100",
    "chk_pct_downtime":    "downtime_pct BETWEEN 0 AND 100",
    "chk_stockout_ratio":  "stockout_rate BETWEEN 0 AND 1",
    "chk_nonneg_volume":   "production_volume >= 0",
    "chk_nonneg_defect":   "defect_rate >= 0",
    "chk_maint_hours":     "maintenance_hours_per_week BETWEEN 0 AND 168",
}
for name, expr in CHECKS.items():
    spark.sql(f"ALTER TABLE {SILVER_TABLE} DROP CONSTRAINT IF EXISTS {name}")
    spark.sql(f"ALTER TABLE {SILVER_TABLE} ADD CONSTRAINT {name} CHECK ({expr})")


spark.sql(f"""COMMENT ON TABLE {SILVER_TABLE} IS
'Silver: cleaned, typed, validated manufacturing defect records. 1 row = 1 production batch record. Key: record_id (SHA-256 of business values). Source: {SOURCE_TABLE}'""")
print("Constraints & comment applied")


# COMMAND ----------


run_log = spark.createDataFrame(
    [(silver_batch_id, step, int(n), run_started_at) for step, n in ROW_TRACE],
    "silver_batch_id string, step string, row_count long, run_started_at timestamp",
)
run_log.write.format("delta").mode("append").saveAsTable(DQ_RUN_LOG)


print(f"Bronze rows     : {spark.table(SOURCE_TABLE).count()}")
print(f"Silver rows     : {spark.table(SILVER_TABLE).count()}")
print(f"Quarantine rows : {spark.table(QUARANTINE_TABLE).count()}")
display(spark.table(DQ_RUN_LOG).filter(F.col("silver_batch_id") == silver_batch_id))
dbutils.notebook.exit(f"PASS|bronze={bronze_count}|valid={valid_count}|quarantine={invalid_count}")
```

Appendix C–E: Gold Transformation, PK/FK และ SQL Code

Gold_Predictive.py มีทั้งการแยกตาราง (ข้อ 2–3), PK/FK validation (ข้อ 4–5), threshold scan และ analytic marts (ข้อ 6), SQL DDL / COPY / Views ของ PostgreSQL (ข้อ 8–10) และ integrity test (ข้อ 11)

Gold_Predictive.py

```
# Databricks notebook source
# /// script
# [tool.databricks.environment]
# environment_version = "6"
# ///
# MAGIC %md
# MAGIC # Gold_Predictive — RDBMS Gold Layer (Delta + Lakebase PostgreSQL)
# MAGIC
# MAGIC **Input:** `silver_predictive.manufacturing_defect_dataset_v2` (จาก `Silver_Predictive_v2`)
# MAGIC
# MAGIC ตามแผนรายงานบทที่ 6: **Split Silver 1 ตาราง ⟶ 6 ตารางตาม Business Domain พร้อม PK/FK** แล้วโหลดเข้า RDBMS (**Databricks Lakebase = PostgreSQL 17**)
# MAGIC และสร้าง analytic marts สำหรับ Dashboard / Business Insight (บทที่ 8)
# MAGIC
# MAGIC ```
# MAGIC                    silver_predictive.manufacturing_defect_dataset_v2
# MAGIC                                  |
# MAGIC        ┌──────────┬──────────┬──┴──┬──────────┬──────────┬──────────┐
# MAGIC        ▼          ▼          ▼     ▼          ▼          ▼
# MAGIC   production  supply_chain  quality  maintenance  inventory  operations
# MAGIC  (PK production_id)  (FK production_id ⟶ production.production_id ในทุกตารางลูก, 1 : 1)
# MAGIC ```
# MAGIC
# MAGIC | Gold table | Domain | PK | FK | Columns |
# MAGIC |---|---|---|---|---|
# MAGIC | `production` | Production | `production_id` | – | record_id (UNIQUE), volume, cost, cost_per_unit, volume_band |
# MAGIC | `supply_chain` | Supply Chain | `supply_chain_id` | `production_id` | supplier_quality_pct, delivery_delay_days, bands |
# MAGIC | `quality` | Quality | `quality_id` | `production_id` | defect_rate, quality_score_pct, **is_high_defect**, bands |
# MAGIC | `maintenance` | Maintenance | `maintenance_id` | `production_id` | maintenance_hours_per_week, downtime_pct, bands |
# MAGIC | `inventory` | Inventory | `inventory_id` | `production_id` | inventory_turnover_ratio, stockout_rate |
# MAGIC | `operations` | Workforce / Safety / Energy / Additive | `operations_id` | `production_id` | productivity, safety, energy, additive |
# MAGIC
# MAGIC ### v2.1 — สิ่งที่เปลี่ยนในชั้น Insight
# MAGIC | # | เดิม | ปัญหา | ใหม่ |
# MAGIC |---|---|---|---|
# MAGIC | 1 | ใช้ `defect_rate` และ `quality_score` เป็น "ปัจจัย" ใน risk score | ทั้งสองตัวเป็น **ตัววัดผลด้านคุณภาพ (outcome)** ไม่ใช่สิ่งที่ฝ่ายผลิตปรับได้โดยตรง ⟶ อธิบาย defect ด้วย defect (วนในตัวเอง) | แยก `driver_role` = **Controllable** (ปรับได้) / **Outcome** (ตัววัดผล) · risk score ใช้เฉพาะ Controllable · Outcome ใช้เป็น **QC alert** |
# MAGIC | 2 | กำหนด threshold ด้วยตา (maint > 11, volume > 700, QS < 80) | ไม่ตรงจุดเปลี่ยนจริง (เช่น volume 701–800 = 80.8% เท่าค่าเฉลี่ย) | **Threshold scan** หา cut-off ที่ high-defect % ต่างกันมากที่สุด ⟶ เก็บใน `mart_threshold_scan` |
# MAGIC | 3 | – | ยังไม่ได้ถามว่า label `is_high_defect` นิยามจากอะไร | **Unknown Question:** เทียบ label กับกฎ threshold ⟶ `mart_label_rule` |


# COMMAND ----------


# MAGIC %pip install pg8000 --quiet


# COMMAND ----------


dbutils.library.restartPython()


# COMMAND ----------


# MAGIC %md
# MAGIC ## 0. Config


# COMMAND ----------


import uuid
from datetime import datetime, timezone
```

```
import pandas as pd
import pyspark.sql.functions as F
from pyspark.sql.window import Window


CATALOG      = "predictive_manufacring_defect"
SILVER_TABLE = f"{CATALOG}.silver_predictive.manufacturing_defect_dataset_v2"
GOLD_SCHEMA  = f"{CATALOG}.gold_predictive"


# Lakebase (PostgreSQL)
LAKEBASE_PROJECT_ID = "manufacturing-defect-gold"
PG_DATABASE         = "databricks_postgres"
PG_SCHEMA           = "gold"
UC_PG_CATALOG       = "manufacturing_gold_pg"          # ลงทะเบียน Lakebase เป็น catalog ใน Unity Catalog


gold_batch_id  = str(uuid.uuid4())
run_started_at = datetime.now(timezone.utc)
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD_SCHEMA}")
print("Gold batch id:", gold_batch_id)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. Load Silver + Input Gate


# COMMAND ----------


silver = spark.table(SILVER_TABLE)
silver_count = silver.count()
print("Silver rows:", silver_count)


assert silver_count > 0, "Silver ว่าง — รัน Silver_Predictive_v2 ก่อน"
assert silver.filter(F.col("record_id").isNull()).count() == 0, "record_id มี NULL"
assert silver.select("record_id").distinct().count() == silver_count, "record_id ไม่ unique"
print("Gold input gate: PASS")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. Surrogate Key `production_id` + Bands
# MAGIC - `production_id` = BIGINT เรียงตาม `record_id` (hash ที่ deterministic) ⟶ รันซ้ำกับข้อมูลชุดเดิมได้ id เดิมทุกครั้ง
# MAGIC - v2.1: ขอบ band ของ maintenance / volume / quality score ปรับให้ตรงจุดเปลี่ยนที่ threshold scan (ข้อ 6.0) พบ เพื่อให้กราฟเห็น "ขั้นบันได" ชัด


# COMMAND ----------


base = silver.withColumn("production_id", F.row_number().over(Window.orderBy("record_id")).cast("bigint"))


def band(col, edges, labels):
    """edges = upper bounds (inclusive) ตามลำดับ; คืน (label, order)"""
    expr_label, expr_order = None, None
    for i, (edge, lab) in enumerate(zip(edges, labels), start=1):
        cond = F.col(col) <= edge
        expr_label = F.when(cond, lab) if expr_label is None else expr_label.when(cond, lab)
        expr_order = F.when(cond, i) if expr_order is None else expr_order.when(cond, i)
    return expr_label.otherwise(labels[-1]), expr_order.otherwise(len(labels))


BANDS = {
    "volume_band":           ("production_volume",          [300, 500, 700, 800, 10**9], ["100-300", "301-500", "501-700", "701-800", "801+"]),
    "supplier_quality_band": ("supplier_quality_pct",       [85, 90, 95, 100],           ["80-85%", "85-90%", "90-95%", "95-100%"]),
    "delivery_delay_band":   ("delivery_delay_days",        [0, 1, 2, 3, 4, 10**6],      ["0 d", "1 d", "2 d", "3 d", "4 d", "5+ d"]),
    "defect_rate_band":      ("defect_rate",                [1, 2, 3, 4, 10**6],         ["0.5-1", "1-2", "2-3", "3-4", "4-5"]),
    "quality_band":          ("quality_score_pct",          [75, 80, 90, 100],           ["60-75", "75-80", "80-90", "90-100"]),
    "maintenance_band":      ("maintenance_hours_per_week", [5, 10, 16, 168],            ["0-5 h", "6-10 h", "11-16 h", "17-23 h"]),
    "downtime_band":         ("downtime_pct",               [1, 2, 3, 4, 100],           ["0-1%", "1-2%", "2-3%", "3-4%", "4-5%"]),
    "stockout_band":         ("stockout_rate",              [0.025, 0.05, 0.075, 1],     ["0-2.5%", "2.5-5%", "5-7.5%", "7.5-10%"]),
```

```
    "productivity_band":    ("worker_productivity_pct",    [85, 90, 95, 100],          ["80-85%", "85-90%", "90-95%", "95-100%"]),
    "energy_eff_band":      ("energy_efficiency_factor",   [0.2, 0.3, 0.4, 1],         ["0.1-0.2", "0.2-0.3", "0.3-0.4", "0.4-0.5"]),
}
for name, (col, edges, labels) in BANDS.items():
    lab, order = band(col, edges, labels)
    base = base.withColumn(name, lab).withColumn(name + "_order", order)


gold_meta = [F.lit(gold_batch_id).alias("gold_batch_id"), F.current_timestamp().alias("gold_processed_at")]


# COMMAND ----------


# MAGIC %md
# MAGIC ## 3. Split Silver ⟶ 6 Domain DataFrames (PK / FK)


# COMMAND ----------


df_production = base.select(
    "production_id", "record_id",
    "production_volume", "production_cost",
    F.round(F.col("production_cost") / F.nullif(F.col("production_volume"), F.lit(0)), 4).alias("cost_per_unit"),
    "volume_band", "volume_band_order",
    "silver_batch_id", *gold_meta,
)


df_supply_chain = base.select(
    F.col("production_id").alias("supply_chain_id"), "production_id",
    "supplier_quality_pct", "delivery_delay_days",
    "supplier_quality_band", "supplier_quality_band_order",
    "delivery_delay_band", "delivery_delay_band_order", *gold_meta,
)


df_quality = base.select(
    F.col("production_id").alias("quality_id"), "production_id",
    "defect_rate", "quality_score_pct", "is_high_defect",
    "defect_rate_band", "defect_rate_band_order", "quality_band", "quality_band_order",
    F.col("_dq_warning_reason").alias("dq_warning_reason"), *gold_meta,
)


df_maintenance = base.select(
    F.col("production_id").alias("maintenance_id"), "production_id",
    "maintenance_hours_per_week", "downtime_pct",
    "maintenance_band", "maintenance_band_order", "downtime_band", "downtime_band_order", *gold_meta,
)


df_inventory = base.select(
    F.col("production_id").alias("inventory_id"), "production_id",
    "inventory_turnover_ratio", "stockout_rate",
    "stockout_band", "stockout_band_order", *gold_meta,
)


df_operations = base.select(
    F.col("production_id").alias("operations_id"), "production_id",
    "worker_productivity_pct", "safety_incidents_count",
    "energy_consumption_kwh", "energy_efficiency_factor",
    F.round(F.col("energy_consumption_kwh") / F.nullif(F.col("production_volume"), F.lit(0)), 4).alias("energy_kwh_per_unit"),
    "additive_process_time_hrs", "additive_material_cost_usd",
    "productivity_band", "productivity_band_order", "energy_eff_band", "energy_eff_band_order", *gold_meta,
)


# (table name, df, pk column)
GOLD_TABLES = [
    ("production",   df_production,   "production_id"),
    ("supply_chain", df_supply_chain, "supply_chain_id"),
    ("quality",      df_quality,      "quality_id"),
    ("maintenance",  df_maintenance,  "maintenance_id"),
    ("inventory",    df_inventory,    "inventory_id"),
    ("operations",   df_operations,   "operations_id"),
]
```

```
for t, df, pk in GOLD_TABLES:
    print(f"{t:<13} columns={len(df.columns):>2}  {df.columns}")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 4. PK / FK Integrity Validation (Spark)
# MAGIC PK ⟶ NULL check ⟶ Duplicate check · FK ⟶ ต้องพบใน parent PK


# COMMAND ----------


parent_keys = df_production.select("production_id")
checks = []
for t, df, pk in GOLD_TABLES:
    rows = df.count()
    pk_null = df.filter(F.col(pk).isNull()).count()
    pk_dup = rows - df.select(pk).distinct().count()
    if t == "production":
        orphan = 0
    else:
        orphan = df.join(parent_keys, "production_id", "left_anti").count()
    status = "PASS" if (rows == silver_count and pk_null == 0 and pk_dup == 0 and orphan == 0) else "FAIL"
    checks.append((t, pk, rows, pk_null, pk_dup, orphan, status))


df_checks = spark.createDataFrame(checks, "table string, pk string, rows long, pk_null long, pk_duplicate long, fk_orphan long, status string")
display(df_checks)
if df_checks.filter("status != 'PASS'").count() > 0:
    raise ValueError("PK/FK validation failed")
print("PK/FK validation (Spark): PASS")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 5. Load Gold ⟶ Delta (Unity Catalog) พร้อม PRIMARY KEY / FOREIGN KEY constraints


# COMMAND ----------


def fq(t): return f"{GOLD_SCHEMA}.{t}"


# ลบ FK เดิมก่อน overwrite (ไม่งั้นแทนที่ตาราง parent ไม่ได้)
for t, _, _ in GOLD_TABLES[1:]:
    if spark.catalog.tableExists(fq(t)):
        spark.sql(f"ALTER TABLE {fq(t)} DROP CONSTRAINT IF EXISTS fk_{t}_production")


for t, df, pk in GOLD_TABLES:
    (df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq(t)))
    spark.sql(f"ALTER TABLE {fq(t)} ALTER COLUMN {pk} SET NOT NULL")
    spark.sql(f"ALTER TABLE {fq(t)} DROP CONSTRAINT IF EXISTS pk_{t}")
    spark.sql(f"ALTER TABLE {fq(t)} ADD CONSTRAINT pk_{t} PRIMARY KEY ({pk})")
    print(f"Delta written: {fq(t)}")


for t, _, _ in GOLD_TABLES[1:]:
    spark.sql(f"ALTER TABLE {fq(t)} ALTER COLUMN production_id SET NOT NULL")
    spark.sql(f"""ALTER TABLE {fq(t)} ADD CONSTRAINT fk_{t}_production
                  FOREIGN KEY (production_id) REFERENCES {fq('production')}(production_id)""")


COMMENTS = {
    "production":   "Gold RDBMS: production batch (parent). PK production_id, UNIQUE record_id",
    "supply_chain": "Gold RDBMS: supplier quality & delivery delay. FK production_id",
    "quality":      "Gold RDBMS: defect rate, quality score, target is_high_defect. FK production_id",
    "maintenance":  "Gold RDBMS: maintenance hours & downtime. FK production_id",
    "inventory":    "Gold RDBMS: inventory turnover & stockout rate. FK production_id",
    "operations":   "Gold RDBMS: workforce, safety, energy, additive manufacturing. FK production_id",
}
for t, c in COMMENTS.items():
    spark.sql(f"COMMENT ON TABLE {fq(t)} IS '{c}'")
```

```
print("UC PK/FK constraints & comments applied")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 6. Analytic Marts (Business Insight layer)
# MAGIC
# MAGIC ### 6.0 แยกบทบาทของตัวแปร: ปัจจัยที่จัดการได้ (Controllable) vs ตัววัดผล (Outcome)
# MAGIC
# MAGIC | บทบาท | ตัวแปร | ใช้ทำอะไร |
# MAGIC |---|---|---|
# MAGIC | **Controllable** — สิ่งที่ทีมปรับได้ก่อน/ระหว่างผลิต | maintenance hours, production volume, supplier quality, delivery delay, downtime, stockout, worker productivity, energy efficiency | หาคันโยก ⟶ ทำ **risk score ก่อนปล่อย batch** |
# MAGIC | **Outcome** — ผลลัพธ์ด้านคุณภาพที่วัดได้หลังผลิต | defect rate, quality score | ใช้เป็น **QC alert / KPI เฝ้าระวัง** ไม่ใช่สาเหตุ (ถ้าใช้อธิบาย high-defect จะเป็นการอธิบาย defect ด้วย defect) |
# MAGIC
# MAGIC **Threshold scan:** แต่ละตัวแปร ลอง cut-off ทุกจุด (percentile 2–98 หรือทุกค่าจำนวนเต็ม) แล้วเลือกจุดที่ high-defect % ของฝั่งเสี่ยงกับฝั่งปลอดภัย **ต่างกันมากที่สุด** (ทั้งสองฝั่งต้องมี ≥ 5% ของ batch)
จากนั้นปัดเป็นตัวเลขที่ใช้สื่อสารง่าย (`NICE_STEP`)
# MAGIC ตัวแปรที่ต่าง ≥ 15 pp ถือเป็น **key factor**


# COMMAND ----------


# (driver, column, domain, role, lever / owner, band column)
DRIVER_CATALOG = [
    ("Maintenance hours/week", "maintenance_hours_per_week", "Maintenance",  "Controllable", "ทีมซ่อมบำรุง: แผน PM / overhaul เครื่องที่ต้องซ่อมบ่อย", "maintenance_band"),
    ("Production volume",      "production_volume",          "Production",   "Controllable", "ฝ่ายวางแผนผลิต: ขนาด batch / QC sampling",          "volume_band"),
    ("Supplier quality",       "supplier_quality_pct",       "Supply Chain", "Controllable", "จัดซื้อ: คัดเลือก/ประเมิน supplier",                  "supplier_quality_band"),
    ("Delivery delay",         "delivery_delay_days",        "Supply Chain", "Controllable", "จัดซื้อ/โลจิสติกส์",                                  "delivery_delay_band"),
    ("Downtime",               "downtime_pct",               "Maintenance",  "Controllable", "ทีมซ่อมบำรุง",                                       "downtime_band"),
    ("Stockout rate",          "stockout_rate",              "Inventory",    "Controllable", "วางแผนสต๊อก",                                         "stockout_band"),
    ("Worker productivity",    "worker_productivity_pct",    "Operations",   "Controllable", "หัวหน้าไลน์ / HR",                                    "productivity_band"),
    ("Energy efficiency",      "energy_efficiency_factor",   "Operations",   "Controllable", "วิศวกรรม / พลังงาน",                                  "energy_eff_band"),
    ("Defect rate",            "defect_rate",                "Quality",      "Outcome",      "ตัววัดผล QC — ใช้เฝ้าระวัง ไม่ใช่คันโยก",             "defect_rate_band"),
    ("Quality score",          "quality_score_pct",          "Quality",      "Outcome",      "ตัววัดผล QC — ใช้เฝ้าระวัง ไม่ใช่คันโยก",             "quality_band"),
]
NICE_STEP = {"production_volume": 50, "defect_rate": 0.5, "quality_score_pct": 5,
             "supplier_quality_pct": 1, "worker_productivity_pct": 1, "downtime_pct": 0.1,
             "stockout_rate": 0.005, "energy_efficiency_factor": 0.01}  # อื่น ๆ (จำนวนเต็ม) ใช้ค่าเดิม
KEY_FACTOR_MIN_GAP_PP = 15
MIN_SIDE_SHARE = 0.05


pdf = base.select("is_high_defect", *[c for _, c, *_ in DRIVER_CATALOG]).toPandas()
n_all = len(pdf)


def scan_threshold(s, y):
    """คืน (cut, gap) ที่ |high-defect% ฝั่ง > cut  −  ฝั่ง ≤ cut| มากที่สุด"""
    cands = sorted(s.unique()) if pd.api.types.is_integer_dtype(s) else sorted(set(s.quantile([i / 100 for i in range(2, 99)]).round(4)))
    best = None
    for c in cands:
        hi = s > c
        k = int(hi.sum())
        if k < MIN_SIDE_SHARE * n_all or n_all - k < MIN_SIDE_SHARE * n_all:
            continue
        gap = y[hi].mean() - y[~hi].mean()
        if best is None or abs(gap) > abs(best[1]):
            best = (float(c), float(gap))
    return best


def fmt(v): return str(int(v)) if float(v).is_integer() else str(v)


scan_rows = []
for driver, col, domain, role, lever, _ in DRIVER_CATALOG:
    s, y = pdf[col], pdf["is_high_defect"]
    raw_cut, gap = scan_threshold(s, y)
    step = NICE_STEP.get(col, 1)
    thr = round(round(raw_cut / step) * step, 4)
    if gap > 0:       # ค่าสูง = เสี่ยง
        op, risky = ">", s > thr
    else:             # ค่าต่ำ = เสี่ยง
        op, risky = "<", s < thr
    risky_pct, safe_pct = y[risky].mean() * 100, y[~risky].mean() * 100
    scan_rows.append((driver, col, domain, role, lever, float(raw_cut), float(thr), op, f"{op} {fmt(thr)}",
                      int(risky.sum()), round(float(risky_pct), 2), round(float(safe_pct), 2),
```

```
                    round(float(risky_pct - safe_pct), 2), bool(risky_pct - safe_pct >= KEY_FACTOR_MIN_GAP_PP)))


mart_scan = spark.createDataFrame(scan_rows,
    """driver string, column_name string, domain string, driver_role string, lever string,
       raw_cut double, threshold double, operator string, rule string, risky_batches long,
       risky_side_high_defect_pct double, safe_side_high_defect_pct double, gap_pp double, is_key_factor boolean""")
mart_scan.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_threshold_scan"))
display(spark.table(fq("mart_threshold_scan")).orderBy("driver_role", F.desc("gap_pp")))


KEY_CONTROLLABLE = [(r[1], r[7], r[6]) for r in scan_rows if r[3] == "Controllable" and r[13]]   # (column, op, threshold)
KEY_OUTCOME      = [(r[1], r[7], r[6]) for r in scan_rows if r[3] == "Outcome" and r[13]]
print("Key controllable factors:", KEY_CONTROLLABLE)
print("Key outcome (QC alert)   :", KEY_OUTCOME)
assert KEY_CONTROLLABLE, "ไม่พบ controllable key factor — ตรวจข้อมูล/threshold"


def flag_sql(col, op, thr): return f"{col} {op} {fmt(thr)}"
CTRL_SQL = [flag_sql(*k) for k in KEY_CONTROLLABLE]
QC_SQL   = [flag_sql(*k) for k in KEY_OUTCOME]
print("Risk score rules (controllable):", CTRL_SQL)
print("QC alert rules  (outcome)     :", QC_SQL)


# COMMAND ----------


# MAGIC %md
# MAGIC ### 6.1 batch_360 — 1 แถว/batch + controllable risk score + QC alert + label rule


# COMMAND ----------


p, s, q, m, i, o = [spark.table(fq(t)).alias(a) for (t, _, _), a in zip(GOLD_TABLES, "psqmio")]


batch_360 = (
    p.join(s, "production_id").join(q, "production_id").join(m, "production_id")
     .join(i, "production_id").join(o, "production_id")
     .select(
        "production_id", "p.record_id",
        "p.production_volume", "p.production_cost", "p.cost_per_unit", "p.volume_band", "p.volume_band_order",
        "s.supplier_quality_pct", "s.delivery_delay_days", "s.supplier_quality_band", "s.supplier_quality_band_order",
        "s.delivery_delay_band", "s.delivery_delay_band_order",
        "q.defect_rate", "q.quality_score_pct", "q.is_high_defect", "q.defect_rate_band", "q.defect_rate_band_order",
        "q.quality_band", "q.quality_band_order",
        "m.maintenance_hours_per_week", "m.downtime_pct", "m.maintenance_band", "m.maintenance_band_order",
        "m.downtime_band", "m.downtime_band_order",
        "i.inventory_turnover_ratio", "i.stockout_rate", "i.stockout_band", "i.stockout_band_order",
        "o.worker_productivity_pct", "o.safety_incidents_count", "o.energy_consumption_kwh",
        "o.energy_efficiency_factor", "o.energy_kwh_per_unit", "o.additive_process_time_hrs",
        "o.additive_material_cost_usd", "o.productivity_band", "o.productivity_band_order",
        "o.energy_eff_band", "o.energy_eff_band_order",
     )
)
for col, op, thr in KEY_CONTROLLABLE + KEY_OUTCOME:
    batch_360 = batch_360.withColumn(f"flag_{col}", F.expr(flag_sql(col, op, thr)).cast("int"))


ctrl_count = sum(F.col(f"flag_{c}") for c, _, _ in KEY_CONTROLLABLE)
qc_any     = sum(F.col(f"flag_{c}") for c, _, _ in KEY_OUTCOME) if KEY_OUTCOME else F.lit(0)
batch_360 = (
    batch_360
    .withColumn("controllable_risk_count", ctrl_count)
    # คงคอลัมน์เดิมไว้ให้ notebook อื่น (เช่น ML_Predictive) ใช้ต่อได้: จำนวน flag ทั้งหมด (controllable + outcome) ตาม threshold ใหม่
    .withColumn("risk_flag_count", ctrl_count + sum(F.col(f"flag_{c}") for c, _, _ in KEY_OUTCOME))
    .withColumn("risk_segment", F.when(F.col("controllable_risk_count") == 0, "Low")
                                 .when(F.col("controllable_risk_count") == 1, "Medium").otherwise("High"))
    .withColumn("risk_segment_order", F.when(F.col("controllable_risk_count") == 0, 1)
                                       .when(F.col("controllable_risk_count") == 1, 2).otherwise(3))
    .withColumn("qc_alert", (qc_any > 0).cast("int"))
    # Unknown Question: label อธิบายได้ด้วยกฎ "เกิน threshold ข้อใดข้อหนึ่ง" หรือไม่
    .withColumn("label_rule_pred", ((ctrl_count + qc_any) > 0).cast("int"))
)
batch_360.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("batch_360"))
b360 = spark.table(fq("batch_360"))
print("batch_360 rows:", b360.count())
```

```
# COMMAND ----------


# 6.2 KPI overview (1 แถว)
kpi = b360.agg(
    F.count("*").alias("total_batches"),
    F.sum("production_volume").alias("total_units"),
    F.round(F.sum("production_cost"), 2).alias("total_production_cost"),
    F.round(F.sum("production_cost") / F.sum("production_volume"), 2).alias("avg_cost_per_unit"),
    F.sum("is_high_defect").alias("high_defect_batches"),
    F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct"),
    F.round(F.avg("defect_rate"), 3).alias("avg_defect_rate"),
    F.round(F.avg("quality_score_pct"), 2).alias("avg_quality_score"),
    F.round(F.avg("supplier_quality_pct"), 2).alias("avg_supplier_quality"),
    F.round(F.avg("maintenance_hours_per_week"), 2).alias("avg_maintenance_hours"),
    F.round(F.avg("downtime_pct"), 2).alias("avg_downtime_pct"),
    F.round(F.avg("stockout_rate") * 100, 2).alias("avg_stockout_pct"),
    F.round(F.avg("worker_productivity_pct"), 2).alias("avg_worker_productivity"),
    F.sum("safety_incidents_count").alias("total_safety_incidents"),
    F.round(F.sum("energy_consumption_kwh"), 0).alias("total_energy_kwh"),
    F.round(F.sum(F.when(F.col("is_high_defect") == 1, F.col("production_cost"))) / F.sum("production_cost") * 100, 2)
        .alias("cost_share_in_high_defect_pct"),
    F.round(F.avg((F.col("risk_segment") == "Low").cast("int")) * 100, 2).alias("low_risk_batch_pct"),
    F.round(F.avg("qc_alert") * 100, 2).alias("qc_alert_batch_pct"),
    F.round(F.avg((F.col("label_rule_pred") == F.col("is_high_defect")).cast("int")) * 100, 2).alias("label_rule_agreement_pct"),
)
kpi.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("kpi_overview"))
display(spark.table(fq("kpi_overview")))


# COMMAND ----------


# 6.3 High-defect % by driver band (long format) + driver_role
catalog_df = spark.createDataFrame([(d, dom, role, lever) for d, _, dom, role, lever, _ in DRIVER_CATALOG],
                                   "driver string, domain string, driver_role string, lever string")
parts = []
for driver, col, domain, role, lever, bcol in DRIVER_CATALOG:
    parts.append(
        b360.groupBy(F.col(bcol).alias("band"), F.col(bcol + "_order").alias("band_order"))
        .agg(F.count("*").alias("batches"),
             F.sum("is_high_defect").alias("high_defect_batches"),
             F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct"),
             F.round(F.avg("defect_rate"), 3).alias("avg_defect_rate"),
             F.round(F.sum("production_cost") / F.sum("production_volume"), 2).alias("avg_cost_per_unit"),
             F.sum("production_volume").alias("total_units"))
        .withColumn("driver", F.lit(driver)).withColumn("domain", F.lit(domain)).withColumn("driver_role", F.lit(role))
    )
from functools import reduce
mart_bands = reduce(lambda a, b: a.unionByName(b), parts).select(
    "driver", "domain", "driver_role", "band_order", "band", "batches", "high_defect_batches",
    "high_defect_pct", "avg_defect_rate", "avg_cost_per_unit", "total_units")
mart_bands.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_driver_bands"))
display(spark.table(fq("mart_driver_bands")).orderBy("driver_role", "driver", "band_order"))


# COMMAND ----------


# 6.4 Driver impact ranking — spread ระหว่าง band + correlation + threshold scan + role
spread = (spark.table(fq("mart_driver_bands")).groupBy("driver")
          .agg(F.round(F.max("high_defect_pct") - F.min("high_defect_pct"), 2).alias("high_defect_pct_spread"),
               F.min("high_defect_pct").alias("best_band_high_defect_pct"),
               F.max("high_defect_pct").alias("worst_band_high_defect_pct")))
corr_row = b360.select(*[F.corr(F.col(c).cast("double"), F.col("is_high_defect").cast("double")).alias(d)
                         for d, c, *_ in DRIVER_CATALOG]).first().asDict()
corr_df = spark.createDataFrame([(d, float(round(v, 4))) for d, v in corr_row.items()], "driver string, corr_with_high_defect double")
mart_impact = (catalog_df.join(spread, "driver").join(corr_df, "driver")
               .join(spark.table(fq("mart_threshold_scan")).select("driver", "threshold", "operator", "rule", "gap_pp", "is_key_factor"), "driver")
               .withColumn("impact_level", F.when(F.col("gap_pp") >= KEY_FACTOR_MIN_GAP_PP, "Strong")
                                            .when(F.col("gap_pp") >= 7, "Moderate").otherwise("Weak")))
mart_impact.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_driver_impact"))
display(spark.table(fq("mart_driver_impact")).orderBy("driver_role", F.desc("gap_pp")))


# COMMAND ----------
```

```
# 6.5 Risk matrix: maintenance band × volume band (ทั้งคู่เป็น Controllable ⟶ บอกได้ว่าควรปรับตรงไหน)
mart_matrix = (b360.groupBy("maintenance_band", "maintenance_band_order", "volume_band", "volume_band_order")
               .agg(F.count("*").alias("batches"), F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct")))
mart_matrix.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_risk_matrix"))


# 6.6 Risk segment (controllable risk score)
total_cost = b360.agg(F.sum("production_cost")).first()[0]
mart_segment = (b360.groupBy("risk_segment", "risk_segment_order", "controllable_risk_count")
                .agg(F.count("*").alias("batches"),
                     F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct"),
                     F.round(F.sum("production_cost"), 2).alias("production_cost"),
                     F.sum("production_volume").alias("total_units"))
                .withColumn("batch_share_pct", F.round(F.col("batches") / F.lit(silver_count) * 100, 2))
                .withColumn("cost_share_pct", F.round(F.col("production_cost") / F.lit(total_cost) * 100, 2)))
mart_segment.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_risk_segment"))
display(spark.table(fq("mart_risk_segment")).orderBy("controllable_risk_count"))


# 6.7 Action matrix: ก่อนผลิต (controllable segment) × หลังผลิต (QC alert)
mart_action = (b360.groupBy("risk_segment", "risk_segment_order", "qc_alert")
               .agg(F.count("*").alias("batches"), F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct"))
               .withColumn("qc_status", F.when(F.col("qc_alert") == 1, "QC alert").otherwise("QC pass"))
               .withColumn("action", F.when((F.col("risk_segment") == "Low") & (F.col("qc_alert") == 0), "ปล่อย batch ได้")
                                      .when(F.col("risk_segment") == "Low", "ตรวจซ้ำก่อนปล่อย (QC จับได้หลังผลิต)")
                                      .otherwise("แก้ที่กระบวนการก่อนผลิต (ซ่อมบำรุง / ขนาด batch)")))
mart_action.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_action_matrix"))
display(spark.table(fq("mart_action_matrix")).orderBy("risk_segment_order", "qc_alert"))


# COMMAND ----------


# MAGIC %md
# MAGIC ### 6.8 Unknown Question — "label `is_high_defect` นิยามจากอะไร?"
# MAGIC ข้อสังเกต: batch ที่ defect rate ต่ำสุด (0.5–1) ยังเป็น high-defect ถึง ~75% ⟶ label ไม่ได้มาจาก defect rate อย่างเดียว
# MAGIC ทดสอบสมมติฐาน: **label = 1 เมื่อเกิน threshold ข้อใดข้อหนึ่ง** (key factor ทั้ง controllable + outcome) แล้วดูว่าตรงกับ label จริงกี่ %


# COMMAND ----------


mart_label = (b360.groupBy("label_rule_pred", "is_high_defect").count()
              .withColumn("share_pct", F.round(F.col("count") / F.lit(silver_count) * 100, 2))
              .withColumn("outcome", F.when(F.col("label_rule_pred") == F.col("is_high_defect"), "rule agrees with label")
                                      .when(F.col("is_high_defect") == 1, "label=1 but no threshold exceeded")
                                      .otherwise("threshold exceeded but label=0")))
mart_label.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_label_rule"))
display(spark.table(fq("mart_label_rule")).orderBy("label_rule_pred", "is_high_defect"))


# high-defect % ต่อชั่วโมงซ่อมบำรุงที่ละค่า ⟶ เห็นจุดกระโดดชัดเจน
mart_curve = (b360.groupBy("maintenance_hours_per_week")
              .agg(F.count("*").alias("batches"), F.round(F.avg("is_high_defect") * 100, 2).alias("high_defect_pct")))
mart_curve.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(fq("mart_maint_hours_curve"))


agree = spark.table(fq("kpi_overview")).first()["label_rule_agreement_pct"]
print(f"กฎ threshold ({' OR '.join(CTRL_SQL + QC_SQL)}) ตรงกับ label {agree}%")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 7. Lakebase (PostgreSQL) — Connect
# MAGIC ใช้ Databricks SDK สร้าง/หา Lakebase project ⟶ endpoint ⟶ ขอ OAuth token อายุสั้น (ไม่มี password ฝังในโค้ด)
# MAGIC ใช้ driver `pg8000` (pure-Python) — `psycopg[binary]` ทำให้ Python kernel บน serverless ค้าง (ทดสอบแล้ว)


# COMMAND ----------


import ssl
import pg8000.native
from pg8000.exceptions import DatabaseError
from databricks.sdk import WorkspaceClient
from databricks.sdk.errors import NotFound
from databricks.sdk.service import postgres as pgapi
```

```
w = WorkspaceClient()
PG_USER = w.current_user.me().user_name


try:
    project = w.postgres.get_project(name=f"projects/{LAKEBASE_PROJECT_ID}")
except NotFound:
    project = w.postgres.create_project(
        project=pgapi.Project(spec=pgapi.ProjectSpec(display_name="Manufacturing Defect Gold")),
        project_id=LAKEBASE_PROJECT_ID).wait()


branch_name = project.status.default_branch
endpoint = next(e for e in w.postgres.list_endpoints(parent=branch_name)
                if e.status.endpoint_type == pgapi.EndpointType.ENDPOINT_TYPE_READ_WRITE)
PG_HOST = endpoint.status.hosts.host
print("Project :", project.name, "| PG", project.status.pg_version)
print("Branch  :", branch_name)
print("Endpoint:", endpoint.name, endpoint.status.current_state)


# Enable endpoint if it was disabled
endpoint = w.postgres.update_endpoint(
    name=endpoint.name,
    endpoint=pgapi.Endpoint(name=endpoint.name, spec=pgapi.EndpointSpec(
        endpoint_type=pgapi.EndpointType.ENDPOINT_TYPE_READ_WRITE,
        disabled=False,
    )),
    update_mask=pgapi.FieldMask(field_mask=["spec.disabled"]),
).wait()
PG_HOST = endpoint.status.hosts.host
print("Endpoint enabled:", endpoint.name, endpoint.status.current_state)


def pg_connect():
    """เปิด connection ใหม่ด้วย OAuth token อายุสั้น (autocommit)"""
    token = w.postgres.generate_database_credential(endpoint=endpoint.name).token
    return pg8000.native.Connection(user=PG_USER, password=token, host=PG_HOST, database=PG_DATABASE,
                                    ssl_context=ssl.create_default_context(), timeout=120)


def run_script(conn, sql):
    """รัน SQL หลาย statement (คั่นด้วย ;)"""
    for stmt in [x.strip() for x in sql.split(";") if x.strip()]:
        conn.run(stmt)


conn = pg_connect()
print(conn.run("select version()")[0][0])
conn.close()


# COMMAND ----------


# MAGIC %md
# MAGIC ## 8. PostgreSQL DDL — 6 ตาราง พร้อม PK / FK / CHECK


# COMMAND ----------


DDL = f"""
CREATE SCHEMA IF NOT EXISTS {PG_SCHEMA};


DROP VIEW IF EXISTS {PG_SCHEMA}.vw_label_rule_check, {PG_SCHEMA}.vw_risk_segment, {PG_SCHEMA}.vw_defect_by_maintenance,
                    {PG_SCHEMA}.vw_kpi_overview, {PG_SCHEMA}.vw_batch_360;


DROP TABLE IF EXISTS {PG_SCHEMA}.supply_chain, {PG_SCHEMA}.quality, {PG_SCHEMA}.maintenance,
                     {PG_SCHEMA}.inventory, {PG_SCHEMA}.operations, {PG_SCHEMA}.production CASCADE;


CREATE TABLE {PG_SCHEMA}.production (
    production_id       BIGINT          PRIMARY KEY,
    record_id           CHAR(64)        NOT NULL UNIQUE,
```

```
    production_volume  INTEGER        NOT NULL CHECK (production_volume >= 0),
    production_cost    NUMERIC(14,4) NOT NULL CHECK (production_cost >= 0),
    cost_per_unit      NUMERIC(14,4),
    volume_band        VARCHAR(20),
    volume_band_order  SMALLINT,
    silver_batch_id    VARCHAR(36),
    gold_batch_id      VARCHAR(36),
    gold_processed_at  TIMESTAMPTZ
);


CREATE TABLE {PG_SCHEMA}.supply_chain (
    supply_chain_id              BIGINT PRIMARY KEY,
    production_id                BIGINT NOT NULL UNIQUE REFERENCES {PG_SCHEMA}.production(production_id) ON DELETE CASCADE,
    supplier_quality_pct         NUMERIC(8,4) CHECK (supplier_quality_pct BETWEEN 0 AND 100),
    delivery_delay_days          INTEGER       CHECK (delivery_delay_days >= 0),
    supplier_quality_band        VARCHAR(20),
    supplier_quality_band_order SMALLINT,
    delivery_delay_band          VARCHAR(20),
    delivery_delay_band_order    SMALLINT,
    gold_batch_id                VARCHAR(36),
    gold_processed_at            TIMESTAMPTZ
);


CREATE TABLE {PG_SCHEMA}.quality (
    quality_id               BIGINT PRIMARY KEY,
    production_id            BIGINT NOT NULL UNIQUE REFERENCES {PG_SCHEMA}.production(production_id) ON DELETE CASCADE,
    defect_rate              NUMERIC(10,6) NOT NULL CHECK (defect_rate >= 0),
    quality_score_pct        NUMERIC(8,4)  CHECK (quality_score_pct BETWEEN 0 AND 100),
    is_high_defect           SMALLINT       NOT NULL CHECK (is_high_defect IN (0,1)),
    defect_rate_band         VARCHAR(20),
    defect_rate_band_order SMALLINT,
    quality_band             VARCHAR(20),
    quality_band_order       SMALLINT,
    dq_warning_reason        TEXT,
    gold_batch_id            VARCHAR(36),
    gold_processed_at        TIMESTAMPTZ
);


CREATE TABLE {PG_SCHEMA}.maintenance (
    maintenance_id               BIGINT PRIMARY KEY,
    production_id                BIGINT NOT NULL UNIQUE REFERENCES {PG_SCHEMA}.production(production_id) ON DELETE CASCADE,
    maintenance_hours_per_week INTEGER       CHECK (maintenance_hours_per_week BETWEEN 0 AND 168),
    downtime_pct                 NUMERIC(8,4) CHECK (downtime_pct BETWEEN 0 AND 100),
    maintenance_band             VARCHAR(20),
    maintenance_band_order       SMALLINT,
    downtime_band                VARCHAR(20),
    downtime_band_order          SMALLINT,
    gold_batch_id                VARCHAR(36),
    gold_processed_at            TIMESTAMPTZ
);


CREATE TABLE {PG_SCHEMA}.inventory (
    inventory_id               BIGINT PRIMARY KEY,
    production_id              BIGINT NOT NULL UNIQUE REFERENCES {PG_SCHEMA}.production(production_id) ON DELETE CASCADE,
    inventory_turnover_ratio NUMERIC(10,4) CHECK (inventory_turnover_ratio >= 0),
    stockout_rate              NUMERIC(8,6)  CHECK (stockout_rate BETWEEN 0 AND 1),
    stockout_band              VARCHAR(20),
    stockout_band_order        SMALLINT,
    gold_batch_id              VARCHAR(36),
    gold_processed_at          TIMESTAMPTZ
);


CREATE TABLE {PG_SCHEMA}.operations (
    operations_id                BIGINT PRIMARY KEY,
    production_id                BIGINT NOT NULL UNIQUE REFERENCES {PG_SCHEMA}.production(production_id) ON DELETE CASCADE,
    worker_productivity_pct      NUMERIC(8,4)  CHECK (worker_productivity_pct BETWEEN 0 AND 100),
    safety_incidents_count       INTEGER        CHECK (safety_incidents_count >= 0),
    energy_consumption_kwh       NUMERIC(14,4) CHECK (energy_consumption_kwh >= 0),
    energy_efficiency_factor     NUMERIC(8,6)  CHECK (energy_efficiency_factor BETWEEN 0 AND 1),
    energy_kwh_per_unit          NUMERIC(14,4),
    additive_process_time_hrs  NUMERIC(10,4) CHECK (additive_process_time_hrs >= 0),
    additive_material_cost_usd NUMERIC(12,4) CHECK (additive_material_cost_usd >= 0),
    productivity_band            VARCHAR(20),
    productivity_band_order      SMALLINT,
    energy_eff_band              VARCHAR(20),
    energy_eff_band_order        SMALLINT,
```

```
    gold_batch_id               VARCHAR(36),
    gold_processed_at           TIMESTAMPTZ
);


COMMENT ON TABLE {PG_SCHEMA}.production    IS 'Parent table: 1 row = 1 production batch record';
COMMENT ON TABLE {PG_SCHEMA}.supply_chain IS 'Supplier quality and delivery delay (1:1 production)';
COMMENT ON TABLE {PG_SCHEMA}.quality       IS 'Defect rate, quality score and target is_high_defect (1:1 production)';
COMMENT ON TABLE {PG_SCHEMA}.maintenance  IS 'Maintenance hours and downtime (1:1 production)';
COMMENT ON TABLE {PG_SCHEMA}.inventory     IS 'Inventory turnover and stockout rate (1:1 production)';
COMMENT ON TABLE {PG_SCHEMA}.operations    IS 'Workforce, safety, energy and additive manufacturing (1:1 production)';
"""


conn = pg_connect()
try:
    conn.run("BEGIN")
    run_script(conn, DDL)
    conn.run("COMMIT")
finally:
    conn.close()
print("PostgreSQL DDL applied")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 9. Load Gold ⟶ PostgreSQL (COPY, parent ก่อน child)


# COMMAND ----------


import io, csv


def pg_columns(conn, table):
    rows = conn.run("""select column_name from information_schema.columns
                       where table_schema = :s and table_name = :t order by ordinal_position""",
                    s=PG_SCHEMA, t=table)
    return [r[0] for r in rows]


load_counts = {}
conn = pg_connect()
try:
    conn.run("BEGIN")                                  # โหลดทั้ง 6 ตารางใน transaction เดียว
    for t, _, _ in GOLD_TABLES:                        # production อยู่ลำดับแรก ⟶ FK ผ่าน
        cols = pg_columns(conn, t)
        rows = spark.table(fq(t)).select(*cols).collect()
        buf = io.StringIO()
        writer = csv.writer(buf)
        for r in rows:
            writer.writerow(["" if v is None else v for v in r])   # ช่องว่าง (ไม่ quote) = NULL ใน CSV COPY
        buf.seek(0)
        conn.run(f"COPY {PG_SCHEMA}.{t} ({', '.join(cols)}) FROM STDIN WITH (FORMAT csv)", stream=buf)
        load_counts[t] = len(rows)
        print(f"Loaded {t:<13} {len(rows):>6} rows")
    conn.run("COMMIT")
except Exception:
    conn.run("ROLLBACK")
    raise
finally:
    conn.close()


# COMMAND ----------


# MAGIC %md
# MAGIC ## 10. PostgreSQL Views สำหรับ BI (SQL ที่ใช้ตอบ Business Questions)
# MAGIC เงื่อนไข risk score / QC alert ใน view สร้างจาก threshold ชุดเดียวกับข้อ 6.0 ⟶ Delta และ PostgreSQL ให้ผลตรงกัน


# COMMAND ----------


ctrl_count_sql = " + ".join(f"({x})::int" for x in CTRL_SQL)
```

```
qc_any_sql     = " OR ".join(f"({x})" for x in QC_SQL) if QC_SQL else "FALSE"
all_count_sql  = " + ".join(f"({x})::int" for x in CTRL_SQL + QC_SQL)


VIEWS = f"""
CREATE OR REPLACE VIEW {PG_SCHEMA}.vw_batch_360 AS
SELECT p.production_id, p.production_volume, p.production_cost, p.cost_per_unit, p.volume_band,
       s.supplier_quality_pct, s.delivery_delay_days,
       q.defect_rate, q.quality_score_pct, q.is_high_defect, q.defect_rate_band, q.quality_band,
       m.maintenance_hours_per_week, m.downtime_pct, m.maintenance_band,
       i.inventory_turnover_ratio, i.stockout_rate,
       o.worker_productivity_pct, o.safety_incidents_count, o.energy_consumption_kwh, o.energy_efficiency_factor,
       ({ctrl_count_sql})                    AS controllable_risk_count,
       ({all_count_sql})                     AS risk_flag_count,
       ({qc_any_sql})::int                   AS qc_alert,
       (({ctrl_count_sql}) > 0 OR {qc_any_sql})::int AS label_rule_pred
FROM {PG_SCHEMA}.production p
JOIN {PG_SCHEMA}.supply_chain s USING (production_id)
JOIN {PG_SCHEMA}.quality      q USING (production_id)
JOIN {PG_SCHEMA}.maintenance  m USING (production_id)
JOIN {PG_SCHEMA}.inventory    i USING (production_id)
JOIN {PG_SCHEMA}.operations   o USING (production_id);


CREATE OR REPLACE VIEW {PG_SCHEMA}.vw_kpi_overview AS
SELECT COUNT(*)                                          AS total_batches,
       SUM(production_volume)                            AS total_units,
       ROUND(SUM(production_cost), 2)                    AS total_production_cost,
       ROUND(SUM(production_cost) / SUM(production_volume), 2) AS avg_cost_per_unit,
       ROUND(AVG(is_high_defect) * 100, 2)               AS high_defect_pct,
       ROUND(AVG(defect_rate), 3)                        AS avg_defect_rate,
       ROUND(AVG(quality_score_pct), 2)                  AS avg_quality_score,
       ROUND(AVG(maintenance_hours_per_week), 2)         AS avg_maintenance_hours,
       ROUND(AVG(downtime_pct), 2)                       AS avg_downtime_pct
FROM {PG_SCHEMA}.vw_batch_360;


CREATE OR REPLACE VIEW {PG_SCHEMA}.vw_defect_by_maintenance AS
SELECT m.maintenance_band, m.maintenance_band_order, COUNT(*) AS batches,
       ROUND(AVG(q.is_high_defect) * 100, 2) AS high_defect_pct
FROM {PG_SCHEMA}.maintenance m JOIN {PG_SCHEMA}.quality q USING (production_id)
GROUP BY 1, 2 ORDER BY 2;


CREATE OR REPLACE VIEW {PG_SCHEMA}.vw_risk_segment AS
SELECT CASE WHEN controllable_risk_count = 0 THEN 'Low' WHEN controllable_risk_count = 1 THEN 'Medium' ELSE 'High' END AS risk_segment,
       COUNT(*) AS batches,
       ROUND(AVG(is_high_defect) * 100, 2) AS high_defect_pct,
       ROUND(SUM(production_cost), 2) AS production_cost
FROM {PG_SCHEMA}.vw_batch_360
GROUP BY 1;


CREATE OR REPLACE VIEW {PG_SCHEMA}.vw_label_rule_check AS
SELECT label_rule_pred, is_high_defect, COUNT(*) AS batches,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS share_pct
FROM {PG_SCHEMA}.vw_batch_360
GROUP BY 1, 2;
"""
conn = pg_connect()
try:
    run_script(conn, VIEWS)
    print("Views created")
    for v in ["vw_kpi_overview", "vw_defect_by_maintenance", "vw_risk_segment", "vw_label_rule_check"]:
        rows = conn.run(f"select * from {PG_SCHEMA}.{v}")
        print(f"\n== {v}")
        print([c["name"] for c in conn.columns])
        for r in rows:
            print(r)
finally:
    conn.close()


# COMMAND ----------


# MAGIC %md
# MAGIC ## 11. PK / FK Integrity Test ใน PostgreSQL (บทที่ 9)
# MAGIC 1. ตรวจ constraint ที่มีจริงใน catalog · 2. orphan check ด้วย LEFT JOIN · 3. row count Delta = PostgreSQL · 4. **Negative tests**: ใส่ FK ที่ไม่มีจริง / PK ซ้ำ / target ผิด ⟶ ต้องถูก DB ปฏิเสธ
```

```
# COMMAND ----------


def expect_rejected(name, sql, sqlstates):
    """Negative test: DB ต้องปฏิเสธ statement นี้ด้วย SQLSTATE ที่คาดไว้"""
    c = pg_connect()
    try:
        c.run(sql)
        return (name, "rejected " + "/".join(sqlstates), "accepted", "FAIL")
    except DatabaseError as e:
        code = e.args[0].get("C") if e.args and isinstance(e.args[0], dict) else str(e)[:40]
        return (name, "rejected " + "/".join(sqlstates), f"SQLSTATE {code}", "PASS" if code in sqlstates else "FAIL")
    finally:
        c.close()


pg_tests = []
conn = pg_connect()
try:
    cons = conn.run("""
        SELECT tc.table_name, tc.constraint_type, tc.constraint_name
        FROM information_schema.table_constraints tc
        WHERE tc.table_schema = :s AND tc.constraint_type IN ('PRIMARY KEY','FOREIGN KEY','UNIQUE')
        ORDER BY 1, 2""", s=PG_SCHEMA)
    print("Constraints in PostgreSQL:")
    for c in cons:
        print("  ", tuple(c))


    for t, _, _ in GOLD_TABLES[1:]:
        orphan = conn.run(f"""SELECT COUNT(*) FROM {PG_SCHEMA}.{t} c
                              LEFT JOIN {PG_SCHEMA}.production p USING (production_id)
                              WHERE p.production_id IS NULL""")[0][0]
        pg_tests.append((f"orphan check {t}", "0 orphan", str(orphan), "PASS" if orphan == 0 else "FAIL"))


    # Reconciliation Delta vs PostgreSQL
    for t, _, _ in GOLD_TABLES:
        n_pg = conn.run(f"SELECT COUNT(*) FROM {PG_SCHEMA}.{t}")[0][0]
        n_dl = spark.table(fq(t)).count()
        pg_tests.append((f"row count {t}", str(n_dl), str(n_pg), "PASS" if n_pg == n_dl == silver_count else "FAIL"))
finally:
    conn.close()


# Negative tests (23503 = foreign_key_violation, 23505 = unique_violation, 23514 = check_violation)
pg_tests.append(expect_rejected("insert orphan FK (production_id=-999)",
    f"INSERT INTO {PG_SCHEMA}.quality (quality_id, production_id, defect_rate, is_high_defect) VALUES (-1, -999, 1.0, 1)", ["23503"]))
pg_tests.append(expect_rejected("insert duplicate PK (production_id=1)",
    f"INSERT INTO {PG_SCHEMA}.production (production_id, record_id, production_volume, production_cost) VALUES (1, repeat('x',64), 1, 1)", ["23505"]))
pg_tests.append(expect_rejected("insert invalid target is_high_defect=5",
    f"INSERT INTO {PG_SCHEMA}.quality (quality_id, production_id, defect_rate, is_high_defect) VALUES (-2, 1, 1.0, 5)", ["23514", "23505"]))


df_pg_tests = spark.createDataFrame(pg_tests, "test string, expected string, actual string, status string")
display(df_pg_tests)
if df_pg_tests.filter("status != 'PASS'").count():
    raise ValueError("PostgreSQL integrity tests failed")
print("PostgreSQL PK/FK integrity: PASS")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 12. ลงทะเบียน Lakebase เป็น Catalog ใน Unity Catalog
# MAGIC ทำให้ query ตาราง PostgreSQL ได้จาก SQL Editor / AI/BI Dashboard ด้วยชื่อ `manufacturing_gold_pg.gold.<table>`


# COMMAND ----------


try:
    w.postgres.get_catalog(name=f"catalogs/{UC_PG_CATALOG}")
    print("UC catalog already registered:", UC_PG_CATALOG)
except Exception:
    try:
```

```
        w.postgres.create_catalog(
            catalog=pgapi.Catalog(spec=pgapi.CatalogCatalogSpec(postgres_database=PG_DATABASE, branch=branch_name)),
            catalog_id=UC_PG_CATALOG).wait()
        print("Registered UC catalog:", UC_PG_CATALOG)
    except Exception as e:
        print("WARNING: register catalog failed (dashboard จะใช้ Delta gold แทน):", e)


try:
    display(spark.sql(f"SELECT * FROM {UC_PG_CATALOG}.{PG_SCHEMA}.vw_kpi_overview"))
except Exception as e:
    print("Query via UC catalog not available yet:", str(e)[:300])


# COMMAND ----------


# MAGIC %md
# MAGIC ## 13. Gold Run Log


# COMMAND ----------


log = spark.createDataFrame(
    [(gold_batch_id, t, int(load_counts.get(t, 0)), run_started_at, datetime.now(timezone.utc)) for t, _, _ in GOLD_TABLES],
    "gold_batch_id string, table_name string, pg_rows long, started_at timestamp, finished_at timestamp")
log.write.format("delta").mode("append").saveAsTable(fq("gold_run_log"))


print("Gold tables (Delta):")
display(spark.sql(f"SHOW TABLES IN {GOLD_SCHEMA}"))
print("Gold layer: PASS")
```

Appendix F: Dashboard Code และ Screenshots

Dashboard_Predictive.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Dashboard_Predictive — Business Insight & AI/BI Dashboard (จาก Gold layer)
# MAGIC
# MAGIC สร้าง/อัปเดต **Databricks AI/BI Dashboard** ด้วยโค้ด (reproducible) จากตาราง Gold:
# MAGIC - **Delta** `predictive_manufacring_defect.gold_predictive.*` : kpi_overview, mart_driver_impact, mart_driver_bands, mart_maint_hours_curve, mart_risk_matrix, mart_action_matrix, mart_label_rule
# MAGIC - **Lakebase PostgreSQL** (ผ่าน UC catalog `manufacturing_gold_pg.gold`) : `vw_kpi_overview` (counter จำนวน batch และ high-defect %) และ `vw_risk_segment` (กราฟ Q8)
# MAGIC
# MAGIC ## Business Questions
# MAGIC | # | คำถาม | ประเภทตัวแปรที่ใช้ตอบ |
# MAGIC |---|---|---|
# MAGIC | 1 | สถานการณ์คุณภาพโดยรวม และกระทบต้นทุนเท่าไร | KPI |
# MAGIC | 2 | ปัจจัยไหน **ที่ทีมปรับได้** สัมพันธ์กับ high-defect มากที่สุด | Controllable vs Outcome |
# MAGIC | 3 | ชั่วโมงซ่อมบำรุงมีผลอย่างไร จุดเปลี่ยนอยู่ตรงไหน | Controllable |
# MAGIC | 4 | Defect rate ระดับไหนควรเตือน (QC monitoring) | Outcome |
# MAGIC | 5 | Quality score ระดับไหนควรเตือน (QC monitoring) | Outcome |
# MAGIC | 6 | batch ใหญ่เสี่ยงกว่าหรือไม่ | Controllable |
# MAGIC | 7 | ปัจจัยไหน *ไม่* ช่วยอธิบาย defect | Controllable |
# MAGIC | 8 | มี operating window ที่ปลอดภัยหรือไม่ (ก่อนผลิต + หลังผลิต) | Controllable × Outcome |
# MAGIC | 9 | **Unknown Question:** label `is_high_defect` นิยามจากอะไร | ทุกตัว |
# MAGIC
# MAGIC กล่อง *Key insights* ดึงตัวเลขจากตาราง Gold ตอนรัน ⟶ ตัวเลขใน dashboard ตรงกับข้อมูลเสมอ


# COMMAND ----------


import json
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.dashboards import Dashboard


w = WorkspaceClient()
FOLDER    = "/Users/69056012@kmitl.ac.th/Predictive_Manufaturring/Predictive_Manu"
DASH_NAME = "Manufacturing Defect - Gold Insights"
G  = "predictive_manufacring_defect.gold_predictive"
PG = "manufacturing_gold_pg.gold"          # Lakebase PostgreSQL (UC-registered)


wh = next(x for x in w.warehouses.list())
WAREHOUSE_ID = wh.id
print("Warehouse:", wh.name, WAREHOUSE_ID)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. ดึงตัวเลขสำหรับกล่อง insight จาก Gold


# COMMAND ----------


kpi    = spark.table(f"{G}.kpi_overview").first().asDict()
scan   = {r["driver"]: r.asDict() for r in spark.table(f"{G}.mart_threshold_scan").collect()}
seg    = {r["risk_segment"]: r.asDict() for r in spark.table(f"{G}.mart_risk_segment").collect()}
action = {(r["risk_segment"], r["qc_alert"]): r.asDict() for r in spark.table(f"{G}.mart_action_matrix").collect()}


key_ctrl = sorted([v for v in scan.values() if v["driver_role"] == "Controllable" and v["is_key_factor"]], key=lambda v: -v["gap_pp"])
key_out  = sorted([v for v in scan.values() if v["driver_role"] == "Outcome" and v["is_key_factor"]], key=lambda v: -v["gap_pp"])
weak     = sorted(v["driver"] for v in scan.values() if v["driver_role"] == "Controllable" and not v["is_key_factor"])
max_weak = max(v["gap_pp"] for v in scan.values() if v["driver_role"] == "Controllable" and not v["is_key_factor"])


def rule(v):   return f"{v['driver']} {v['rule']}"
def side(v):   return f"{v['risky_side_high_defect_pct']:.1f}% vs {v['safe_side_high_defect_pct']:.1f}%"


low_pass  = action.get(("Low", 0), {})
low_alert = action.get(("Low", 1), {})
INSIGHTS = [
    "### Key insights & actions",
```

```
    f"1. **{kpi['high_defect_pct']}% ของ batch เป็น high-defect** และคิดเป็น {kpi['cost_share_in_high_defect_pct']}% ของต้นทุน ⟶ ตั้ง high-defect % เป็น KPI หลัก",
    "2. **ปัจจัยที่ทีมปรับได้ (Controllable)** ที่มีผลชัด: " + " · ".join(f"**{rule(v)}** ⟶ {side(v)}" for v in key_ctrl),
    "3. **Maintenance hours** มีจุดกระโดด ไม่ใช่ค่อย ๆ เพิ่ม ⟶ ทำ predictive maintenance / overhaul เครื่องที่ต้องซ่อมเกิน threshold ก่อน (ชั่วโมงซ่อมสูงอาจเป็น *อาการ* ของเครื่องไม่เสถียร ไม่ใช่สาเหตุ)",
    "4. **Batch ใหญ่** เสี่ยงขึ้นเฉพาะช่วงบนสุด แต่ต้นทุน/หน่วยต่ำกว่า ⟶ เพิ่ม QC sampling ใน batch ใหญ่แทนการลดขนาด",
    "5. **Defect rate / Quality score เป็นตัววัดผล (Outcome)** ไม่ใช่คันโยก ⟶ ใช้เป็น **QC alert**: " + " · ".join(f"{rule(v)}" for v in key_out),
    f"6. {', '.join(weak)} ⟶ ต่างกันไม่เกิน {max_weak:.1f} pp ⟶ **ไม่ใช่คันโยกหลัก**",
    f"7. **Operating window:** ไม่มี controllable flag + QC ผ่าน ({low_pass.get('batches', 0)} batch) ⟶ high-defect เพียง **{low_pass.get('high_defect_pct', 0)}%** · ไม่มี flag แต่ QC เตือน ⟶ {low_alert.get('high_defect_pct', 0)}%",
    f"8. **Unknown Question:** กฎ \"เกิน threshold ข้อใดข้อหนึ่ง\" ตรงกับ label **{kpi['label_rule_agreement_pct']}%** ⟶ `is_high_defect` น่าจะเป็น *label ที่สร้างจากกฎ* ไม่ใช่การนับของเสียจริง ต้องถามเจ้าของข้อมูลก่อนนำไปใช้ตัดสินใจ/ทำโมเดล",
]
for l in INSIGHTS:
    print(l)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. Datasets & Widgets


# COMMAND ----------


def ds(name, display, sql):
    return {"name": name, "displayName": display, "queryLines": [sql]}


DATASETS = [
    ds("ds_kpi",      "KPI overview (Delta)", f"SELECT * FROM {G}.kpi_overview"),
    ds("ds_kpi_pg",  "KPI overview (Lakebase PostgreSQL)",
       f"SELECT total_batches, CAST(high_defect_pct AS DOUBLE) AS high_defect_pct FROM {PG}.vw_kpi_overview"),
    ds("ds_impact",  "Driver impact (Controllable vs Outcome)",
       f"SELECT concat(driver, ' (', rule, ')') AS driver_rule, driver_role, gap_pp FROM {G}.mart_driver_impact"),
    ds("ds_curve",    "High-defect % by maintenance hour",
       f"SELECT concat(lpad(CAST(maintenance_hours_per_week AS STRING), 2, '0'), ' h') AS hours, batches, high_defect_pct FROM {G}.mart_maint_hours_curve"),
    ds("ds_volume",  "Production volume bands",
       f"SELECT concat(band_order, '. ', band) AS band_label, batches, high_defect_pct, avg_cost_per_unit FROM {G}.mart_driver_bands WHERE driver = 'Production volume'"),
    ds("ds_defect",  "Defect rate bands (outcome)",
       f"SELECT concat(band_order, '. ', band) AS band_label, batches, high_defect_pct FROM {G}.mart_driver_bands WHERE driver = 'Defect rate'"),
    ds("ds_quality", "Quality score bands (outcome)",
       f"SELECT concat(band_order, '. ', band) AS band_label, batches, high_defect_pct FROM {G}.mart_driver_bands WHERE driver = 'Quality score'"),
    ds("ds_segment_pg", "Controllable risk segment (Lakebase PostgreSQL)",
       f"SELECT CASE risk_segment WHEN 'Low' THEN '1. Low (0 flag)' WHEN 'Medium' THEN '2. Medium (1 flag)' ELSE '3. High (2 flags)' END AS risk_segment, "
       f"batches, CAST(high_defect_pct AS DOUBLE) AS high_defect_pct, CAST(production_cost AS DOUBLE) AS production_cost FROM {PG}.vw_risk_segment"),
    ds("ds_matrix",  "Risk matrix (maintenance × volume)",
       f"SELECT concat(maintenance_band_order, '. ', maintenance_band) AS maintenance_band, concat(volume_band_order, '. ', volume_band) AS volume_band, batches, high_defect_pct FROM {G}.mart_risk_matrix"),
    ds("ds_action",  "Action matrix (controllable segment × QC)",
       f"SELECT concat(risk_segment_order, '. ', risk_segment) AS risk_segment, qc_status, batches, high_defect_pct FROM {G}.mart_action_matrix"),
    ds("ds_label",    "Unknown question — label vs threshold rule",
       f"SELECT outcome, SUM(`count`) AS batches FROM {G}.mart_label_rule GROUP BY outcome"),
]


def q(dataset, fields):
    return [{"name": "main_query", "query": {"datasetName": dataset,
             "fields": [{"name": f, "expression": f"`{f}`"} for f in fields], "disaggregated": True}}]


def counter(name, dataset, field, title, x, y, w_=1, h=2):
    return {"widget": {"name": name, "queries": q(dataset, [field]),
                       "spec": {"version": 2, "widgetType": "counter",
                                "encodings": {"value": {"fieldName": field, "displayName": title}},
                                "frame": {"showTitle": True, "title": title}}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}


def bar(name, dataset, xf, yf, title, x, y, w_=3, h=6, xlabel=None, ylabel=None, horizontal=False, color=None):
    enc_cat = {"fieldName": xf, "scale": {"type": "categorical", "sort": {"by": "natural-order"}}, "displayName": xlabel or xf}
    enc_val = {"fieldName": yf, "scale": {"type": "quantitative"}, "displayName": ylabel or yf}
    if horizontal:                                  # เรียงแท่งจากมากไปน้อย
        enc_cat["scale"]["sort"] = {"by": "x-reversed"}
    enc = {"x": enc_val, "y": enc_cat} if horizontal else {"x": enc_cat, "y": enc_val}
    fields = [xf, yf]
    if color:
        enc["color"] = {"fieldName": color, "scale": {"type": "categorical"}, "displayName": color}
        fields.append(color)
    enc["label"] = {"show": True}
```

```
    return {"widget": {"name": name, "queries": q(dataset, fields),
                       "spec": {"version": 3, "widgetType": "bar", "encodings": enc,
                                "frame": {"showTitle": True, "title": title}}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}


def heatmap(name, dataset, xf, yf, cf, title, x, y, w_=3, h=7, xlabel=None, ylabel=None):
    return {"widget": {"name": name, "queries": q(dataset, [xf, yf, cf]),
                       "spec": {"version": 3, "widgetType": "heatmap",
                                "encodings": {"x": {"fieldName": xf, "scale": {"type": "categorical", "sort": {"by": "natural-order"}}, "displayName": xlabel or xf},
                                              "y": {"fieldName": yf, "scale": {"type": "categorical", "sort": {"by": "natural-order"}}, "displayName": ylabel or yf},
                                              "color": {"fieldName": cf, "scale": {"type": "quantitative", "colorRamp": {"mode": "scheme", "scheme": "yelloworangered"}}, "displayName": "High-defect %"},
                                              "label": {"show": True}},
                                "frame": {"showTitle": True, "title": title}}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}


def text(name, lines, x, y, w_=6, h=2):
    lines = [l if l.endswith("\n") else l + "\n" for l in lines]   # 1 element = 1 บรรทัด markdown
    return {"widget": {"name": name, "multilineTextboxSpec": {"lines": lines}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}


LAYOUT = [
    text("t_title", ["## Manufacturing Defect — Gold Layer Insights",
                     "Source: Bronze ⟶ Silver ⟶ **Gold (Delta + Lakebase PostgreSQL)** · target = `is_high_defect` · "
                     "**Controllable** = ปัจจัยที่ทีมปรับได้ · **Outcome** = ตัววัดผลคุณภาพ (ใช้เป็น QC alert)"], 0, 0, 6, 2),
    counter("c_batches",   "ds_kpi_pg", "total_batches",          "Total batches (PostgreSQL)",       0, 2),
    counter("c_high",      "ds_kpi_pg", "high_defect_pct",        "High-defect % (PostgreSQL)",       1, 2),
    counter("c_costshare", "ds_kpi",    "cost_share_in_high_defect_pct", "Cost in high-defect batches (%)",  2, 2),
    counter("c_lowrisk",   "ds_kpi",    "low_risk_batch_pct",     "No controllable flag (%)",         3, 2),
    counter("c_qc",        "ds_kpi",    "qc_alert_batch_pct",     "QC alert batches (%)",             4, 2),
    counter("c_label",     "ds_kpi",    "label_rule_agreement_pct",  "Label = threshold rule (%)",    5, 2),


    bar("b_impact", "ds_impact", "driver_rule", "gap_pp",
        "Q2/Q7 · Gap of high-defect % at best threshold (pp) — Controllable vs Outcome", 0, 4, 3, 7,
        xlabel="Driver (threshold)", ylabel="Gap (pp)", horizontal=True, color="driver_role"),
    bar("b_segment", "ds_segment_pg", "risk_segment", "high_defect_pct",
        "Q8 · High-defect % by controllable risk segment (from Lakebase PostgreSQL)", 3, 4, 3, 7,
        xlabel="Controllable risk segment", ylabel="High-defect %"),


    bar("b_curve", "ds_curve", "hours", "high_defect_pct", "Q3 · High-defect % by maintenance hours/week (step at threshold)", 0, 11, 3, 6,
        xlabel="Maintenance hours/week", ylabel="High-defect %"),
    bar("b_volume", "ds_volume", "band_label", "high_defect_pct", "Q6 · High-defect % by production volume", 3, 11, 3, 6,
        xlabel="Production volume", ylabel="High-defect %"),
    bar("b_defect", "ds_defect", "band_label", "high_defect_pct", "Q4 · QC monitoring (outcome) · High-defect % by defect rate", 0, 17, 3, 6,
        xlabel="Defect rate", ylabel="High-defect %"),
    bar("b_quality", "ds_quality", "band_label", "high_defect_pct", "Q5 · QC monitoring (outcome) · High-defect % by quality score", 3, 17, 3, 6,
        xlabel="Quality score", ylabel="High-defect %"),


    heatmap("h_matrix", "ds_matrix", "volume_band", "maintenance_band", "high_defect_pct",
            "Controllable risk matrix · maintenance hours × production volume", 0, 23, 3, 7,
            xlabel="Production volume", ylabel="Maintenance hours/week"),
    heatmap("h_action", "ds_action", "qc_status", "risk_segment", "high_defect_pct",
            "Q8 · Operating window · before production (segment) × after production (QC)", 3, 23, 3, 7,
            xlabel="QC status (outcome)", ylabel="Controllable risk segment"),


    bar("b_label", "ds_label", "outcome", "batches",
        "Q9 · Unknown Question — does a simple threshold rule reproduce is_high_defect?", 0, 30, 3, 7,
        xlabel="Rule vs label", ylabel="Batches", horizontal=True),
    text("t_insight", INSIGHTS, 3, 30, 3, 7),
]


SPEC = {"datasets": DATASETS, "pages": [{"name": "overview", "displayName": "Gold Insights", "layout": LAYOUT}]}
print("widgets:", len(LAYOUT), "datasets:", len(DATASETS))


# COMMAND ----------


# MAGIC %md
# MAGIC ## 3. Create หรือ Update (idempotent) แล้ว publish
```

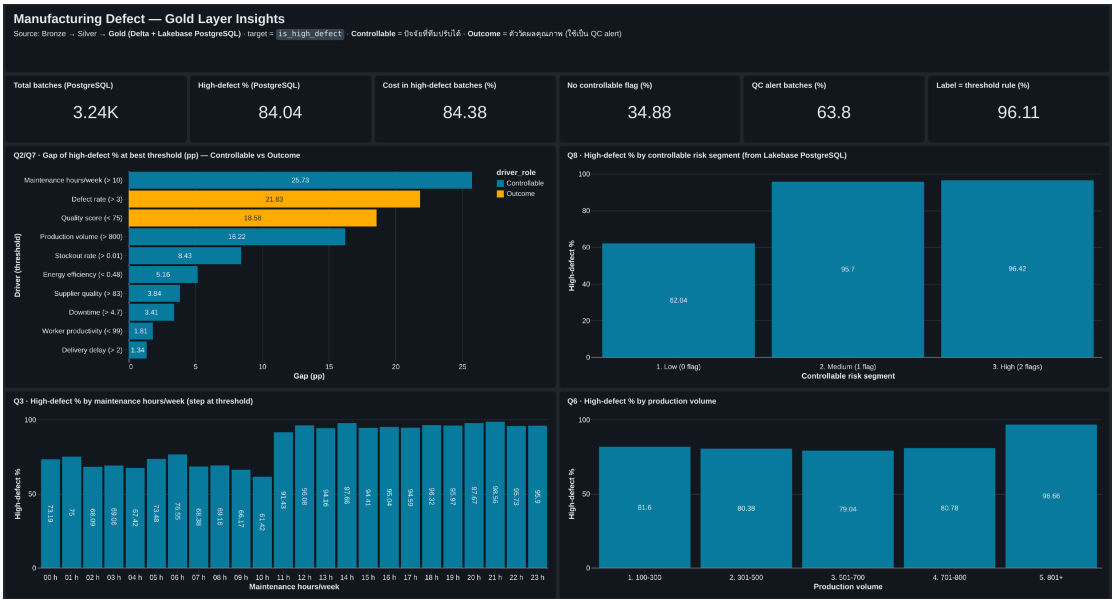
```
# COMMAND ----------


existing = [d for d in w.lakeview.list() if d.display_name == DASH_NAME]


# เก็บหน้าอื่นที่ notebook อื่นเพิ่มไว้ (เช่นหน้า ML จาก ML_Predictive) — แทนที่เฉพาะหน้า overview
if existing:
    cur = json.loads(w.lakeview.get(existing[0].dashboard_id).serialized_dashboard or "{}")
    other_pages = [pg for pg in cur.get("pages", []) if pg.get("name") != "overview"]
    used = {qq["query"]["datasetName"] for pg in other_pages for it in pg.get("layout", [])
            for qq in it.get("widget", {}).get("queries", []) if "query" in qq}
    my_ds = {d["name"] for d in DATASETS}
    extra_ds = [d for d in cur.get("datasets", []) if d["name"] in used and d["name"] not in my_ds]
    SPEC = {"datasets": DATASETS + extra_ds, "pages": SPEC["pages"] + other_pages}
    print("Kept other pages:", [pg.get("displayName") for pg in other_pages])


payload = Dashboard(display_name=DASH_NAME, parent_path=FOLDER, warehouse_id=WAREHOUSE_ID,
                    serialized_dashboard=json.dumps(SPEC))
if existing:
    dash = w.lakeview.update(dashboard_id=existing[0].dashboard_id,
                             dashboard=Dashboard(display_name=DASH_NAME, warehouse_id=WAREHOUSE_ID,
                                                 serialized_dashboard=json.dumps(SPEC)))
    print("Updated dashboard:", dash.dashboard_id)
else:
    dash = w.lakeview.create(dashboard=payload)
    print("Created dashboard:", dash.dashboard_id)


w.lakeview.publish(dashboard_id=dash.dashboard_id, warehouse_id=WAREHOUSE_ID, embed_credentials=True)
host = w.config.host.rstrip("/")
print("Dashboard URL:", f"{host}/dashboardsv3/{dash.dashboard_id}/published")
print("DASHBOARD_ID", dash.dashboard_id)
```



ภาพที่ 27: Dashboard (ส่วนบน)



ภาพที่ 28: Dashboard (ส่วนล่าง)

Appendix G: DQ Test Code

DQ_Test_Dirty_Data.py

```
# Databricks notebook source
# /// script
# [tool.databricks.environment]
# environment_version = "6"
# ///
# MAGIC %md
# MAGIC # DQ_Test_Dirty_Data — พิสูจน์ว่ากลไก Data Quality ทำงานจริง
# MAGIC
# MAGIC **ปัญหา:** ข้อมูลต้นทางสะอาด (0 placeholder, 0 แถวซ้ำ, 0 แถวเสีย) ⟶ ใน production run ไม่มีแถวไหนเข้า quarantine จึงยังพิสูจน์ไม่ได้ว่า rule ทำงานจริง
# MAGIC
# MAGIC **วิธีทดสอบ:**
# MAGIC 1. สร้างไฟล์ทดสอบ = แถวจริง 200 แถว (สะอาด) + **แถวสกปรกที่ฉีดเข้าไปเอง 15 แถว** ครอบคลุมทุก rule
# MAGIC 2. รัน **notebook จริง** `Bronze_Predictive_v2` และ `Silver_Predictive_v2` (โค้ดชุดเดียวกับ production) โดยส่ง parameter ให้เขียนลง schema `dq_test` ⟶ **ไม่ทับตารางจริง**
# MAGIC 3. ตรวจผลทีละ test case: คาดว่าเกิดอะไร ⟶ เกิดจริงอะไร ⟶ PASS/FAIL แล้วบันทึกลง `dq_test.test_results`
# MAGIC
# MAGIC | กลุ่ม | สิ่งที่ทดสอบ | ผลที่คาด |
# MAGIC |---|---|---|
# MAGIC | Bronze | แถว format เสีย (คอลัมน์เกิน / ขาด) | เก็บใน `_corrupt_record` ⟶ Silver ส่ง quarantine |
# MAGIC | Silver – Missing | placeholder `N/A`, ค่าว่าง ในคอลัมน์บังคับ | ⟶ NULL ⟶ quarantine (required) |
# MAGIC | Silver – Missing | placeholder ในคอลัมน์ไม่บังคับ | ⟶ NULL แต่ **ยังเป็น valid** (ไม่ตัดแถวเกินจำเป็น) |
# MAGIC | Silver – Type | ข้อความในช่องตัวเลข, ทศนิยมในช่อง INT | quarantine (cast error) |
# MAGIC | Silver – Range | %, สัดส่วน 0–1, ค่าติดลบ, ชั่วโมง > 168, target ไม่ใช่ 0/1 | quarantine |
# MAGIC | Silver – Dedup | แถวซ้ำทุกคอลัมน์ | เก็บ 1 แถว อีกแถว quarantine |
# MAGIC | Silver – Cleaning | ช่องว่างหน้า/หลังตัวเลข | trim แล้ว valid |
# MAGIC | Silver – Warning | defect_rate > 4.5 แต่ label = 0 | valid + ติดธง warning |


# COMMAND ----------


import csv, io, os
import pyspark.sql.functions as F


CATALOG     = "predictive_manufacring_defect"
TEST_SCHEMA = f"{CATALOG}.dq_test"
SOURCE_FILE = "/Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/manufacturing_defect_dataset.csv"
TEST_DIR    = "/Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/dq_test"
TEST_FILE   = f"{TEST_DIR}/manufacturing_defect_dirty.csv"
N_CLEAN     = 200


spark.sql(f"CREATE SCHEMA IF NOT EXISTS {TEST_SCHEMA}")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. สร้างไฟล์ข้อมูลสกปรก
# MAGIC แต่ละ test case ใช้แถวจริงคนละแถว (แถวที่ 201 เป็นต้นไป) แล้วแก้ค่า **1 จุด** ให้ผิดตามที่ต้องการทดสอบ


# COMMAND ----------


with open(SOURCE_FILE, encoding="utf-8-sig") as f:
    rows = list(csv.reader(f))
header, data = rows[0], rows[1:]
col = {c: i for i, c in enumerate(header)}


clean_rows = [r[:] for r in data[:N_CLEAN]]
pool = iter(data[N_CLEAN:])          # แถวจริงที่ยังไม่ถูกใช้ สำหรับทำ test case


def mutate(changes):
    r = next(pool)[:]
    for c, v in changes.items():
        r[col[c]] = v
    return r
```

```
# (case_id, คำอธิบาย, แถวที่จะเขียน, ผลที่คาด, ข้อความที่ต้องเจอ)
# expected: quarantine = ต้องอยู่ใน quarantine และ _dq_reason มีข้อความนี้
#           valid      = ต้องอยู่ใน Silver
#           warning    = ต้องอยู่ใน Silver และ _dq_warning_reason มีข้อความนี้
#           corrupt    = Bronze ต้องเก็บไว้ใน _corrupt_record
CASES = [
    ("T01", "placeholder 'N/A' ใน defect_rate (คอลัมน์บังคับ)",   mutate({"DefectRate": "N/A"}),          "quarantine", "defect_rate: required value is NULL"),
    ("T02", "ค่าว่างใน DefectStatus (target)",                    mutate({"DefectStatus": ""}),           "quarantine", "is_high_defect: required value is NULL"),
    ("T03", "ข้อความ 'abc' ใน ProductionCost",                    mutate({"ProductionCost": "abc"}),      "quarantine", "production_cost: invalid value for double"),
    ("T04", "ทศนิยม 12.5 ใน MaintenanceHours (INT)",              mutate({"MaintenanceHours": "12.5"}),   "quarantine", "maintenance_hours_per_week: non-integer value for int"),
    ("T05", "DefectStatus = 2 (ต้องเป็น 0/1)",                    mutate({"DefectStatus": "2"}),          "quarantine", "is_high_defect: value must be 0 or 1"),
    ("T06", "QualityScore = 150 (เกิน 100%)",                     mutate({"QualityScore": "150"}),        "quarantine", "quality_score_pct: value must be between 0 and 100"),
    ("T07", "StockoutRate = 5 (สัดส่วนต้องอยู่ 0–1)",             mutate({"StockoutRate": "5"}),          "quarantine", "stockout_rate: value must be between 0 and 1"),
    ("T08", "ProductionVolume = -10 (ติดลบ)",                     mutate({"ProductionVolume": "-10"}),    "quarantine", "production_volume: value cannot be negative"),
    ("T09", "MaintenanceHours = 200 (เกิน 168 ชม./สัปดาห์)",      mutate({"MaintenanceHours": "200"}),    "quarantine", "maintenance_hours_per_week: cannot exceed 168"),
    ("T10", "แถวซ้ำกับแถวสะอาดแถวแรกทุกคอลัมน์",                 clean_rows[0][:],                       "quarantine", "exact duplicate of another record"),
    ("T11", "placeholder 'N/A' ใน EnergyEfficiency (ไม่บังคับ)",  mutate({"EnergyEfficiency": "N/A"}),    "valid",      None),
    ("T12", "ช่องว่างหน้า/หลังตัวเลข ' 512 '",                    mutate({"ProductionVolume": " 512 "}),  "valid",      None),
    ("T13", "defect_rate 4.8 แต่ label = 0",                      mutate({"DefectRate": "4.8", "DefectStatus": "0"}), "warning", "defect_rate>4.5 but is_high_defect=0"),
]
# แถวที่ format เสีย (จำนวนคอลัมน์ไม่ตรง) — เขียนเป็นบรรทัดดิบ
row_extra = next(pool) + ["EXTRA_VALUE"]
row_short = next(pool)[:-3]
CORRUPT_CASES = [
    ("T14", "แถวมีคอลัมน์เกิน (18 ค่า)", row_extra, "corrupt", "malformed CSV row"),
    ("T15", "แถวมีคอลัมน์ขาด (14 ค่า)",  row_short, "corrupt", "malformed CSV row"),
]


buf = io.StringIO()
w = csv.writer(buf, lineterminator="\n")
w.writerow(header)
w.writerows(clean_rows)
w.writerows([c[2] for c in CASES])
w.writerows([c[2] for c in CORRUPT_CASES])


dbutils.fs.mkdirs(TEST_DIR)
dbutils.fs.put(TEST_FILE, buf.getvalue(), overwrite=True)


ALL_CASES = CASES + CORRUPT_CASES
n_lines = N_CLEAN + len(ALL_CASES)
print(f"Test file: {TEST_FILE}")
print(f"Data lines = {N_CLEAN} clean + {len(ALL_CASES)} test cases = {n_lines}")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. รัน Bronze และ Silver ตัวจริง ด้วย parameter ของชุดทดสอบ


# COMMAND ----------


PARAMS_BRONZE = {
    "source_file":  TEST_FILE,
    "bronze_table": f"{TEST_SCHEMA}.bronze_dirty",
    "ingest_log":   f"{TEST_SCHEMA}.ingestion_log",
}
PARAMS_SILVER = {
    "source_table":     f"{TEST_SCHEMA}.bronze_dirty",
    "silver_table":     f"{TEST_SCHEMA}.silver_dirty",
    "quarantine_table": f"{TEST_SCHEMA}.quarantine_dirty",
    "dq_run_log":       f"{TEST_SCHEMA}.dq_run_log",
}
NOTEBOOK_DIR = "/Users/69056012@kmitl.ac.th/Predictive_Manufaturring/Predictive_Manu"
TIMEOUT = 1800  # serverless sub-notebook cold start can take >15 min


print("Bronze ⟶", dbutils.notebook.run(f"{NOTEBOOK_DIR}/Bronze_Predictive_v2", TIMEOUT, PARAMS_BRONZE))
print("Silver ⟶", dbutils.notebook.run(f"{NOTEBOOK_DIR}/Silver_Predictive_v2", TIMEOUT, PARAMS_SILVER))


# COMMAND ----------


# MAGIC %md
```

```
# MAGIC ## 3. ตรวจผลทีละ test case


# COMMAND ----------


bronze = spark.table(PARAMS_BRONZE["bronze_table"])
silver = spark.table(PARAMS_SILVER["silver_table"])
quar   = spark.table(PARAMS_SILVER["quarantine_table"])


quar_reasons   = [r["_dq_reason"] for r in quar.select("_dq_reason").collect()]
warn_reasons   = [r["_dq_warning_reason"] for r in silver.select("_dq_warning_reason").collect() if r["_dq_warning_reason"]]
corrupt_bronze = bronze.filter(F.col("_corrupt_record").isNotNull()).count()


results = []
def add(case_id, desc, expected, actual, ok):
    results.append((case_id, desc, expected, actual, "PASS" if ok else "FAIL"))


for case_id, desc, _, expected, text in CASES:
    if expected == "quarantine":
        hit = sum(text in r for r in quar_reasons)
        add(case_id, desc, f"quarantine: {text}", f"พบใน quarantine {hit} แถว", hit == 1)


# T11: placeholder ในคอลัมน์ไม่บังคับ ⟶ NULL แต่ valid
n_null_eff = silver.filter(F.col("energy_efficiency_factor").isNull()).count()
add("T11", CASES[10][1], "valid, energy_efficiency_factor = NULL", f"Silver มี NULL {n_null_eff} แถว", n_null_eff == 1)


# T12: trim แล้วต้องได้ตัวเลข 512 และอยู่ใน Silver
n_512 = silver.filter(F.col("production_volume") == 512).count()
other_valid_rows = clean_rows + [CASES[10][2], CASES[12][2]]          # แถว valid อื่นที่อาจมี 512 อยู่แล้ว
n_512_other = sum(1 for r in other_valid_rows if r[col["ProductionVolume"]].strip() == "512")
add("T12", CASES[11][1], "valid, production_volume = 512", f"Silver มี 512 จาก test {n_512 - n_512_other} แถว", n_512 - n_512_other == 1)


# T13: warning
hit_warn = sum(CASES[12][4] in r for r in warn_reasons)
add("T13", CASES[12][1], "valid + warning", f"warning {hit_warn} แถว", hit_warn == 1)


# T14–T15: corrupt
hit_corrupt = sum("malformed CSV row" in r for r in quar_reasons)
add("T14–T15", "แถว format เสีย 2 แถว", "Bronze corrupt = 2, quarantine = 2",
    f"Bronze corrupt = {corrupt_bronze}, quarantine = {hit_corrupt}", corrupt_bronze == 2 and hit_corrupt == 2)


# ภาพรวม (reconciliation)
exp_valid = N_CLEAN + 3                       # 200 แถวสะอาด + T11, T12, T13
exp_quar  = len(ALL_CASES) - 3                # ที่เหลือทั้งหมดต้องเข้า quarantine
n_b, n_s, n_q = bronze.count(), silver.count(), quar.count()
add("R1", "Bronze rows = บรรทัดในไฟล์", str(n_lines), str(n_b), n_b == n_lines)
add("R2", "Silver valid rows", str(exp_valid), str(n_s), n_s == exp_valid)
add("R3", "Quarantine rows", str(exp_quar), str(n_q), n_q == exp_quar)
add("R4", "valid + quarantine = Bronze", str(n_b), str(n_s + n_q), n_s + n_q == n_b)


# lineage ต้องครบทุกแถว (รวม source_file_modified_at ที่เพิ่มใน v2.1)
lineage_null = silver.filter(F.col("source_file").isNull() | F.col("source_file_modified_at").isNull()
                             | F.col("bronze_batch_id").isNull() | F.col("silver_batch_id").isNull()).count()
add("R5", "Lineage ครบทุกแถวใน Silver", "0 แถวที่ lineage ว่าง", f"{lineage_null} แถว", lineage_null == 0)


df_results = spark.createDataFrame(results, "case_id string, test_case string, expected string, actual string, status string") \
                  .withColumn("tested_at", F.current_timestamp())
df_results.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{TEST_SCHEMA}.test_results")
display(df_results)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 4. หลักฐานสำหรับรายงาน: quarantine แยกตามเหตุผล + row trace
```

```
# COMMAND ----------


display(quar.groupBy("_dq_reason").count().orderBy("_dq_reason"))
display(spark.table(PARAMS_SILVER["dq_run_log"]).orderBy(F.desc("run_started_at")).limit(10))


n_fail = df_results.filter("status != 'PASS'").count()
print(f"Test cases: {df_results.count()} | FAIL: {n_fail}")
if n_fail:
    raise AssertionError("DQ test มีกรณีที่ไม่ผ่าน — ดูตาราง test_results")
print("DQ mechanism test: ALL PASS")


# COMMAND ----------


# MAGIC %md
# MAGIC ## (ไม่บังคับ) ล้างตารางทดสอบ
# MAGIC เอา `#` ออกแล้วรัน cell ด้านล่างเมื่อไม่ต้องการเก็บหลักฐานแล้ว


# COMMAND ----------


# spark.sql(f"DROP SCHEMA IF EXISTS {TEST_SCHEMA} CASCADE")
# dbutils.fs.rm(TEST_DIR, recurse=True)
```

Appendix H: Automation Code

Bronze_AutoLoader.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Bronze_AutoLoader — Incremental Bronze ด้วย Auto Loader (ใช้ใน Databricks Workflow)
# MAGIC
# MAGIC **แนวทาง C (แบบผสม):** Bronze รับเฉพาะ **ไฟล์ใหม่** ที่วางใน `landing/` แบบ append ⟶ Silver / Gold / Dashboard คำนวณใหม่จาก Bronze ทั้งหมด (ข้อมูลหลักพันแถว คำนวณใหม่ได้เร็วและผลตรงกับรายงานเสมอ)
# MAGIC
# MAGIC | หัวข้อ | Bronze_Predictive_v2 (batch) | Bronze_AutoLoader (incremental) |
# MAGIC |---|---|---|
# MAGIC | อ่านไฟล์ | อ่าน 1 ไฟล์ทั้งไฟล์ | Auto Loader (`cloudFiles`) อ่านเฉพาะไฟล์ใหม่ใน `landing/` (จำไฟล์ที่อ่านแล้วใน checkpoint) |
# MAGIC | เขียนตาราง | overwrite | **append** ลงตารางเดิม `bronze_preditive.manufacturing_defect_dataset_v2` ⟶ Silver/Gold/Alert ใช้ต่อได้ทันที |
# MAGIC | การ parse | ทุกคอลัมน์ STRING + `PERMISSIVE` + `_corrupt_record` | **เหมือนเดิมทุกอย่าง** (กลไก DQ ที่ทดสอบแล้วยังใช้ได้) |
# MAGIC | ตรวจ header | ก่อนโหลด | ตรวจ **ทีละไฟล์ใหม่** (schema drift ⟶ FAIL) |
# MAGIC | Reconciliation | ไฟล์ = ตาราง | **ทีละไฟล์**: บรรทัดในไฟล์ = แถวที่โหลดจากไฟล์นั้น |
# MAGIC | ส่งต่อ task ถัดไป | – | `dbutils.jobs.taskValues`: `new_rows`, `bronze_batch_id` ⟶ Job ใช้ตัดสินใจว่าจะรัน Silver ต่อหรือไม่ |
# MAGIC
# MAGIC # MAGIC ⚠️ ห้ามรัน `Bronze_Predictive_v2` ด้วยค่า default หลังเปิดใช้ Workflow (จะ overwrite ตาราง Bronze ที่ Auto Loader สะสมไว้) — ถ้าต้องการโหลดใหม่ทั้งหมด ให้ตั้ง widget `full_refresh = true`


# COMMAND ----------


# MAGIC %md
# MAGIC ## 0. Config (widgets)


# COMMAND ----------


import uuid
from datetime import datetime, timezone
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType


CATALOG       = "predictive_manufacring_defect"
BRONZE_SCHEMA = f"{CATALOG}.bronze_preditive"


dbutils.widgets.text("landing_dir",  "/Volumes/predictive_manufacring_defect/bronze_preditive/landing/")
dbutils.widgets.text("bronze_table", f"{BRONZE_SCHEMA}.manufacturing_defect_dataset_v2")
dbutils.widgets.text("ingest_log",   f"{BRONZE_SCHEMA}.ingestion_log")
dbutils.widgets.text("checkpoint",   "/Volumes/predictive_manufacring_defect/bronze_preditive/checkpoints/bronze_autoloader")
dbutils.widgets.dropdown("full_refresh", "false", ["false", "true"])


LANDING_DIR  = dbutils.widgets.get("landing_dir")
BRONZE_TABLE = dbutils.widgets.get("bronze_table")
INGEST_LOG   = dbutils.widgets.get("ingest_log")
CHECKPOINT   = dbutils.widgets.get("checkpoint")
FULL_REFRESH = dbutils.widgets.get("full_refresh") == "true"


EXPECTED_COLUMNS = [
    "ProductionVolume", "ProductionCost", "SupplierQuality", "DeliveryDelay",
    "DefectRate", "QualityScore", "MaintenanceHours", "DowntimePercentage",
    "InventoryTurnover", "StockoutRate", "WorkerProductivity", "SafetyIncidents",
    "EnergyConsumption", "EnergyEfficiency", "AdditiveProcessTime",
    "AdditiveMaterialCost", "DefectStatus",
]


spark.sql(f"CREATE VOLUME IF NOT EXISTS {BRONZE_SCHEMA}.landing")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {BRONZE_SCHEMA}.checkpoints")


bronze_batch_id = str(uuid.uuid4())
run_started_at  = datetime.now(timezone.utc)
print("Landing dir   :", LANDING_DIR)
print("Bronze table  :", BRONZE_TABLE)
print("Bronze batch id:", bronze_batch_id)


# COMMAND ----------
```

```
# MAGIC %md
# MAGIC ## 1. (ถ้าเลือก) Full refresh — ล้างตารางและ checkpoint เพื่ออ่านทุกไฟล์ใน landing ใหม่


# COMMAND ----------


if FULL_REFRESH:
    spark.sql(f"DROP TABLE IF EXISTS {BRONZE_TABLE}")
    dbutils.fs.rm(CHECKPOINT, recurse=True)
    print("Full refresh: dropped table and checkpoint")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. Extract — Auto Loader อ่านเฉพาะไฟล์ใหม่ (STRING ทั้งหมด + `_corrupt_record`)
# MAGIC `trigger(availableNow=True)` = อ่านไฟล์ใหม่ที่มีอยู่ตอนนี้ให้หมดแล้วหยุด (เหมาะกับ Job ที่ถูกปลุกเมื่อมีไฟล์มาใหม่)


# COMMAND ----------


raw_schema = StructType(
    [StructField(c, StringType(), True) for c in EXPECTED_COLUMNS]
    + [StructField("_corrupt_record", StringType(), True)]
)


stream = (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("mode", "PERMISSIVE")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .option("encoding", "UTF-8")
    .option("pathGlobFilter", "*.csv")
    .schema(raw_schema)
    .load(LANDING_DIR)
    .withColumn("_source_file",              F.col("_metadata.file_path"))
    .withColumn("_source_file_modified_at", F.col("_metadata.file_modification_time"))
    .withColumn("_ingested_at",              F.current_timestamp())
    .withColumn("_bronze_batch_id",          F.lit(bronze_batch_id))
)


q = (
    stream.writeStream.format("delta")
    .option("checkpointLocation", CHECKPOINT)
    .outputMode("append")
    .trigger(availableNow=True)
    .toTable(BRONZE_TABLE)
)
q.awaitTermination()
print("Auto Loader finished")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 3. ตรวจไฟล์ใหม่ทีละไฟล์: header (schema drift) + reconciliation


# COMMAND ----------


bronze_tbl  = spark.table(BRONZE_TABLE)
new_rows_df = bronze_tbl.filter(F.col("_bronze_batch_id") == bronze_batch_id)
new_rows    = new_rows_df.count()
files       = sorted(r["_source_file"] for r in new_rows_df.select("_source_file").distinct().collect())
print(f"New rows this run: {new_rows} | new files: {len(files)}")


log_rows, all_ok = [], True
for f in files:
```

```
    lines   = spark.read.text(f)
    header  = [c.strip() for c in lines.first()["value"].lstrip("").strip().split(",")]
    n_lines = lines.filter(F.length(F.trim("value")) > 0).count() - 1
    loaded  = new_rows_df.filter(F.col("_source_file") == f).count()
    corrupt = new_rows_df.filter((F.col("_source_file") == f) & F.col("_corrupt_record").isNotNull()).count()
    if header != EXPECTED_COLUMNS:
        status = "SCHEMA_DRIFT"
    elif n_lines != loaded:
        status = "FAIL"
    else:
        status = "PASS"
    all_ok &= status == "PASS"
    log_rows.append((bronze_batch_id, f, BRONZE_TABLE, int(n_lines), int(loaded), int(corrupt),
                     status, run_started_at, datetime.now(timezone.utc)))
    print(f"  {f.split('/')[-1]}: file={n_lines} loaded={loaded} corrupt={corrupt} ⟶ {status}")


if not log_rows:
    log_rows.append((bronze_batch_id, None, BRONZE_TABLE, 0, 0, 0, "NO_NEW_FILES",
                     run_started_at, datetime.now(timezone.utc)))


log_schema = """batch_id string, source_file string, target_table string,
                source_rows long, loaded_rows long, corrupt_rows long,
                status string, started_at timestamp, finished_at timestamp"""
spark.createDataFrame(log_rows, log_schema).write.format("delta").mode("append").saveAsTable(INGEST_LOG)
display(spark.table(INGEST_LOG).orderBy(F.desc("finished_at")).limit(10))


spark.sql(f"""COMMENT ON TABLE {BRONZE_TABLE} IS
'Bronze (raw, append-only via Auto Loader) of manufacturing defect CSV files in landing volume. All business columns STRING. Metadata: _source_file, _source_file_modified_at, _ingested_at, _bronze_batch_id.
Malformed rows kept in _corrupt_record.'""")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 4. ส่งค่าให้ task ถัดไป (Job) + Gate


# COMMAND ----------


total_rows = spark.table(BRONZE_TABLE).count()
print(f"Bronze total rows: {total_rows}")


dbutils.jobs.taskValues.set(key="new_rows", value=int(new_rows))
dbutils.jobs.taskValues.set(key="bronze_batch_id", value=bronze_batch_id)


if not all_ok:
    raise ValueError("Bronze AutoLoader: schema drift หรือ reconciliation ไม่ผ่าน — ดู ingestion_log")
print("Bronze AutoLoader: PASS")
dbutils.notebook.exit(f"PASS|new_rows={new_rows}|total={total_rows}")
```

Setup_Workflow.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Setup_Workflow — สร้าง / อัปเดต Databricks Workflow (Job) ด้วยโค้ด
# MAGIC
# MAGIC **v2 (23 ต.ค.):** dq_test รันขนานกับ bronze_autoloader (เดิมต่อกัน รวม ~22.5 นาที) · alert task retry 2 ครั้ง (กัน SQL Warehouse หยุดกลางคัน) · ค่าเริ่มต้น widget เป็น `false` ทั้งคู่ (อัปเดต Job อย่างเดียว ไม่ล้างข้อมูล ไม่สั่งรัน)
# MAGIC
# MAGIC รัน notebook นี้ครั้งเดียว (Run all) ⟶ ได้ Job **`Manufacturing_Defect_Pipeline`** ที่รันอัตโนมัติเมื่อมีไฟล์ CSV ใหม่ใน `landing/`
# MAGIC รันซ้ำได้ (idempotent): ถ้ามี Job ชื่อนี้อยู่แล้วจะอัปเดตค่าแทนการสร้างใหม่
# MAGIC
# MAGIC ```
# MAGIC  [file arrival: /Volumes/.../bronze_preditive/landing/]
# MAGIC            │
# MAGIC            ├──▶ dq_test (ทดสอบกลไก DQ) ──────────────────────────────────────┐
# MAGIC            └──▶ bronze_autoloader ──▶ has_new_data? ──(true)──┬──▶ silver ──▶ gold ──▶ dashboard ──▶ ml_predictive
# MAGIC              (Auto Loader)       (new_rows > 0)          (รอทั้งสองผ่าน)      │
# MAGIC                                                            ├──▶ alert_high_defect
# MAGIC                                                            └──▶ alert_quality_gate
# MAGIC  ถ้า task ใดล้มเหลว ⟶ อีเมลแจ้งเจ้าของ Job
```

```
# MAGIC ```
# MAGIC
# MAGIC | Task | Notebook / ประเภท | หน้าที่ |
# MAGIC |---|---|---|
# MAGIC | `dq_test` | DQ_Test_Dirty_Data | ทดสอบกลไก Data Quality ทุกรอบ (schema `dq_test`) **วันชนานกับ Bronze** เพื่อลดเวลา — Silver จะเริ่มเมื่อ DQ test ผ่านเท่านั้น |
# MAGIC | `bronze_autoloader` | Bronze_AutoLoader | รับเฉพาะไฟล์ใหม่จาก `landing/` แบบ append + reconciliation ทีละไฟล์ |
# MAGIC | `has_new_data` | Condition task | `new_rows > 0` ? ถ้าไม่มีแถวใหม่ ⟶ ข้าม task ที่เหลือ |
# MAGIC | `silver` ⟶ `gold` ⟶ `dashboard` | notebook v2.1 เดิม | คำนวณใหม่ทั้งชุดจาก Bronze, โหลด PostgreSQL, publish dashboard |
# MAGIC | `ml_predictive` | ML_Predictive | เทรนโมเดลใหม่จากข้อมูลล่าสุด |
# MAGIC | `alert_high_defect`, `alert_quality_gate` | SQL Alert เดิม 2 ตัว | ตรวจ KPI / quality gate หลัง Gold อัปเดต แล้วแจ้งเตือนทางอีเมล |


# COMMAND ----------


# MAGIC %md
# MAGIC ## 0. Config


# COMMAND ----------


import os, json
from databricks.sdk import WorkspaceClient


w = WorkspaceClient()


dbutils.widgets.dropdown("reset_bronze", "false", ["true", "false"])  # true เฉพาะครั้งแรก (ย้าย Bronze มาใช้ Auto Loader) — ค่าเริ่มต้น false กันล้างตารางโดยไม่ตั้งใจ
dbutils.widgets.dropdown("run_now", "false", ["true", "false"])


JOB_NAME     = "Manufacturing_Defect_Pipeline"
EMAIL        = w.current_user.me().user_name
NB_DIR       = os.path.dirname(dbutils.notebook.entry_point.getDbutils().notebook().getContext().notebookPath().get())


CATALOG      = "predictive_manufacring_defect"
SRC_FILE     = "/Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/manufacturing_defect_dataset.csv"
LANDING_DIR  = "/Volumes/predictive_manufacring_defect/bronze_preditive/landing/"
CHECKPOINT   = "/Volumes/predictive_manufacring_defect/bronze_preditive/checkpoints/bronze_autoloader"
BRONZE_TABLE = f"{CATALOG}.bronze_preditive.manufacturing_defect_dataset_v2"


ALERT_NAMES  = {"alert_high_defect": "High-defect % above target", "alert_quality_gate": "Quality gate failed"}


wh = next(x for x in w.warehouses.list())
WAREHOUSE_ID = wh.id
print("Notebook folder:", NB_DIR)
print("Owner / email  :", EMAIL)
print("Warehouse      :", wh.name, WAREHOUSE_ID)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. เตรียม landing volume + ไฟล์ batch แรก


# COMMAND ----------


spark.sql(f"CREATE VOLUME IF NOT EXISTS {CATALOG}.bronze_preditive.landing")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {CATALOG}.bronze_preditive.checkpoints")


existing = [f.name for f in dbutils.fs.ls(LANDING_DIR)]
if not any(n.endswith(".csv") for n in existing):
    dbutils.fs.cp(SRC_FILE, LANDING_DIR + "manufacturing_defect_batch_001.csv")
    print("Copied batch 001 into landing")
display(dbutils.fs.ls(LANDING_DIR))


# COMMAND ----------
```

```
# MAGIC %md
# MAGIC ## 2. ย้าย Bronze มาใช้ Auto Loader (ครั้งแรกเท่านั้น)
# MAGIC ตาราง Bronze เดิมถูกโหลดแบบ overwrite ⟶ ล้างตาราง + checkpoint เพื่อให้ Auto Loader สร้างใหม่จากไฟล์ใน `landing/` (ไม่ให้แถวซ้ำ)


# COMMAND ----------


if dbutils.widgets.get("reset_bronze") == "true":
    spark.sql(f"DROP TABLE IF EXISTS {BRONZE_TABLE}")
    dbutils.fs.rm(CHECKPOINT, recurse=True)
    print("Bronze table + checkpoint reset ⟶ Auto Loader จะอ่านทุกไฟล์ใน landing ในรอบแรก")
else:
    print("Skip reset")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 3. หา ID ของ SQL Alert เดิม 2 ตัว


# COMMAND ----------


def list_alerts():
    for path in ("/api/2.0/alerts", "/api/2.0/sql/alerts"):
        try:
            res = w.api_client.do("GET", path, query={"page_size": 100})
            items = res.get("alerts", res.get("results", res)) if isinstance(res, dict) else res
            if items:
                return items
        except Exception as e:
            print(f"list {path} failed: {str(e)[:120]}")
    return []


alerts = list_alerts()
ALERT_IDS = {}
for key, name in ALERT_NAMES.items():
    hit = next((a for a in alerts if a.get("display_name", a.get("name")) == name), None)
    if hit:
        ALERT_IDS[key] = hit["id"]
print("Alert IDs:", ALERT_IDS)


# COMMAND ----------


# MAGIC %md
# MAGIC ## 4. นิยาม Job (Jobs API 2.2)


# COMMAND ----------


def nb(task_key, notebook, depends=None, params=None, timeout=3600):
    t = {"task_key": task_key,
         "notebook_task": {"notebook_path": f"{NB_DIR}/{notebook}", "source": "WORKSPACE",
                           "base_parameters": params or {}},
         "timeout_seconds": timeout}
    if depends:
        t["depends_on"] = depends
    return t


CORE_TASKS = [
    nb("dq_test", "DQ_Test_Dirty_Data"),
    nb("bronze_autoloader", "Bronze_AutoLoader", None, {"full_refresh": "false"}),          # v2: รันขนานกับ dq_test
    {"task_key": "has_new_data",
     "depends_on": [{"task_key": "bronze_autoloader"}],
     "condition_task": {"op": "GREATER_THAN",
                        "left": "{{tasks.bronze_autoloader.values.new_rows}}", "right": "0"}},
    nb("silver", "Silver_Predictive_v2", [{"task_key": "has_new_data", "outcome": "true"},
                                          {"task_key": "dq_test"}]),                        # รอทั้งข้อมูลใหม่ + DQ test ผ่าน
    nb("gold", "Gold_Predictive", [{"task_key": "silver"}]),
```

```
    nb("dashboard", "Dashboard_Predictive", [{"task_key": "gold"}]),
    nb("ml_predictive", "ML_Predictive", [{"task_key": "dashboard"}]),
]


def alert_tasks(style):
    """style: 'alert_task' (SQL Alert รุ่นใหม่) หรือ 'sql_task' (รุ่นเดิม)"""
    out = []
    for key, aid in ALERT_IDS.items():
        if style == "alert_task":
            body = {"alert_task": {"alert_id": aid, "warehouse_id": WAREHOUSE_ID,
                                   "subscribers": [{"user_name": EMAIL}]}}
        else:
            body = {"sql_task": {"warehouse_id": WAREHOUSE_ID,
                                 "alert": {"alert_id": aid, "subscriptions": [{"user_name": EMAIL}]}}}
        # v2: retry — SQL Warehouse บน Free Edition อาจหยุดกลางคัน ("warehouse was stopped") ⟶ ลองใหม่อัตโนมัติ 2 ครั้ง
        out.append({"task_key": key, "depends_on": [{"task_key": "gold"}],
                    "max_retries": 2, "min_retry_interval_millis": 60000, "retry_on_timeout": True,
                    "timeout_seconds": 900, **body})
    return out


def settings(tasks, with_trigger=True):
    s = {
        "name": JOB_NAME,
        "description": "Manufacturing Defect Medallion pipeline: DQ test ⟶ Bronze (Auto Loader) ⟶ Silver ⟶ Gold (Delta + Lakebase) ⟶ Dashboard ⟶ ML + Alerts",
        "tasks": tasks,
        "max_concurrent_runs": 1,
        "queue": {"enabled": True},
        "email_notifications": {"on_failure": [EMAIL]},
        "tags": {"project": "manufacturing_defect", "layer": "medallion"},
    }
    if with_trigger:
        s["trigger"] = {"pause_status": "UNPAUSED",
                        "file_arrival": {"url": LANDING_DIR,
                                         "min_time_between_triggers_seconds": 60,
                                         "wait_after_last_change_seconds": 60}}
    return s


# COMMAND ----------


# MAGIC %md
# MAGIC ## 5. Create / Update Job
# MAGIC ลองแบบครบก่อน (file arrival + alert task) ถ้า API ไม่รองรับบางส่วนจะถอยไปแบบถัดไป และพิมพ์บอกว่าใช้แบบไหน


# COMMAND ----------


found = [j for j in w.jobs.list(name=JOB_NAME)]
job_id = found[0].job_id if found else None


def upsert(s):
    global job_id
    if job_id:
        w.api_client.do("POST", "/api/2.2/jobs/reset", body={"job_id": job_id, "new_settings": s})
    else:
        job_id = w.api_client.do("POST", "/api/2.2/jobs/create", body=s)["job_id"]
    return job_id


ATTEMPTS = [
    ("file arrival + alert_task", lambda: settings(CORE_TASKS + alert_tasks("alert_task"))),
    ("file arrival + sql_task alert", lambda: settings(CORE_TASKS + alert_tasks("sql_task"))),
    ("file arrival (ไม่มี alert task)", lambda: settings(CORE_TASKS)),
    ("ไม่มี trigger (Run now เอง)", lambda: settings(CORE_TASKS, with_trigger=False)),
]
used = None
for label, build in ATTEMPTS:
    try:
        upsert(build())
        used = label
        break
    except Exception as e:
        print(f"✗ {label}: {str(e)[:300]}")
if used is None:
    raise RuntimeError("สร้าง Job ไม่สำเร็จ — ดู error ด้านบน")
```

```
host = w.config.host.rstrip("/")
print(f"✓ Job ready ({used}) ⟶ {host}/jobs/{job_id}")


# COMMAND ----------


# MAGIC %md
# MAGIC ## 6. รันรอบแรก (โหลด batch 001 ผ่าน Auto Loader)


# COMMAND ----------


if dbutils.widgets.get("run_now") == "true":
    run = w.jobs.run_now(job_id=job_id)
    print(f"Started run {run.run_id} ⟶ {host}/jobs/{job_id}/runs/{run.run_id}")
    print("ดูความคืบหน้าในหน้า Jobs & Pipelines (ประมาณ 10–15 นาที)")
```

Demo_Drop_New_Batch.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Demo_Drop_New_Batch — จำลองไฟล์ batch ใหม่ เพื่อ demo Workflow
# MAGIC
# MAGIC รัน notebook นี้ระหว่างนำเสนอ ⟶ เขียนไฟล์ `manufacturing_defect_batch_002.csv` ลง `landing/` ⟶ **Job ถูกปลุกเองภายใน ~1–2 นาที** (file arrival trigger)
# MAGIC
# MAGIC ไฟล์นี้เป็น **ข้อมูลจำลองสำหรับ demo** (seed คงที่ ได้ไฟล์เดิมทุกครั้ง) สร้างจากแถวจริงแบบสุ่มแล้วขยับค่าเล็กน้อย (±2%) พร้อมใส่กรณีที่ pipeline ต้องจับได้:
# MAGIC
# MAGIC | ส่วนของไฟล์ | จำนวน | ผลที่คาด |
# MAGIC |---|---|---|
# MAGIC | แถวใหม่ (ค่าขยับจากแถวจริง) | 300 | เข้า Silver/Gold |
# MAGIC | แถวซ้ำกับ batch 001 ทุกคอลัมน์ (เช่น ส่งไฟล์ซ้ำบางส่วน) | 5 | quarantine: exact duplicate |
# MAGIC | แถวค่าผิด (QualityScore 150 / DefectStatus 2 / ProductionCost "abc") | 3 | quarantine ตาม rule |
# MAGIC
# MAGIC หลังรันจบ: Bronze 3,240 ⟶ **3,548** · Silver valid ⟶ **3,540** · quarantine ⟶ **8** (และ Alert "Quality gate failed" จะส่งอีเมลเพราะมีแถวเข้า quarantine)


# COMMAND ----------


import csv, io, random


SRC_FILE    = "/Volumes/predictive_manufacring_defect/bronze_preditive/bronze_predic/manufacturing_defect_dataset.csv"
LANDING_DIR = "/Volumes/predictive_manufacring_defect/bronze_preditive/landing/"
OUT_NAME    = "manufacturing_defect_batch_002.csv"


def make_batch(rows, seed=2026, n_new=300, n_dup=5):
    header, data = rows[0], rows[1:]
    col = {c: i for i, c in enumerate(header)}
    rnd = random.Random(seed)
    INT_COLS = {"ProductionVolume", "DeliveryDelay", "MaintenanceHours", "SafetyIncidents", "DefectStatus"}
    BOUNDS = {"ProductionVolume": (100, 999), "SupplierQuality": (80, 100), "DefectRate": (0.5, 5),
              "QualityScore": (60, 100), "DowntimePercentage": (0, 5), "StockoutRate": (0, 0.1),
              "WorkerProductivity": (80, 100), "EnergyEfficiency": (0.1, 0.5)}
    out = []
    for _ in range(n_new):
        r = rnd.choice(data)[:]
        for c, i in col.items():
            if c in ("DefectStatus", "DeliveryDelay", "MaintenanceHours", "SafetyIncidents"):
                continue                                  # คงค่าจำนวนเต็มเชิงหมวด/label เดิม
            v = float(r[i]) * (1 + rnd.uniform(-0.02, 0.02))
            lo, hi = BOUNDS.get(c, (0, float("inf")))
            v = min(max(v, lo), hi)
            r[i] = str(int(round(v))) if c in INT_COLS else repr(v)
        out.append(r)
    out += [rnd.choice(data)[:] for _ in range(n_dup)]          # แถวซ้ำกับ batch 001
    bad = [("QualityScore", "150"), ("DefectStatus", "2"), ("ProductionCost", "abc")]
    for c, v in bad:
        r = rnd.choice(data)[:]
        r[col[c]] = v
        out.append(r)
    return header, out
```

```
# COMMAND ----------


with open(SRC_FILE, encoding="utf-8-sig") as f:
    rows = list(csv.reader(f))
header, out = make_batch(rows)


buf = io.StringIO()
wr = csv.writer(buf, lineterminator="\n")
wr.writerow(header)
wr.writerows(out)
dbutils.fs.put(LANDING_DIR + OUT_NAME, buf.getvalue(), overwrite=False)
print(f"Wrote {len(out)} rows ⟶ {LANDING_DIR}{OUT_NAME}")
print("ไปที่ Jobs & Pipelines ⟶ Manufacturing_Defect_Pipeline เพื่อดู run ที่ถูก trigger")
display(dbutils.fs.ls(LANDING_DIR))
```

Reset_Demo.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Reset_Demo — รีเซ็ตกลับเป็นสถานะก่อน demo (batch 001 = 3,240 แถว)
# MAGIC
# MAGIC ใช้หลังซ้อม demo เพื่อให้ **ตัวเลขกลับมาตรงกับรายงาน** และวันนำเสนอสามารถรัน `Demo_Drop_New_Batch` ได้อีกครั้ง
# MAGIC
# MAGIC | ขั้น | ทำอะไร | ทำไม |
# MAGIC |---|---|---|
# MAGIC | 1 | **ย้าย** (ไม่ลบ) `manufacturing_defect_batch_002.csv` จาก `landing/` ไป `demo_archive/` | ให้ `landing/` เหลือแค่ batch 001 และเก็บไฟล์ไว้ตรวจย้อนได้ (การย้ายไฟล์ออกไม่ปลุก Job) |
# MAGIC | 2 | สั่งรัน Job 1 รอบด้วย `full_refresh = true` | Bronze_AutoLoader ล้างตาราง + checkpoint แล้วอ่านไฟล์ใน `landing/` ใหม่ทั้งหมด ⟶ Silver / Gold / PostgreSQL / Dashboard / ML คำนวณใหม่จาก 3,240 แถว |
# MAGIC | 3 | ตรวจจำนวนแถว (หลัง Job จบ) | ยืนยันว่า Bronze = Silver = 3,240 และ quarantine = 0 |
# MAGIC
# MAGIC กด **Run all ครั้งเดียว** เพื่อสั่ง reset ⟶ รอ Job จบ (ปกติ 15–40 นาที) ⟶ กลับมารัน **cell ข้อ 0 (Config) และข้อ 3** เพื่อตรวจผล (ไม่ต้อง Run all ซ้ำ)
# MAGIC
# MAGIC v4: (1) ถ้ามี run ของ Job ค้างอยู่ จะ **ไม่สั่งรันซ้ำ** (กันกด Run all 2 ครั้งแล้ว run ต่อคิวกัน) (2) notebook **ไม่นั่งรอ** Job เพราะ Free Edition จำกัดจำนวน serverless compute — notebook ที่รออยู่จะแย่งโควตา ทำให้ SQL Warehouse ของ alert เปิดไม่ได้ (RESOURCE_EXHAUSTED) และ task ช้าลง


# COMMAND ----------


# MAGIC %md
# MAGIC ## 0. Config


# COMMAND ----------


from databricks.sdk import WorkspaceClient


w = WorkspaceClient()


dbutils.widgets.dropdown("run_job", "true", ["true", "false"])


JOB_NAME     = "Manufacturing_Defect_Pipeline"
CATALOG      = "predictive_manufacring_defect"
LANDING_DIR  = "/Volumes/predictive_manufacring_defect/bronze_preditive/landing/"
ARCHIVE_DIR  = "/Volumes/predictive_manufacring_defect/bronze_preditive/landing_archive/"
DEMO_FILE    = "manufacturing_defect_batch_002.csv"
BRONZE_TABLE = f"{CATALOG}.bronze_preditive.manufacturing_defect_dataset_v2"
SILVER_TABLE = f"{CATALOG}.silver_predictive.manufacturing_defect_dataset_v2"
QUAR_TABLE   = f"{CATALOG}.silver_predictive.manufacturing_defect_dataset_quarantine_v2"


def task_states(run_id):
    r = w.jobs.get_run(run_id)
    for t in r.tasks or []:
        st = t.state.result_state or t.state.life_cycle_state
        print(f"  {t.task_key:<20} {st}")
    return r


def counts(label):
```

```
    b = spark.table(BRONZE_TABLE).count()
    s = spark.table(SILVER_TABLE).count()
    q = spark.table(QUAR_TABLE).count()
    print(f"{label:<8} Bronze={b:,}  Silver={s:,}  Quarantine={q:,}")
    return b, s, q


counts("ก่อน")
display(dbutils.fs.ls(LANDING_DIR))


# COMMAND ----------


# MAGIC %md
# MAGIC ## 1. ย้ายไฟล์ demo ออกจาก landing (เก็บไว้ใน landing_archive)
# MAGIC เก็บไว้นอก `landing/` (คนละ volume) เพื่อให้ Auto Loader ไม่อ่านไฟล์นี้อีก


# COMMAND ----------


from datetime import datetime


spark.sql(f"CREATE VOLUME IF NOT EXISTS {CATALOG}.bronze_preditive.landing_archive")


names = [f.name for f in dbutils.fs.ls(LANDING_DIR)]
if DEMO_FILE in names:
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    target = f"{ARCHIVE_DIR}{stamp}_{DEMO_FILE}"
    dbutils.fs.mv(LANDING_DIR + DEMO_FILE, target)
    print("Moved ⟶", target)
else:
    print("ไม่มีไฟล์ demo ใน landing (อาจรีเซ็ตไปแล้ว)")


display(dbutils.fs.ls(LANDING_DIR))


# COMMAND ----------


# MAGIC %md
# MAGIC ## 2. รัน Job 1 รอบแบบ full refresh
# MAGIC `notebook_params` ส่ง `full_refresh=true` ให้ task `bronze_autoloader` เฉพาะรอบนี้ (ค่าใน Job ยังเป็น `false` เหมือนเดิม)


# COMMAND ----------


job = next(iter(w.jobs.list(name=JOB_NAME)), None)
assert job, f"ไม่พบ Job {JOB_NAME} — รัน Setup_Workflow ก่อน"


host = w.config.host.rstrip("/")


# ถ้ามี run ค้างอยู่แล้ว (เช่น เคยกด Run all ไปแล้ว) ⟶ ไม่สั่งรันซ้ำ
active = list(w.jobs.list_runs(job_id=job.job_id, active_only=True))
if active:
    RUN_ID = active[0].run_id
    print(f"มี run กำลังทำงานอยู่แล้ว ⟶ ไม่สั่งรันซ้ำ (run {RUN_ID})")
elif dbutils.widgets.get("run_job") == "true":
    RUN_ID = w.jobs.run_now(job_id=job.job_id,
                            notebook_params={"full_refresh": "true"}).run_id
    print(f"Started reset run {RUN_ID}")
else:
    RUN_ID = None
    print("ข้ามการรัน Job (run_job = false)")


if RUN_ID:
    print(f"⟶ {host}/jobs/{job.job_id}/runs/{RUN_ID}")
    print("Job ใช้เวลาประมาณ 15–40 นาที — notebook นี้ **ไม่รอ** (v4) เพื่อไม่กินโควตา serverless ของ Free Edition")
    print("ระหว่างรอ: ปิด/Terminate compute ของ notebook นี้ได้เลย แล้วกลับมารัน cell ข้อ 0 (Config) + ข้อ 3 เมื่อ Job จบ")
```

```
# COMMAND ----------



# MAGIC %md
# MAGIC ## 3. ตรวจผลหลัง Job จบ
# MAGIC คาดว่า: Bronze = 3,240 · Silver = 3,240 · Quarantine = 0



# COMMAND ----------



job = next(iter(w.jobs.list(name=JOB_NAME)), None)
last = next(iter(w.jobs.list_runs(job_id=job.job_id, limit=1)), None)
if last:
    print(f"run ล่าสุด {last.run_id}: {last.state.life_cycle_state} {last.state.result_state or ''}")
    task_states(last.run_id)
    if last.state.life_cycle_state.value not in ("TERMINATED", "SKIPPED", "INTERNAL_ERROR"):
        print("⚠️ Job ยังรันอยู่ — ตัวเลขด้านล่างอาจยังไม่ใช่ค่าสุดท้าย รอให้ run จบแล้วรัน cell นี้ใหม่")
b, s, q = counts("หลัง")
if (b, s, q) == (3240, 3240, 0):
    print("Reset สำเร็จ: ตัวเลขตรงกับรายงานแล้ว และพร้อมรัน Demo_Drop_New_Batch อีกครั้ง")
else:
    print("ยังไม่ตรง — ถ้า Job ยังรันไม่จบให้รอแล้วรัน cell นี้ใหม่")
```

Appendix I: ML_Predictive Code

ML_Predictive.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # ML_Predictive — ทำนาย high-defect จาก Gold + MLflow
# MAGIC
# MAGIC **Input:** `gold_predictive.batch_360` · **Target:** `is_high_defect`
# MAGIC
# MAGIC | ขั้น | รายละเอียด |
# MAGIC |---|---|
# MAGIC | 1 | แบ่ง train/test 80/20 แบบ stratified (random_state=42) |
# MAGIC | 2 | Baseline: กฎ 4 risk flags (`risk_flag_count >= 1` ⟶ high defect) |
# MAGIC | 3 | Logistic Regression (StandardScaler + class_weight="balanced") |
# MAGIC | 4 | Gradient Boosting |
# MAGIC | 5 | วัดผล ROC-AUC, F1, Precision, Recall, Balanced accuracy + confusion matrix ⟶ log ลง MLflow |
# MAGIC | 6 | Register โมเดลที่ดีที่สุด (ROC-AUC) ใน Unity Catalog `gold_predictive.defect_model` |
# MAGIC | 7 | เขียน `predictions`, `model_comparison`, `model_feature_importance` และเพิ่มหน้า **Model** ใน dashboard |
# MAGIC
# MAGIC หมายเหตุ: target ไม่สมดุล (84% high defect) จึงไม่ใช้ accuracy อย่างเดียว


# COMMAND ----------


# MAGIC %pip install "mlflow>=2.14" scikit-learn --quiet


# COMMAND ----------


dbutils.library.restartPython()


# COMMAND ----------


import json
import numpy as np
import pandas as pd
import mlflow
from mlflow.models import infer_signature
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (roc_auc_score, f1_score, precision_score, recall_score,
                             balanced_accuracy_score, accuracy_score, confusion_matrix)
from databricks.sdk import WorkspaceClient


w = WorkspaceClient()
ME = w.current_user.me().user_name
C = "predictive_manufacring_defect"
G = f"{C}.gold_predictive"
MODEL_NAME = f"{G}.defect_model"
EXPERIMENT = f"/Users/{ME}/manufacturing_defect_model"


FEATURES = [
    "production_volume", "production_cost", "supplier_quality_pct", "delivery_delay_days",
    "defect_rate", "quality_score_pct", "maintenance_hours_per_week", "downtime_pct",
    "inventory_turnover_ratio", "stockout_rate", "worker_productivity_pct", "safety_incidents_count",
    "energy_consumption_kwh", "energy_efficiency_factor", "additive_process_time_hrs", "additive_material_cost_usd",
]
TARGET = "is_high_defect"


pdf = spark.table(f"{G}.batch_360").select("production_id", "risk_flag_count", "risk_segment", TARGET, *FEATURES).toPandas()
pdf[FEATURES] = pdf[FEATURES].astype("float64")
pdf[TARGET] = pdf[TARGET].astype(int)
print("rows:", len(pdf), "| high-defect %:", round(pdf[TARGET].mean() * 100, 2))


train_df, test_df = train_test_split(pdf, test_size=0.2, stratify=pdf[TARGET], random_state=42)
X_train, y_train = train_df[FEATURES], train_df[TARGET]
```

```
X_test, y_test = test_df[FEATURES], test_df[TARGET]
print("train:", len(train_df), "test:", len(test_df))


# COMMAND ----------


def evaluate(y_true, y_pred, y_score):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return {
        "roc_auc": float(roc_auc_score(y_true, y_score)),
        "f1": float(f1_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred)),
        "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
    }


mlflow.set_registry_uri("databricks-uc")
mlflow.set_experiment(EXPERIMENT)


results, models, run_ids, uris = {}, {}, {}, {}


# Baseline: 4 risk flags
with mlflow.start_run(run_name="baseline_4_flags") as run:
    y_pred = (test_df["risk_flag_count"] >= 1).astype(int)
    m = evaluate(y_test, y_pred, test_df["risk_flag_count"])
    mlflow.log_params({"model": "rule: risk_flag_count >= 1",
                       "flags": "maint>10, volume>800, defect_rate>3, quality<75"})
    mlflow.log_metrics({k: v for k, v in m.items()})
    results["Baseline (4 flags)"] = m; run_ids["Baseline (4 flags)"] = run.info.run_id


CANDIDATES = {
    "Logistic Regression": Pipeline([("scaler", StandardScaler()),
                                     ("clf", LogisticRegression(class_weight="balanced", max_iter=2000, random_state=42))]),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=200, learning_rate=0.05, max_depth=3, random_state=42),
}
for name, model in CANDIDATES.items():
    with mlflow.start_run(run_name=name.lower().replace(" ", "_")) as run:
        model.fit(X_train, y_train)
        proba = model.predict_proba(X_test)[:, 1]
        pred = (proba >= 0.5).astype(int)
        m = evaluate(y_test, pred, proba)
        mlflow.log_params({"model": name, "features": len(FEATURES), "test_size": 0.2, "random_state": 42,
                           **({"n_estimators": 200, "learning_rate": 0.05, "max_depth": 3} if "Boost" in name
                              else {"class_weight": "balanced", "max_iter": 2000})})
        mlflow.log_metrics(m)
        sig = infer_signature(X_train.head(50), model.predict_proba(X_train.head(50))[:, 1])
        info = mlflow.sklearn.log_model(model, artifact_path="model", signature=sig, input_example=X_train.head(5),
                                        serialization_format="cloudpickle")
        results[name] = m; models[name] = model; run_ids[name] = run.info.run_id; uris[name] = info.model_uri


cmp = pd.DataFrame(results).T.reset_index().rename(columns={"index": "model"})
display(cmp[["model", "roc_auc", "f1", "precision", "recall", "balanced_accuracy", "accuracy", "tn", "fp", "fn", "tp"]].round(4))


# COMMAND ----------


# เลือกโมเดลที่ดีที่สุดตาม ROC-AUC (เฉพาะโมเดล ML) แล้ว register ใน Unity Catalog
best = max(models, key=lambda n: results[n]["roc_auc"])
print("Best model:", best, "ROC-AUC =", round(results[best]["roc_auc"], 4))
reg = mlflow.register_model(uris[best], MODEL_NAME)
print("Registered:", MODEL_NAME, "version", reg.version)
try:
    from mlflow import MlflowClient
    MlflowClient().set_registered_model_alias(MODEL_NAME, "champion", reg.version)
    print("Alias champion ->", reg.version)
except Exception as e:
    print("alias skipped:", str(e)[:200])


# COMMAND ----------
```

```
# Feature importance
if best == "Gradient Boosting":
    imp = models[best].feature_importances_
else:
    imp = np.abs(models[best].named_steps["clf"].coef_[0])
fi = pd.DataFrame({"feature": FEATURES, "importance": imp}).sort_values("importance", ascending=False)
fi["model"] = best
gb_imp = pd.DataFrame({"feature": FEATURES, "importance": models["Gradient Boosting"].feature_importances_, "model": "Gradient Boosting"})
lr_imp = pd.DataFrame({"feature": FEATURES, "importance": np.abs(models["Logistic Regression"].named_steps["clf"].coef_[0]), "model": "Logistic Regression"})
fi_all = pd.concat([gb_imp, lr_imp]).sort_values(["model", "importance"], ascending=[True, False])
display(fi_all)



# Predictions ทุกแถว (บอกด้วยว่าอยู่ใน train หรือ test)
pdf["split"] = np.where(pdf["production_id"].isin(test_df["production_id"]), "test", "train")
pdf["prob_high_defect"] = models[best].predict_proba(pdf[FEATURES])[:, 1]
pdf["pred_high_defect"] = (pdf["prob_high_defect"] >= 0.5).astype(int)
pdf["baseline_pred"] = (pdf["risk_flag_count"] >= 1).astype(int)
pdf["model_name"] = best
pdf["model_version"] = int(reg.version)



pred_sdf = spark.createDataFrame(pdf[["production_id", "split", "is_high_defect", "prob_high_defect", "pred_high_defect",
                                      "baseline_pred", "risk_flag_count", "risk_segment", "model_name", "model_version"]])
pred_sdf.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{G}.predictions")



cmp["is_best"] = cmp["model"] == best
cmp["mlflow_run_id"] = cmp["model"].map(run_ids)
spark.createDataFrame(cmp).write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{G}.model_comparison")
spark.createDataFrame(fi_all).write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{G}.model_feature_importance")
print("Tables written: predictions, model_comparison, model_feature_importance")
display(spark.sql(f"""
SELECT risk_segment, COUNT(*) batches, ROUND(AVG(is_high_defect)*100,2) actual_high_pct,
       ROUND(AVG(prob_high_defect)*100,2) avg_pred_prob_pct
FROM {G}.predictions WHERE split='test' GROUP BY risk_segment ORDER BY risk_segment"""))



# COMMAND ----------



# MAGIC %md
# MAGIC ## เพิ่มหน้า "Model" ใน AI/BI Dashboard (idempotent — แทนที่หน้าเดิมถ้ามี)



# COMMAND ----------



from databricks.sdk.service.dashboards import Dashboard
DASH_NAME = "Manufacturing Defect - Gold Insights"
dash = next(d for d in w.lakeview.list() if d.display_name == DASH_NAME)
full = w.lakeview.get(dash.dashboard_id)
spec = json.loads(full.serialized_dashboard)
WAREHOUSE_ID = full.warehouse_id or next(x for x in w.warehouses.list()).id



def ds(name, display, sql): return {"name": name, "displayName": display, "queryLines": [sql]}
def q(dataset, fields):
    return [{"name": "main_query", "query": {"datasetName": dataset,
             "fields": [{"name": f, "expression": f"`{f}`"} for f in fields], "disaggregated": True}}]
def counter(name, dataset, field, title, x, y, w_=2, h=2):
    return {"widget": {"name": name, "queries": q(dataset, [field]),
            "spec": {"version": 2, "widgetType": "counter", "encodings": {"value": {"fieldName": field, "displayName": title}},
                     "frame": {"showTitle": True, "title": title}}}, "position": {"x": x, "y": y, "width": w_, "height": h}}
def bar(name, dataset, xf, yf, title, x, y, w_=3, h=6, horizontal=False, color=None, sort_value=False):
    cat = {"fieldName": xf, "scale": {"type": "categorical", "sort": {"by": ("x-reversed" if horizontal else "y-reversed") if sort_value else "natural-order"}}, "displayName": xf}
    val = {"fieldName": yf, "scale": {"type": "quantitative"}, "displayName": yf}
    enc = {"x": val, "y": cat} if horizontal else {"x": cat, "y": val}
    fields = [xf, yf]
    if color:
        enc["color"] = {"fieldName": color, "scale": {"type": "categorical"}, "displayName": color}; fields.append(color)
    enc["label"] = {"show": True}
    return {"widget": {"name": name, "queries": q(dataset, fields),
            "spec": {"version": 3, "widgetType": "bar", "encodings": enc, "frame": {"showTitle": True, "title": title}}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}
def text(name, lines, x, y, w_=6, h=2):
    return {"widget": {"name": name, "multilineTextboxSpec": {"lines": [l + "\n" for l in lines]}},
            "position": {"x": x, "y": y, "width": w_, "height": h}}
```

```
M_DATASETS = [
    ds("m_best", "Best model", f"SELECT model, ROUND(roc_auc,4) AS roc_auc, ROUND(f1,4) AS f1, ROUND(recall,4) AS recall FROM {G}.model_comparison WHERE is_best"),
    ds("m_cmp", "Model comparison", f"SELECT model, metric, ROUND(value,4) AS value FROM {G}.model_comparison "
        "UNPIVOT (value FOR metric IN (roc_auc, f1, precision, recall, balanced_accuracy))"),
    ds("m_fi", "Feature importance (best)", f"SELECT feature, ROUND(importance,4) AS importance FROM {G}.model_feature_importance "
        f"WHERE model = (SELECT model FROM {G}.model_comparison WHERE is_best) ORDER BY importance DESC LIMIT 10"),
    ds("m_seg", "Predicted vs actual by segment (test)", f"SELECT risk_segment, measure, pct FROM (SELECT risk_segment, "
        f"ROUND(AVG(is_high_defect)*100,2) AS actual_high_pct, ROUND(AVG(prob_high_defect)*100,2) AS predicted_prob_pct "
        f"FROM {G}.predictions WHERE split='test' GROUP BY risk_segment) UNPIVOT (pct FOR measure IN (actual_high_pct, predicted_prob_pct))"),
]
b = results[best]; base = results["Baseline (4 flags)"]
M_LAYOUT = [
    text("m_title", ["## Predictive model — high-defect",
                     f"Best: **{best}** (UC `{MODEL_NAME}` v{reg.version}) · test set {len(test_df)} batches · MLflow experiment `{EXPERIMENT}`"], 0, 0, 6, 2),
    counter("m_c_auc", "m_best", "roc_auc", "Best ROC-AUC", 0, 2),
    counter("m_c_f1", "m_best", "f1", "Best F1", 2, 2),
    counter("m_c_rec", "m_best", "recall", "Best recall", 4, 2),
    bar("m_b_cmp", "m_cmp", "metric", "value", "Model vs baseline (4 flags) on test set", 0, 4, 6, 7, color="model"),
    bar("m_b_fi", "m_fi", "feature", "importance", "Top 10 features (best model)", 0, 11, 3, 7, horizontal=True, sort_value=True),
    bar("m_b_seg", "m_seg", "risk_segment", "pct", "Actual high-defect % vs predicted probability by risk segment (test)", 3, 11, 3, 7, color="measure"),
    text("m_note", [
        "### อ่านผล",
        f"- Baseline 4 flags: ROC-AUC {base['roc_auc']:.3f}, F1 {base['f1']:.3f}, recall {base['recall']:.3f}",
        f"- {best}: ROC-AUC {b['roc_auc']:.3f}, F1 {b['f1']:.3f}, recall {b['recall']:.3f}",
        "- Target ไม่สมดุล (84% high) ดู ROC-AUC / balanced accuracy ประกอบ F1",
    ], 0, 18, 6, 3),
]
spec["datasets"] = [d for d in spec["datasets"] if not d["name"].startswith("m_")] + M_DATASETS
spec["pages"] = [p for p in spec["pages"] if p["name"] != "model"] + [{"name": "model", "displayName": "Model", "layout": M_LAYOUT}]
w.lakeview.update(dashboard_id=dash.dashboard_id, dashboard=Dashboard(display_name=DASH_NAME, warehouse_id=WAREHOUSE_ID,
                                                                      serialized_dashboard=json.dumps(spec)))
w.lakeview.publish(dashboard_id=dash.dashboard_id, warehouse_id=WAREHOUSE_ID, embed_credentials=True)
print("Dashboard updated with page 'Model'")


# COMMAND ----------


# ทดลองเพิ่ม: ใช้เฉพาะตัวแปรที่รู้ก่อนผลิต (ตัด Outcome ออก) เพื่อดู label leakage
OUTCOME = ["defect_rate", "quality_score_pct"]
PRE_FEATURES = [f for f in FEATURES if f not in OUTCOME]   # 14 ตัวแปร


gb_pre = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05, max_depth=3, random_state=42)
gb_pre.fit(train_df[PRE_FEATURES], y_train)


proba_pre = gb_pre.predict_proba(test_df[PRE_FEATURES])[:, 1]
pred_pre = (proba_pre >= 0.5).astype(int)
m_pre = evaluate(y_test, pred_pre, proba_pre)


with mlflow.start_run(run_name="gb_pre_production_only"):
    mlflow.log_params({"model": "Gradient Boosting", "features": len(PRE_FEATURES), "excluded": ",".join(OUTCOME)})
    mlflow.log_metrics(m_pre)


print("GB (pre-production features only)")
print(f"  ROC-AUC {m_pre['roc_auc']:.4f}  F1 {m_pre['f1']:.4f}  BalAcc {m_pre['balanced_accuracy']:.4f}  "
      f"CM tn={m_pre['tn']} fp={m_pre['fp']} fn={m_pre['fn']} tp={m_pre['tp']}")


# COMMAND ----------


print("ML_SUMMARY")
for k, v in results.items():
    print(f"  {k:<22} ROC-AUC {v['roc_auc']:.4f}  F1 {v['f1']:.4f}  P {v['precision']:.4f}  R {v['recall']:.4f}  BalAcc {v['balanced_accuracy']:.4f}  CM tn={v['tn']} fp={v['fp']} fn={v['fn']} tp={v['tp']}")
print("  best:", best, "| UC model:", MODEL_NAME, "v", reg.version)
print("  top features:", ", ".join(fi["feature"].head(5)))
```